

**МИНОБРНАУКИ РОССИИ**  
**федеральное государственное бюджетное**  
**образовательное учреждение высшего образования**  
**«Череповецкий государственный университет»**

|                                        |                                                                                 |
|----------------------------------------|---------------------------------------------------------------------------------|
| Институт (факультет)                   | Информационных технологий                                                       |
| Направление подготовки (специальность) | 09.04.04 Программная инженерия<br>(Разработка программно-информационных систем) |
| Выпускающая кафедра                    | Математического и программного обеспечения ЭВМ                                  |

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА**

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| Название работы | Программное обеспечение системы контроля перемещения объектов мостовыми кранами |
|-----------------|---------------------------------------------------------------------------------|

|          |                                      |
|----------|--------------------------------------|
| Студента | Адамова Даниила Сергеевича<br>Ф.И.О. |
|----------|--------------------------------------|

**ДОПУСТИТЬ К ЗАЩИТЕ**

|                                                |                  |
|------------------------------------------------|------------------|
| Директор института (декан факультета)          | Ершов Е.В.       |
| Заведующий выпускающей кафедрой                | Ершов Е.В.       |
| Руководитель выпускной квалификационной работы | Ершов Е.В.       |
| Нормоконтролер                                 | Виноградова Л.Н. |
| Выпускник                                      | Адамов Д. С.     |

Череповец  
2023 г.

## Содержание

|                                                                                                                      |    |
|----------------------------------------------------------------------------------------------------------------------|----|
| Введение.....                                                                                                        | 6  |
| 1 Характеристика проблемы контроля перемещения объектов мостовыми кранами.....                                       | 10 |
| 1.1 Анализ существующих методов и аппаратно-программных средств контроля перемещения объектов мостовыми кранами..... | 10 |
| 1.2 Описание перемещения объектов мостовыми кранами как объекта оптического контроля .....                           | 13 |
| 1.2.1 Определение наличия объекта в захвате мостового крана.....                                                     | 13 |
| 1.2.2 Определение текущего местоположения объекта .....                                                              | 16 |
| 1.3 Постановка задачи, разработка требований к системе контроля перемещения объектов мостовыми кранами.....          | 19 |
| 1.3.1 Построение дерева проблем.....                                                                                 | 19 |
| 1.3.2 Построение дерева целей .....                                                                                  | 20 |
| 1.3.3 Построение дерева задач .....                                                                                  | 22 |
| 1.3.4 Требования к математическому обеспечению .....                                                                 | 23 |
| 1.3.5 Требования к информационному обеспечению.....                                                                  | 24 |
| 1.3.6 Требования к аппаратно-программному обеспечению.....                                                           | 24 |
| 1.3.7 Требования к информационной и программной совместимости.....                                                   | 25 |
| 1.4 Выводы по разделу.....                                                                                           | 25 |
| 2 Математическое и алгоритмическое обеспечение системы контроля перемещения сталь-ковшей мостовыми кранами.....      | 27 |
| 2.1 Метод позиционирования мостового крана .....                                                                     | 27 |
| 2.2 Алгоритм определения состояния сталь-ковша в захвате крана .....                                                 | 34 |
| 2.3 Алгоритм определения местоположения по наблюдаемой сцене .....                                                   | 42 |

|                                                                                                                                                |    |
|------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.4 Алгоритм локализации объектов на крановом пути .....                                                                                       | 43 |
| 2.5 Алгоритм анализа состояния мостового крана .....                                                                                           | 45 |
| 2.6 Выводы по разделу .....                                                                                                                    | 46 |
| 3 Проектирование информационного и программного обеспечения системы<br>контроля перемещения объектов мостовыми кранами .....                   | 48 |
| 3.1 Концептуальное моделирование программного обеспечения .....                                                                                | 48 |
| 3.1.1 Модель «Черного ящика» .....                                                                                                             | 48 |
| 3.1.2 Модель состава .....                                                                                                                     | 50 |
| 3.1.3 Модель структуры .....                                                                                                                   | 52 |
| 3.1.4 Модель «белого ящика» .....                                                                                                              | 55 |
| 3.2 Проектирование информационного обеспечения .....                                                                                           | 56 |
| 3.2.1 Моделирование потоков информации в проектируемой системе .....                                                                           | 56 |
| 3.3 Проектирование программного обеспечения .....                                                                                              | 58 |
| 3.3.1 Выбор технологии проектирования, среды и языка программирования .....                                                                    | 58 |
| 3.3.2 Разработка логической модели .....                                                                                                       | 64 |
| 3.3.3 Разработка физической модели .....                                                                                                       | 89 |
| 3.3.4 Разработка интерфейсов .....                                                                                                             | 92 |
| 3.4 Выводы по разделу 3 .....                                                                                                                  | 93 |
| 4 Результаты экспериментальной проверки алгоритмического обеспечения<br>системы контроля перемещения сталь-ковшей мостовыми кранами .....      | 94 |
| 4.1 Описание эксперимента и экспериментальных данных проверки<br>алгоритмов позиционирования, обнаружения и классификации сталь-ковша<br>..... | 94 |
| 4.2 Описание методики настройки алгоритмического и программного<br>обеспечения системы контроля перемещения объектов мостовыми кранами .....   | 97 |

|                                                                                                                                                    |     |
|----------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 4.2.1 Настройка алгоритма позиционирования по маркерам .....                                                                                       | 97  |
| 4.2.2 Настройка алгоритма определения состояния сталь-ковша в захвате мостового крана.....                                                         | 99  |
| 4.2.3 Настройка алгоритма определения начала движения .....                                                                                        | 99  |
| 4.2.4 Настройка алгоритма определения действия захвата .....                                                                                       | 100 |
| 4.2.5 Настройка сервиса агрегации состояний кранов цеха .....                                                                                      | 101 |
| 4.2.6 Настройка уведомлений о проблемах с видеопотоком.....                                                                                        | 102 |
| 4.2.7 Настройка сервиса Nginx.....                                                                                                                 | 103 |
| 4.2.8 Настройка задач сервиса Nomad.....                                                                                                           | 104 |
| 4.3 Описание результатов проверки алгоритмов системы контроля перемещения объектов мостовыми кранами.....                                          | 104 |
| 4.3.1 Результаты экспериментальной проверки метода определения местоположения по маркерам.....                                                     | 104 |
| 4.3.2 Результаты экспериментальной проверки алгоритма определения состояния захвата крана .....                                                    | 105 |
| 4.3.3 Результаты экспериментальной проверки алгоритма определения текущего действия захвата крана .....                                            | 106 |
| 4.3.4 Результаты экспериментальной проверки алгоритма определения наличия движения крана .....                                                     | 107 |
| 4.3.5 Результаты экспериментальной проверки алгоритма определения местоположения по наблюдаемой сцене .....                                        | 108 |
| 4.4 Результаты экспериментальной проверки модулей и классов программного обеспечения системы контроля перемещения объектов мостовыми кранами ..... | 109 |
| 4.5 Перспективы применения предложенных технических решений .....                                                                                  | 113 |
| 4.6 Выводы по разделу 4.....                                                                                                                       | 114 |

|                                                                               |     |
|-------------------------------------------------------------------------------|-----|
| Заключение .....                                                              | 115 |
| Список литературы .....                                                       | 118 |
| Приложение 1. Устав проекта.....                                              | 123 |
| Приложение 2. Текст программы .....                                           | 137 |
| Приложение 3. Спецификация.....                                               | 152 |
| Приложение 4. Руководство пользователя .....                                  | 161 |
| Приложение 5. Пример конфигурации алгоритма позиционирования по маркерам..... | 168 |
| Приложение 6. Пример конфигурации сервиса исполнения нейронных сетей .....    | 169 |
| Приложение 7. Шаблон конфигурации Nginx .....                                 | 170 |
| Приложение 8. Пример рабочей нагрузки оркестратора Nomad .....                | 171 |
| Приложение 9. Акт внедрения.....                                              | 173 |

## Введение

**Актуальность темы.** В настоящее время, многие металлургические предприятия используют системы производственного планирования для повышения эффективности процесса производства стали.

В процессе обработки стали, её перемещают между агрегатами цеха в сталь-ковшах.

Отклонение реального процесса обработки от нормативного, приводит к снижению температуры в сталь-ковше. При достижении определённой температуры, его необходимо отправлять на дополнительный нагрев.

Он происходит при помощи опускания в сталь-ковш электродов и включения тока. Таких стандов в цехе ограниченное количество.

При плотном производственном графике, внеплановый дополнительный нагрев означает распространение волны опозданий на последующие операции. Кроме того, электроды довольно дорогие, что означает рост себестоимости конечного продукта.

Эти проблемы можно решить с помощью построения системой производственного планирования оптимального плана перемещения сталь-ковшей в цехе. Одним из возможных способов перемещения сталь-ковша является перемещение с помощью мостового крана.

На сегодняшний день, для сталеплавильного производства нет решения для получения местоположения сталь-ковша на крановом пути.

Например, в сфере транспортной логистики задача отслеживания переносимых грузов мостовым краном зачастую решается с помощью радиометок. В сталеплавильном производстве их применение не представляется возможным как из-за высоких температур при разливке стали, так и из-за радиопомех.

Существуют технические средства и научные труды, описывающие способ получения информации о наличии в захвате крана сталь-ковша или получения его местоположения.

Так, например, Трофименко Я.М. и Ершов Е.В. предложили модель для идентификации сталь-ковшей в сталеплавильном производстве [1].

Также существует множество моделей мостовых кранов различных производителей. Некоторые модели поддерживают сбор данных средствами АСУ ТП, например, вес объекта в захвате крана.

Основная часть грузоподъемных кранов введена в эксплуатацию в советское время. Их техническое состояние не вполне способно удовлетворять растущие потребности производства. При этом, ввиду непрерывного производства, модернизация мостового крана, подразумевающая демонтаж или длительный простой, недопустима.

Вышеперечисленные проблемы возможно решить средствами технического зрения.

Универсальные методы технического зрения позволяют выполнять неразрушающий производственный контроль. Эти системы используют цифровые интеллектуальные камеры, а также программное обеспечение, обрабатывающее изображение. В металлургии техническое зрение нашло широкое применение для контроля дефектов выпускаемой продукции.

Таким образом, актуальность темы заключается в том, что необходимо создать такое программное обеспечение, которое будет определять местоположение сталь-ковша на крановом пути без необходимости проведения модернизации рельсовых путей крана, цепей управления, и внесения изменений в конструкцию грузоподъемного механизма.

**Цель работы.** Целью настоящей работы является повышение оперативности локализации сталь-ковшей на крановом пути и определение их состояния в захвате мостового крана.

Для достижения поставленной цели решены следующие задачи:

- выполнен анализ предметной области контроля перемещения объектов мостовыми кранами;

- выполнено теоретическое обоснование задачи контроля перемещения объектов мостовыми кранами;
- выполнено проектирование программного и информационного обеспечения системы контроля перемещения объектов мостовыми кранами;
- проведена экспериментальная проверка программного обеспечения.

**Объект исследования.** Объектом исследования являются оптико-электронные системы слежения за объектами металлургического производства.

**Предмет исследования.** Предметом исследования настоящей работы является математическое, алгоритмическое, программное обеспечение системы контроля перемещения объектов мостовыми кранами.

**Методы исследования.** Достоверность полученных результатов подтверждается применяемыми методами исследования: анализ, математическое моделирование, абстрагирование, эксперимент, машинное обучение, а также алгоритмы компьютерной обработки изображений, основы теории построения алгоритмов, программные и языковые средства современных компьютерных технологий.

**Научная новизна.** Научная новизна результатов работы заключается в:

1. Разработан метод позиционирования, отличающийся использованием маркеров на нескольких кадрах, что позволяет выполнить локализацию мостового крана в условиях перекрытия одной или нескольких меток позиционирования.

2. Разработан алгоритм локализации объектов на крановом пути, отличающийся возможностью определения текущего состояния сталь-ковша в захвате мостового крана.

**Практическая значимость** состоит в разработке программного обеспечения, методик настройки, предоставлении высокоуровневым системам возможности получения состояния сталь-ковша, повышении оперативности локализации сталь-ковша в сталеплавильном производстве.



**Апробация результатов.** Результаты диссертационной работы докладывались на студенческих научных конференциях: ЧГУ СНК-2022, СНК «Регион искусственного интеллекта» 2022, ЧГУ СНК-2023. Также по теме опубликована 14.11.2022 научная статья на тему «Контроль перемещения сталь-ковшей мостовыми кранами на основе методов машинного зрения» в рецензируемом научном журнале «Вестник Череповецкого государственного университета».

Структура и объем магистерской диссертации. Диссертация состоит из введения, четырех глав, заключения и приложения. Содержит 173 страницы машинописного текста, включая 50 страниц приложений, 58 рисунков, 32 таблицы и 5 формул.

## 1 Характеристика проблемы контроля перемещения объектов мостовыми кранами

### 1.1 Анализ существующих методов и аппаратно-программных средств контроля перемещения объектов мостовыми кранами

Задачу контроля перемещения объектов мостовыми кранами можно разделить на две части:

1. Определение местоположения объекта на крановом пути.
2. Определение наличия объекта в захвате мостового крана.

Если рассматривать такие технологии глобального позиционирования как GPS, ГЛОНАСС, сотовые сети, то можно сказать, что определение местоположения с их помощью осложняется помехами, возникающими из-за наличия стен и межэтажных перекрытий, а также электромагнитным шумом, вызываемым производственными процессами [2]. Ввиду этих факторов увеличивается погрешность измерений до десятков, сотен метров.

Существуют и другие способы определения местоположения мостового крана:

- определение координат с помощью перевода сопротивления участка рельсы в величину длины [3];
- определение местоположения с помощью магнитострикционного преобразователя [4];
- определение позиции с помощью энкодера на двигателе мостового крана [5];
- определение позиции с помощью кодового рельса и фотоэлектрических датчиков [6].

Перечисленные выше методы не подходят ввиду необходимости модернизации мостового крана, и последующего внесения изменений в паспорт грузоподъёмного механизма. Что в конечном итоге приведёт к повторному прохождению процедуры пуска подъёмного средства в работу и

длительному простоем оборудования [7]. Без учёта длительности модернизации мостового крана, время на внесение изменений в сведения, содержащихся в реестре Ростехнадзора может достигать 20 рабочих дней [8].

Ещё один способ – радарные системы. Эти системы предназначены для точного 2D позиционирования кранов и любых типов транспортных средств. Такие системы определяют местоположение с помощью метода триангуляции. Для этого используются приёмопередающие устройства, которые служат локальными стационарными опорными точками. Они подходят для установки внутри или снаружи помещений и обычно монтируются на существующих опорах освещения и стенах зданий [9]. Минусом такого способа является чувствительность к окружающим металлическим конструкциям. А также то, что наличие в зоне работы системы металлической пыли способно полностью блокировать её работу.

Другой способ – применение дальномеров или оптических линеек. Этот подход определения местоположения мостового крана имеет следующие недостатки:

- дальномёры и оптические линейки крайне уязвимы к загрязнениям. Их точность зависит от степени загрязнения и измеряемого расстояния;
- датчик, установленный на кране, подвержен вибрациям во время его движения. Поэтому излучаемый оптический луч может не попадать в отражатель, особенно на дистанциях свыше 50 м;
- если на одном пути работает несколько кранов, то известны случаи, когда излученный одним датчиком луч принимался другим, что приводило к неверному измерению положения кранов [10].

Определение местоположения мостового крана, а значит и объекта в его захвате возможно средствами машинного зрения:

- с помощью обучения нейронной сети для распознавания текущей позиции по наблюдаемой сцене;

- с помощью распознавания уникальных графических элементов на кадре и определение позиции по ним.

Первый вариант решения имеет большие трудозатраты ввиду того, что для его реализации требуется подготовка большой выборки изображений, каждое из которых должно дополнительно содержать описание текущего местоположения на нём. Этот вариант имеет такой недостаток как: при каждом внедрении ПО на другой объект контроля придётся собирать новую выборку, производить разметку и дообучение нейронной сети.

Второй вариант прост в реализации. Чтобы его реализовать, необходимо разместить уникальные графические элементы и производить их распознавание средствами оптического контроля.

В дальнейшем, оба метода могут сочетаться, давая преимущество в виде резервного способа определения местоположения подъёмного средства на крановом пути.

В качестве таких уникальных графических элементов лучше всего использовать 2D коды. Они не могут быть ошибочно распознаны подпрограммой считывания 2D кодов так как их шаблон не может быть повторен с помощью конструкций цеха и оптической перспективы на считываемом изображении. Далее такие 2D коды будут называться маркерами.

Зная местоположение мостового крана на крановом пути, можно утверждать, что переносимый им объект находится там же, где и мостовой кран. Это утверждение верно только в том случае, когда мостовой кран удерживает этот объект в захвате.

Определение наличия объекта в захвате мостового крана можно реализовать следующими способами:

- получением информации от крановых весов [11]. Этот способ невозможно реализовать на всех мостовых кранах так как не все оснащены подобным устройством. А модернизация мостового крана приведёт к

длительному простоя из-за согласования и внесения изменений в паспорт грузоподъёмного механизма, а также самого процесса;

- лазерным 3D-сканером. Для определения наличия сталь-ковшей меньше всего подходит из-за цены, устойчивости к помехам от высокотемпературных объектов и запылённости воздуха;
- распознаванием объектов с помощью технического зрения.

Техническое зрение не всегда предполагает обучение нейронных сетей. В некоторых случаях, объекты на кадре можно определять исходя из их параметров. Например: объекты некоторой формы или объекты определённого цвета.

В случае с детекцией сталь-ковшей распознавание на основании признаков объекта не подходит ввиду того, что в кадре может находиться сразу несколько сталь-ковшей и достоверность определения резко снизится. Определение по цвету или яркости также не имеет смысла из-за того, что даже пустые сталь-ковши могут ярко светиться.

Лучший вариант для решения такой задачи - обучение нейронной сети для определения наличия объекта в захвате по кадру.

## 1.2 Описание перемещения объектов мостовыми кранами как объекта оптического контроля

### 1.2.1 Определение наличия объекта в захвате мостового крана

В данной работе перемещение объектов мостовыми кранами рассматривается на примере металлургического производства. Примером перемещаемых объектов в настоящей работе являются сталь-ковши, используемые в конвертерных цехах и миксерных отделениях металлургической компании НЛМК.

Ориентация изображения, получаемого с устройства захвата кадра, и угол обзора на нём играют важную роль во время определения наличия сталь-ковша в захвате мостового крана.

Необходимо подобрать эти параметры так, чтобы на получаемом изображении был виден весь возможный путь сталь-ковша во время операции его подъёма или опускания.

Для этих целей подойдёт вертикальная ориентация кадра, а угол обзора должен быть настроен так, чтобы была видна наземная часть цеха и самая верхняя точка подъёма захвата крана.

На рис. 1 представлена концептуальная схема наблюдения за захватом крана. На рис. 2 представлен пример изображения, получаемого с правильно настроенной камеры.

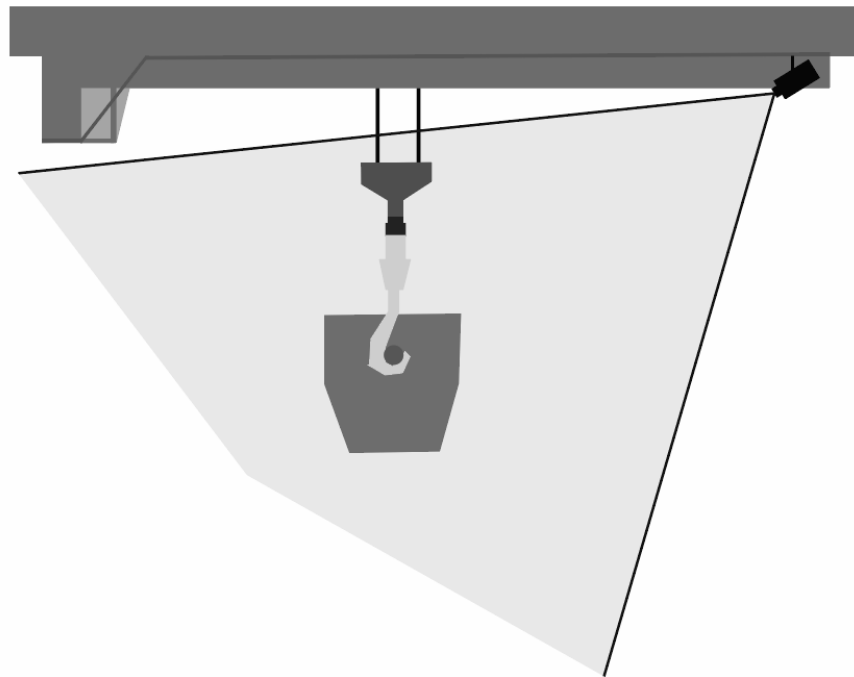


Рис. 1. Концептуальная схема наблюдения за захватом крана

Захват мостового крана может перемещаться вверх и вниз, а также отдаляться от камеры и приближаться к ней. Для технологических операций, значимую роль играет только подъём и опускание сталь-ковша. Например: постановка или снятие сталь-ковша с сталевого.

В захвате мостового крана может как присутствовать сталь-ковш, так и отсутствовать. Сам сталь-ковш может быть как пустым, так и заполненным жидким металлом.



Рис. 2. Пример изображения с ковшевой камеры крана

При этом, все сталь-ковши имеют стандартизированные размеры, в разных цехах могут использоваться разные сталь-ковши в различных комбинациях. Например: в цехе 1 используется только большой сталь-ковш, а в миксерном отделении цеха используется и малый и большой сталь-ковш.

Исходя из вышеописанного, можно получить следующий перечень возможных состояний захвата мостового крана:

- захват крана пуст;
- малый ковш пуст;
- малый ковш полон;
- большой ковш пуст;
- большой ковш полон.

К перечисленным выше состояниям также необходимо добавить состояние неопределённости.

Также, исходя из выделенных ранее основных путей перемещения захвата, можно выделить два возможных действия:

- захват поднимается;
- захват опускается.

Как и в предыдущем наборе состояний, для набора действий захвата необходимо добавить состояние неопределённости.

Состояние захвата и производимое над ним действие необходимо вышестоящей системе для полноты картины текущего процесса. Например, опускание захвата и смена состояния с полного ковша на пустой в конкретной позиции цеха может означать то, что содержимое сталь-ковша было перелито в определённый агрегат.

### 1.2.2 Определение текущего местоположения объекта

Зная о наличии сталь-ковша в захвате мостового крана, можно утверждать, что он находится на той же позиции что и сам мостовой кран.

Местоположение переносимого объекта необходимо для понимания фактического состояния запланированных операций перемещения сталь-ковша.

Для определения фактического местоположения сталь-ковша, переносимого мостовым краном, начальник смены периодически связывается с машинистом крана.

Полученная таким образом информация, позволяет ему сопоставить фактическое положение дел с суточным планом всех агрегатов.

Такой план составляется специалистом практически вручную с использованием доступных средств автоматизации. Это связано с тем, что у таких систем нет возможности пересчитывать график работы оборудования в реальном времени в зависимости от положения и статуса каждого ковша. Ввиду этого, план составляется один или два раза в смену.



Внедрение описываемого программного продукта в систему производственного планирования, позволит пересчитывать оптимальный график работы оборудования [12] при выходе ситуации за план.

Мостовой кран перемещается по подкрановым путям, расположенным вдоль пути крана в цехе. На рис. 3 представлен пример их расположения в цехе.

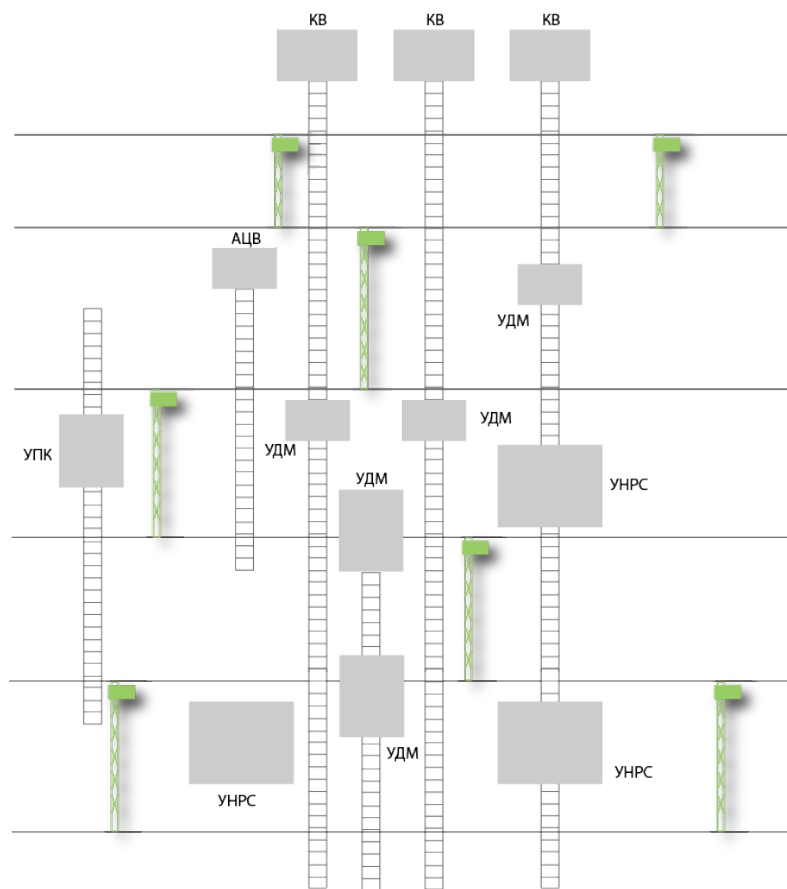


Рис. 3. Пример расположения крановых путей в цехе

В одном цехе находится несколько крановых путей, а на одном пути может находиться несколько мостовых кранов, определение местоположения стальной ковша с помощью камер, расположенных не на кранах, является неверным способом решения задачи позиционирования так как с наземных точек угол обзора не позволяет дополнительно определить состояние стальной ковша.

Верным решением является расположение видеокамеры на самом кране. При этом, для захвата большего числа маркеров ориентация изображения

должна быть горизонтальной. А оптическая схема должна быть настроена таким образом, чтобы камера наблюдала противоположную ей стену цеха.

Если разместить маркеры на наземной части цеха, то они могут перекрываться как техникой, так и могут быть повреждены во время рабочих процессов в цехе. Таким образом, правильным местом для размещения маркеров является противоположная от видеокамеры стена.

Каждый маркер должен иметь уникальный номер, и, для определения точного местоположения в цехе, соотноситься с координатой места его установки относительно кранового пути.

Таким образом, распознав маркер или несколько маркеров в кадре можно понять, где именно находится на текущий момент мостовой кран.

На рис. 4 представлена концептуальная схема позиционирования мостового крана.

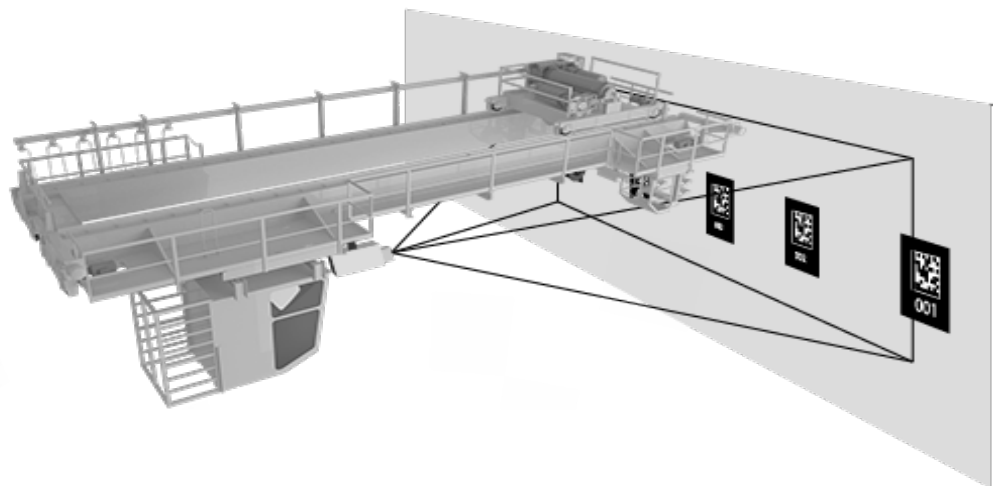


Рис. 4. Концептуальная схема позиционирования мостового крана

### 1.3 Постановка задачи, разработка требований к системе контроля перемещения объектов мостовыми кранами

#### 1.3.1 Построение дерева проблем

Разрабатываемая система обладает следующим рядом потенциальных проблем:

- алгоритм определения позиции плохо настроен;
- маркеры в кадре перекрыты;
- малый объем выборки при обучении детектора и классификатора захвата крана;
- оптическая схема детектора и классификатора захвата при обучении и использовании различается;
- наличие артефактов на изображении (“битые пиксели”, возникающие при проблемах трансляции видеопотока по сети);
- отсутствие подключения к камере позиционирования или камере наблюдения за захватом крана;
- не определяется или определяется неправильная позиция крана;
- не определяется или определяется неправильное состояние захвата.

Согласно вышеприведенному перечню потенциальных проблем, можно выделить корень дерева проблем и его побочные ветви.

Основная проблема – не полноценное определение состояния крана. Полноценным же состоянием мостового крана является такое состояние, которое включает в себя и текущую позицию и состояние захвата.

Дерево проблем представлено на рис. 5.

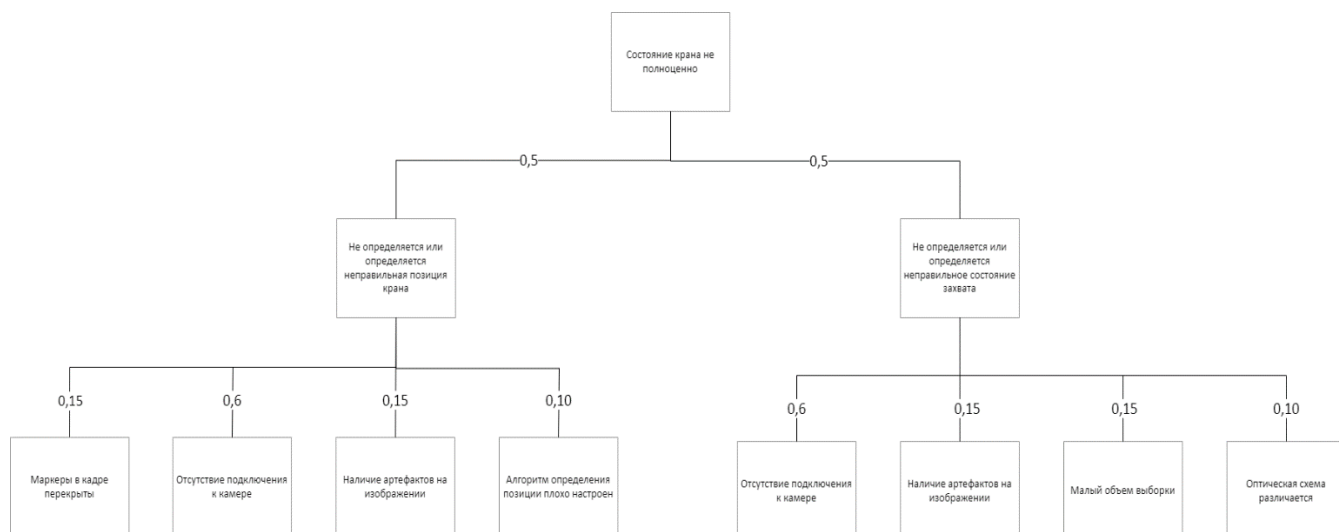


Рис. 5. Дерево проблем

### 1.3.2 Построение дерева целей

Главная цель системы - предоставление информации о кране вышестоящей системе планирования производственных процессов.

Для её достижения необходимо:

- распознавать состояние захвата и сталь-ковша в нём;
- определять производимое захватом действие;
- определять наличие движения крана;
- определять текущую позицию мостового крана. В том числе, когда нет ни одного маркера в поле зрения камеры;
- производить оповещение ответственных лиц при потере сигнала с камеры.

Маркеры могут пропасть из поля зрения камеры при высокой загрязнённости воздуха в цехе или при перекрытии их тросами, ковшом, или же другим мостовым краном.

Сигнал с камеры может теряться из-за сетевых проблем. Например, недостаточная дальность распространения сигнала wi-fi, физическое повреждение камеры из-за высоких температур.

Оповещение о проблемах с камерой можно реализовать с помощью сторонних инструментов. Например, с помощью сервиса сбора метрик Prometheus. В этом случае, необходимо не только предоставлять метрики работы камер системы, но и описать в сервисе правило [13], которое при наступлении определенного события произведёт оповещение лиц, ответственных за техническое состояние системы.

Проблему с отсутствием маркеров в кадре можно решить с помощью использования дополнительной камеры, например, камеры наблюдения за ковшом.

Такой способ позволит избавиться от перекрытия маркеров тросами, захватом, или ковшом. При этом, поле зрения обеих камер может перекрыть другой мостовой кран, если он подъедет достаточно близко.

Такое расположение кранов – частный случай, так как на большинстве крановых путей находится по одному мостовому крану. Чтобы избежать ситуации, когда маркеры первого крана перекрыты вторым и оба крана синхронно перемещаются по крановому пути в одну сторону. А затем второй кран отъезжает и происходит резкий скачок позиции первого крана, ввиду появления маркеров в кадре. Необходимо предусмотреть дополнительный механизм определения местоположения, который будет срабатывать при отсутствии маркеров на изображении с видеокамеры. Таким механизмом может выступать, позиционирование по наблюдаемой сцене.

Дерево целей представлено на рис. 6.



Рис. 6. Дерево целей

### 1.3.3 Построение дерева задач

В результате анализа проблем и определения целей было построено дерево задач, которое представлено на рис. 7.

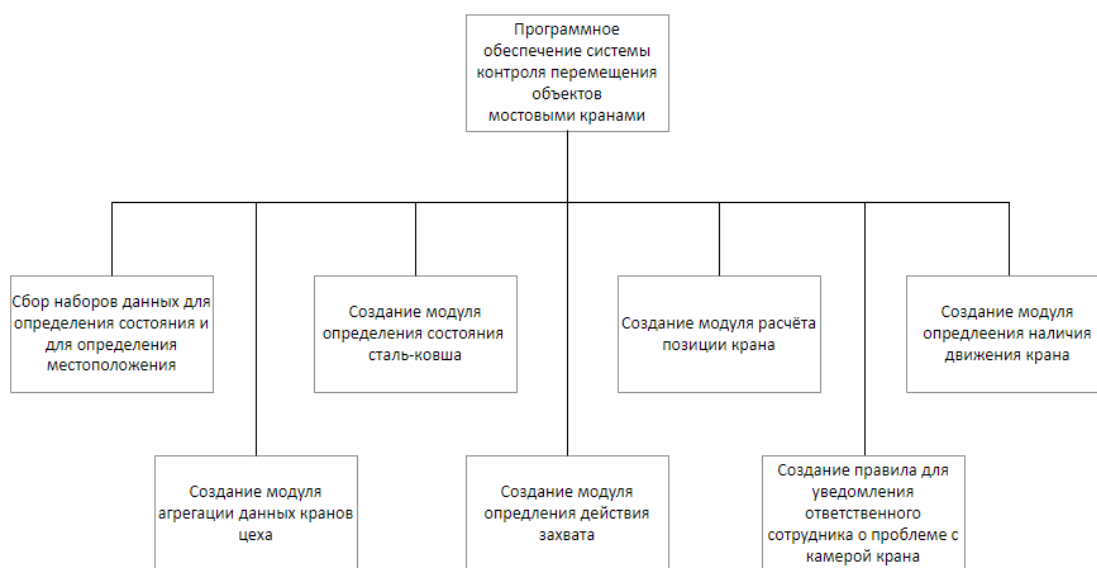


Рис. 7. Дерево задач

Согласно представленному дереву задач, для реализации программного обеспечения, необходимо создать:

- модуль для определения состояния сталь-ковша в захвате крана;

- модуль для определения текущего местоположения мостового крана;
- модуль для сбора текущего состояния кранов в цехе и предоставления собранной информации заказчику в виде единого пакета данных;
- модуль для определения производимого захватом действия;
- модуль для определения наличия движения мостового крана.

Помимо прочего, необходимо собрать выборку изображений с ковшевой камеры для обучения набора нейронных сетей, которые будут определять текущее состояние захвата и сталь-ковша в нём.

Также необходимо собрать выборку для обучения нейронной сети позиционирования по наблюдаемой сцене. Такая выборка может быть собрана только после опытной эксплуатации программного обеспечения ввиду того, что для каждого кадра необходимо сохранять описание текущей позиции мостового крана.

#### 1.3.4 Требования к математическому обеспечению

Система контроля перемещения объектов мостовыми кранами должна работать в режиме реального времени, обладать частотой распознавания состояния сталь-ковша на видеокадре, а также определения местоположения крана не реже одного распознавания в секунду.

Алгоритм определения состояния сталь-ковша крана должен работать как с большими, так и с малыми сталь-ковшами. Точность детекции и классификации сталь-ковша должна быть не менее 97 %.

Алгоритм определения местоположения мостового крана должен работать в любом цехе и иметь точность определения позиции крана не менее 98 % и погрешность позиционирования не более трёх метров.

### 1.3.5 Требования к информационному обеспечению

Входные данные для работы программного обеспечения следующие:

- видеопотоки камер позиционирования;
- видеопотоки камер слежения за захватом;
- конфигурация системы;
- веса применяемых нейронных сетей;
- данные крановых весов.

Выходные данные:

- текущий кадр видеоканала определённого крана;
- состояние кранов цеха;
- детальное состояние кранов цеха.

### 1.3.6 Требования к аппаратно-программному обеспечению

Надёжное (устойчивое) функционирование программы должно быть обеспечено выполнением совокупности организационно-технических мероприятий, перечень которых приведен ниже:

- организацией бесперебойного питания технических средств;
- использованием лицензионного программного обеспечения;
- регулярным выполнением рекомендаций Министерства труда и социального развития РФ, изложенных в Постановлении от 23 июля 1998 г. «Об утверждении межотраслевых типовых норм времени на работы по сервисному обслуживанию ПЭВМ и оргтехники и сопровождению программных средств»;
- регулярным выполнением требований ГОСТ 51188-98. Защита информации. Испытания программных средств на наличие компьютерных вирусов.



### 1.3.7 Требования к информационной и программной совместимости

Для стабильной работы разрабатываемого ПО, на компьютере должно быть установлено следующее программное обеспечение:

- Microsoft Windows Server 2019 Standard;
- NET Framework 4.8;
- NET 7 Runtime;
- установленный драйвер NVidia версии 451.48 или более поздняя версия, совместимая с CUDA 11.0;
- Nomad версии 1.3.1 и выше;
- Consul версии 1.12.2 и выше;
- Consul template версии 0.29.0 и выше;
- Nginx версии 1.23.0 и выше;
- Loki версии 2.7.1 и выше;
- Grafana версии 9.3.1 и выше;
- Prometheus версии 2.36.2 и выше.

Программа должна функционировать на серверах со следующими характеристиками:

- процессор Intel Xeon Gold с частотой 2,99 ГГц или большей;
- объем ОЗУ минимум 32 ГБ;
- видеокарта Nvidia GeForce 1070Ti или более мощная с поддержкой технологии CUDA 11.0;
- дополнительные 1 ТБ свободного места на жёстком диске.

### 1.4 Выводы по разделу

В данном разделе был выполнен обзор имеющихся средств, решающих как задачу определения местоположения сталь-ковша на крановом пути, так и задачу определения состояния сталь-ковша в захвате крана.

Он показал, что на сегодняшний день, для сталеплавильного производства нет решения, позволяющего определить местоположение и состояние сталь-ковша на крановом пути. Следовательно, тема магистерской диссертации является актуальной.

Также было выявлено, что для определения местоположения мостового крана, а следовательно, и позиции сталь-ковша на крановом пути, лучше всего использовать методы машинного зрения. С их помощью будут распознаваться маркеры. По распознанным маркерам будет определяться текущее местоположение мостового крана.

Для решения проблемы перекрытия маркеров тросами, захватом, или ковшом, необходимо во время определения местоположения использовать дополнительную видеокамеру, которой может являться камера наблюдения за захватом и сталь-ковшом.

Частный случай перекрытия маркеров другим мостовым краном, может быть решен использованием запасного механизма определения местоположения мостового крана – распознаванием позиции по наблюдаемой сцене.

Также было выявлено, что для определения состояния сталь-ковша в захвате мостового крана лучше всего использовать нейронные сети.

В первой главе была выполнена постановка задачи магистерской диссертации, сформированы требования к разрабатываемой системе, определены проблемы, которые могут возникнуть в процессе реализации, а также предложены пути их решения.

## 2 Математическое и алгоритмическое обеспечение системы контроля перемещения сталь-ковшей мостовыми кранами

### 2.1 Метод позиционирования мостового крана

Как было сказано ранее, для определения позиции мостового крана будут использоваться 2D коды. В качестве идентификаторов могут выступать такие 2D коды как: Datamatrix, QR-code, Aztec Code.

Datamatrix, по сравнению с QR-code и Aztec Code занимает на 30–60% меньшую площадь [14] и обладает более высоким уровнем коррекции ошибок, что позволяет восстановить до 30% информации при повреждении метки [14].

В качестве маркеров позиционирования предполагается использовать комбинированный маркер, содержащий datamatrix-код для машинного чтения и дублирующий цифровой код для визуальной ориентации машиниста крана и для исключения путаницы при монтаже и настройке системы.

Для хорошей визуальной различимости цифрового кода на изображении с камеры, высота кода должна составлять от 20 пикселей. Нумерация маркеров предполагается в диапазоне от «001» до «999».

Маркер должен быть размещен на плоском твердом носителе, исключаящем изломы и искажения кода, например, напечатан матовой краской на металлическом листе.

Для позиционирования мостового крана по маркерам предполагается использование метода, описанного в патентном документе [15].

Согласно описанному в документе методу, для надежного позиционирования крана на всем маршруте движения необходимо обеспечить попадание в поле зрения камеры хотя бы трех кодов в каждый момент времени.

Маркеры должны быть размещены вдоль пути движения крана и лежать в вертикальной плоскости параллельной пути движения крана.

В реальных условиях сталеплавильного производства соблюдение условия наличия трёх кодов в кадре не всегда представляется возможным. Например,

перекрытие маркеров сталь-ковшом или тросами подъёмного механизма. Пример случая перекрытия маркера тросами представлен на рис. 8.



Рис. 8. Пример перекрытия маркера тросами подъёмного механизма

Надёжность метода можно повысить, используя для позиционирования дополнительную камеру.

В таком случае, появится другая перспектива на наблюдаемые первой камерой маркеры. А также появится возможность расширить поле зрения первого устройства захвата изображений.

Однако, для этого необходимо усовершенствовать метод расчета местоположения, который будет основываться на данных сразу нескольких камер. Для получения итогового местоположения мостового крана необходимо будет найти средневзвешенное по мгновенным позициям каждой камеры.

На рис. 9 изображен маркер, который закрыт тросами для камеры позиционирования. Кадр получен с другой камеры.

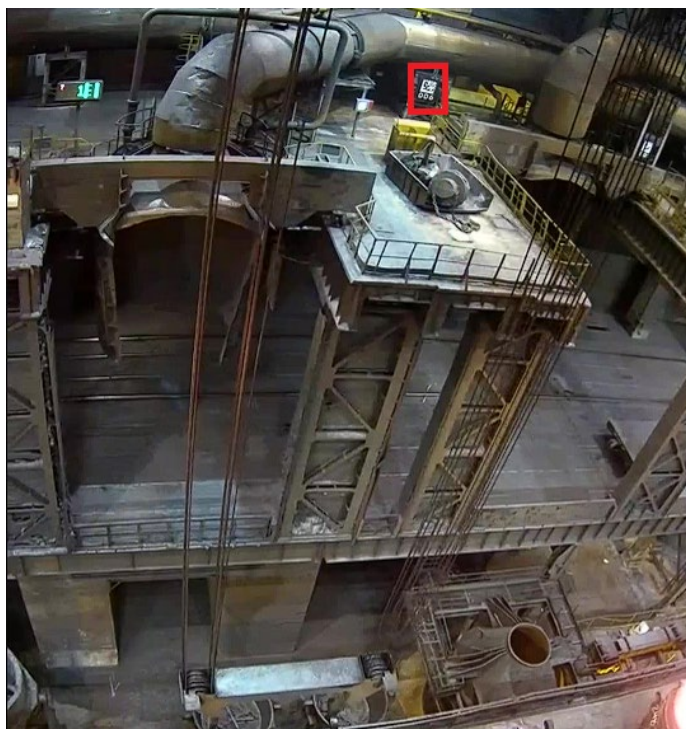


Рис. 9. Пример перекрытого маркера тросами с другой камеры

Прежде чем определять местоположение по маркерам, необходимо настроить используемый метод получения мгновенной позиции.

Для этого измеряются расстояния между центрами маркеров, и информация о их фактическом расположении на крановом пути записывается в таблицу для фиксации расположения установленных маркеров [15].

Для определения позиции крана используется информация о фактическом расположении маркеров в системе координат цеха из таблицы и информация о положении распознанных маркеров на кадре [15].

Для привязки координат видеокамеры к внешней системе отсчета необходимо задать отображение пиксельных координат на изображении в соответствующие им координаты во внешней системе отсчета [15].

В качестве способа выполнения привязки координат видеокамеры к внешней системе отсчета используется функция, задающая для каждого столбца изображения его отклонение от фактического положения крана [15]:

$$D_x = f(c), \quad (1)$$

где:

$c$  - номер столбца изображения,

$D_x$  - разница координат наблюдаемой в столбце  $c$  точки на линии установки маркеров и координат крана.

Данная функция может быть задана таблично, что позволяет учесть радиальные искажения оптики, неизбежно возникающие при широких углах обзора объектива видеокамеры [15].

На рис. 10 представлен пример изображения для настройки привязки координат к внешней системе отсчёта.

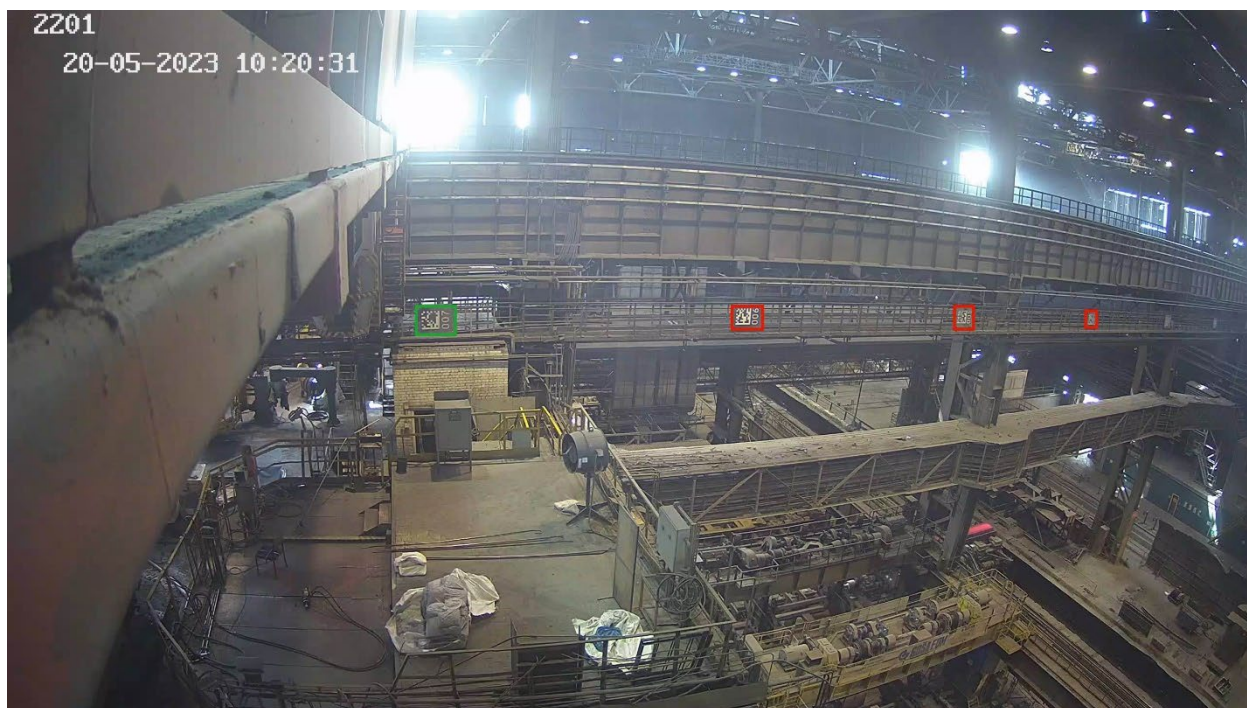


Рис. 10. Пример изображения для настройки привязки координат к внешней системе отсчёта

На рис. 10 имеется координата на противоположной стене, которая расположена прямо напротив камеры крана. То есть, направление от камеры к этой точке даёт примерно перпендикуляр на стену.

Также, на изображении имеются маркеры справа от этой координаты и один из маркеров расположен в ней. Такой маркер на рисунке 5 отмечен зелёной рамкой.

Горизонтальные координаты маркеров дадут номера столбцов изображения.

Из фактических положений этих маркеров в цехе вычитается фактическое положение, на котором расположен маркер по координате, дающей перпендикуляр. В таком случае, получим набор значений отклонений маркеров на изображении от фактического положения камеры крана

В табл. 1 представлен пример результата настройки метода определения мгновенной позиции.

Таблица 1

Результат настройки метода определения мгновенной позиции

|                                    |     |        |        |        |
|------------------------------------|-----|--------|--------|--------|
| Столбец<br>изображения,<br>пиксель | 790 | 1564   | 2002   | 2262   |
| Смещение, м                        | 0   | -13,59 | -26,09 | -37,54 |

Вернемся к этапу оценки позиции крана на основе информации о положении на изображении распознанных маркеров. В применяемом методе для каждого из распознанных маркеров  $M_i$  на основе информации о номере столбца  $c_i$ , в котором расположен центр этого маркера и функции привязки, определяется отклонение позиции крана от позиции маркера  $Dx_i$ . Далее используя данные о фактическом расположении  $x(M_i)$  распознанного маркера  $M_i$ , вычисляется положение крана на основе информации только о рассматриваемом маркере:

$$x_i = x(M_i) - Dx_i, \quad (2)$$

где:

$x_i$  – положение крана по рассматриваемому маркеру,

$M_i$  – распознанный маркер,

$Dx_i$  – отклонение позиции крана от фактической позиции маркера.

Выполнив такую оценку позиции крана для каждого распознанного на текущем кадре маркера и усреднив их значения получим среднюю мгновенную оценку позиции крана [15].

Полученная мгновенная позиция может иметь некоторое смещение от фактической позиции крана ввиду того, что камера может быть установлена левее или правее мостового крана. Для решения этой проблемы, необходимо к мгновенной позиции крана прибавить значение смещения камеры от крана. По следующей формуле:

$$x = p + o, \quad (3)$$

где:

$p$  – мгновенная позиция,

$o$  – смещение камеры от мостового крана.

После этого, для получения местоположения мостового крана сразу по нескольким камерам, необходимо рассчитать средневзвешенную позицию по формуле:

$$x = \frac{\sum_{i=1}^n p_i * w_i}{\sum_{i=1}^n w_i}, \quad (4)$$

где:

$n$  - количество кадров, участвующих в расчёте позиции,

$p_i$  – скорректированная позиция крана, рассчитанная по  $i$ -тому кадру,

$w_i$  – количество обнаруженных на  $i$ -том кадре маркеров.



Затем выполняется описанное в усовершенствованном методе сглаживание позиции крана на основе средневзвешенной позиции и истории.

В результате усовершенствования метода, был получен алгоритм расчёта местоположения мостового крана по нескольким камерам. Он представлен на рис. 11.

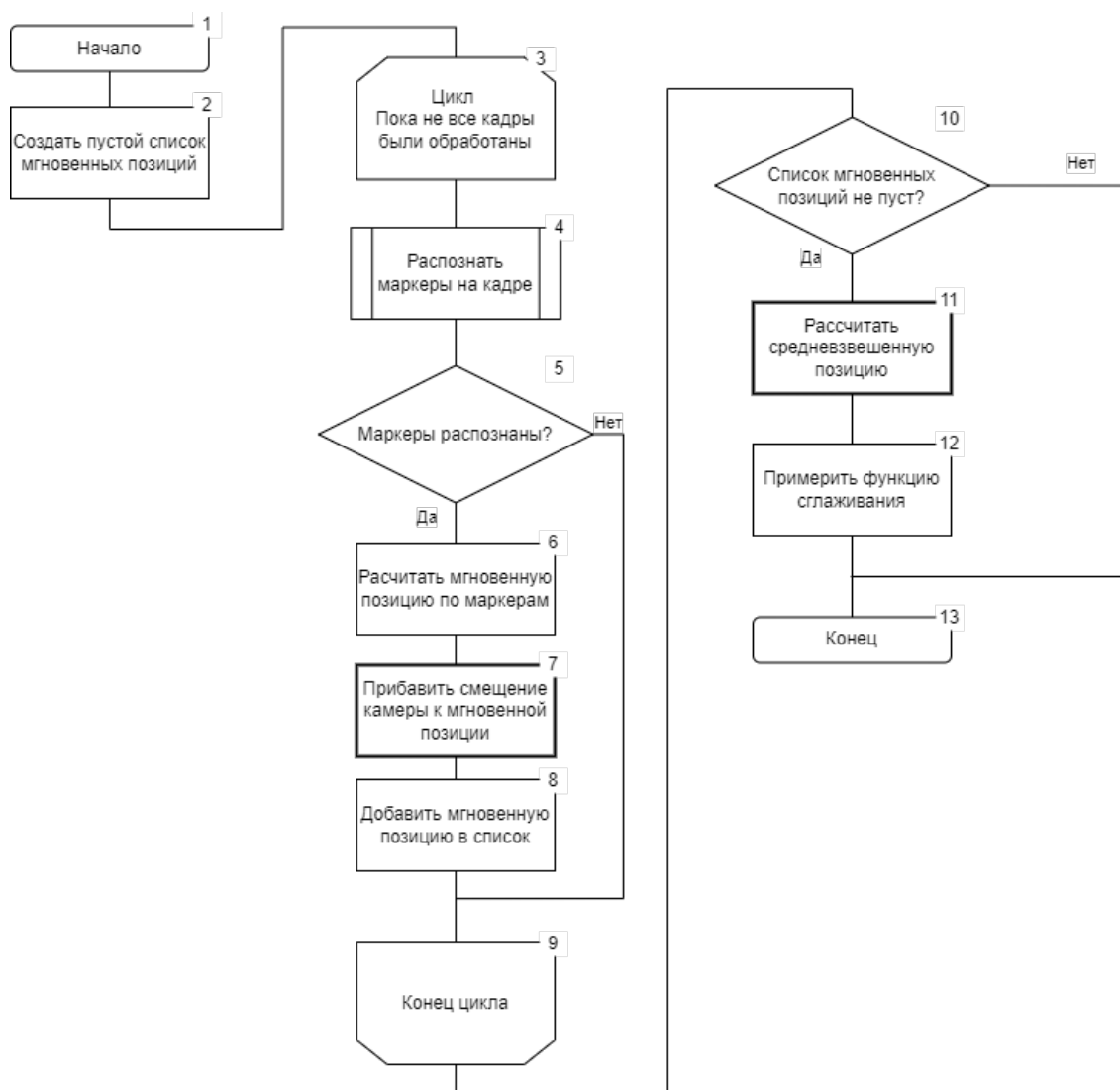


Рис. 11. Алгоритм расчёта местоположения мостового крана по нескольким камерам

Согласно представленному алгоритму, в начале инициализируется список мгновенных позиций.

После этого, по каждому кадру, переданному на вход алгоритма, сервисом распознавания 2D кодов распознаются маркеры.

Если маркеры на кадре были распознаны, то рассчитывается мгновенная позиция по ним, прибавляется к рассчитанной позиции значение смещения для корректировки расстояния между фактическим местоположением мостового крана и местом установки камеры, мгновенная позиция добавляется в ранее созданный список.

После обработки всех кадров, если в списке есть хотя бы одна мгновенная позиция, то выполняется расчёт средневзвешенной позиции по нескольким камерам и выполняется сглаживание результата расчёта.

## 2.2 Алгоритм определения состояния сталь-ковша в захвате крана

Для определения состояния сталь-ковша применяется ансамбль [16] нейронных сетей. Он состоит из двух идущих друг за другом моделей. Выход одной модели является входом последующей.

Этот подход используется ввиду схожести определяемых классов. Например, на рис. 12 представлен пустой ковш, металл из которого недавно перелили в один из агрегатов цеха. Стенки ковша всё ещё светятся.



Рис. 12. Пример пустого сталь-ковша после перелива содержимого

На рис. 13 представлен пример полного сталь-ковша. Корка на поверхности ещё не успела сформироваться и застыть. Из-за жидкого металла он также светится.



Рис. 13. Пример полного сталь-ковша

Ситуации на изображениях выглядят почти идентично, но на самом деле они отличаются.

Со временем стенки сталь-ковша остывают. Это отображено на рис. 14.



Рис. 14. Сталь-ковш с остывающими стенками

Со временем изменения происходят и с полным сталь-ковшом. Там появляется корка на поверхности и два класса становятся более различимыми. Пример представлен на рис. 15.



Рис. 15. Сталь-ковш с коркой на поверхности

Для обучения нейронных сетей необходима размеченная выборка. Исходные данные для разметки были получены путём извлечения кадров из записанных с ковшевой камеры видеофайлов с помощью библиотеки ffmpeg [17].

Полученные кадры были размечены с помощью инструмента аннотирования компьютерного зрения CVAT [18]. На рис. 16 представлен пример разметки обучающей выборки.

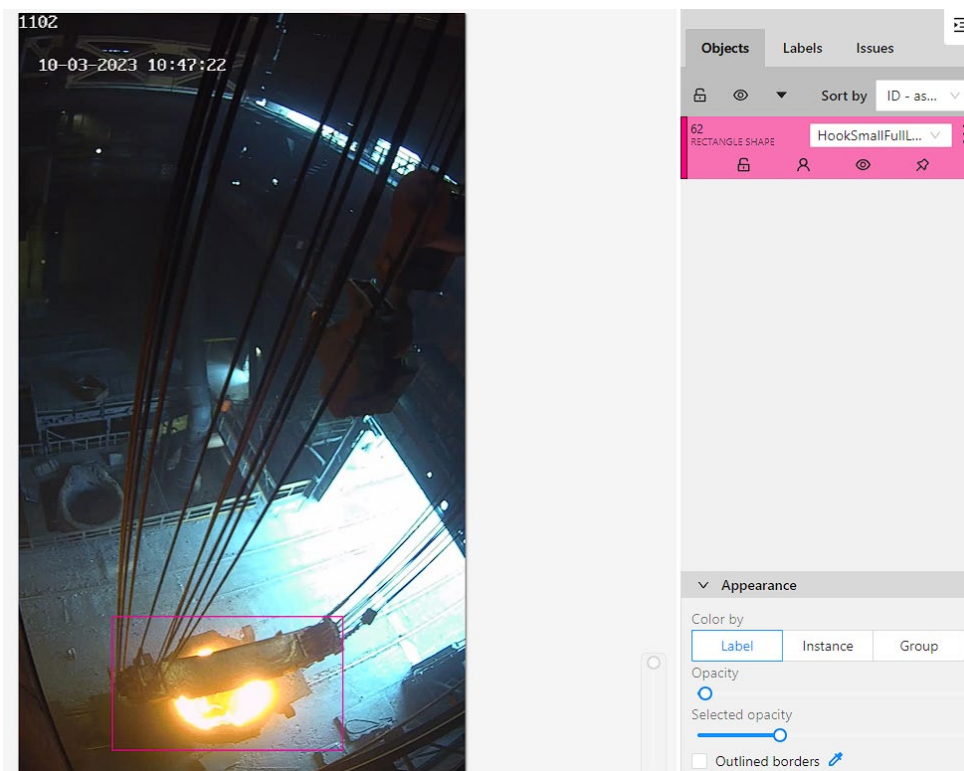


Рис. 16. Пример разметки обучающей выборки

После разметки, средствами инструмента экспортируется аннотация к набору данных. Аннотация представляет собой файл в формате JSON, который содержит в себе путь к изображению и рамку с заданным при разметке классом. Пример файла аннотации представлен на рис. 17.

```
<image id="1" name="[2023-03-04_02-30-42-763][1][62.39]-fde5edfc-0a86-45a5-a0be-4ecfcd06784.jpeg" width="1080" height="1920">
  <box label="HookSmallFullLadle" source="manual" occluded="0" xtl="199.48" ytl="1161.81" xbr="1080.00" ybr="1728.72" z_order="0">
  </box>
</image>
<image id="2" name="[2023-03-04_03-49-28-059][1][62.37]-835db963-3cef-44b7-84bc-b124c73c87e9.jpeg" width="1080" height="1920">
  <box label="HookEmpty" source="manual" occluded="0" xtl="52.51" ytl="802.54" xbr="451.44" ybr="1082.49" z_order="0">
  </box>
</image>
<image id="3" name="[2023-03-04_08-19-43-682][1][56.84]-46beb54a-47db-42d8-852c-66c43f98950d.jpeg" width="1080" height="1920">
  <box label="HookEmpty" source="manual" occluded="0" xtl="162.15" ytl="1269.13" xbr="712.72" ybr="1591.07" z_order="0">
  </box>
</image>
<image id="4" name="[2023-03-04_08-20-43-938][1][59.11]-98a647b7-86c9-4948-bf02-dd764263f010.jpeg" width="1080" height="1920">
```

Рис. 17. Пример содержимого файла аннотации

На текущий момент, существует множество готовых фреймворков, реализующих различные архитектуры нейронных сетей, что позволяет не формировать архитектуру сети с нуля, а также ускоряет проверку предложенного решения экспериментальным путём.

В данной работе для проверки решения по контролю перемещения стальной использовалась нейронная сеть YOLOv4 [19].

YOLO – это одноступенчатый алгоритм обнаружения объектов, предложенный Джозефом Редмоном и соавторами в 2015 году. Данный подход рассматривает обнаружение объекта на изображении как проблему регрессии, исходными данными для которых являются пиксели входного изображения, а искомым значением координаты рамки, ограничивающей объект, и распределение вероятностей по классам. Одноступенчатым алгоритм является потому, что изображение проходит через сеть только один раз, прежде чем будут сделаны предсказания по локализации и классификации объектов на данном изображении [19].

В компании Малленом Системс имеется множество готовых инструментов для работы с четвёртой версией этой архитектуры. Например: скрипт для конвертации обученной модели в формат открытого стандарта ONNX [20].

Также имеется и готовый набор скриптов для её обучения. Он основан на базе фреймворка Darknet. Для запуска обучения требуется указать путь к файлу аннотации и задать параметры в файле конфигурации.

Готовый набор скриптов произведёт аугментацию изображений и разделит набор данных на 3 выборки: тренировочную, тестовую, выборку для валидации. 70% данных будет приходиться на обучающую выборку, а 10% на тестовую. Остальные 20% размеченных данных будет составлять выборка для валидации.

Аугментация проводится по заданным в качестве параметров типам модификации данных. Для детекции и классификации с помощью YOLOv4 использовались такие типы аугментации как:

- наложение гауссовского шума;
- случайное контрастирование.

В результате обучения, набор готовых скриптов, выдаст наилучший по метрикам файл весов и выведет для него значение метрик Precision и Recall.

Перед обучением модели были настроены параметры обучения и гиперпараметры [21]. По факту, они подбираются методом проб и ошибок ввиду того, что нет возможности сразу же подобрать такой набор настроек, с которым бы модель могла за короткий срок достичь требуемой точности.

В качестве настраиваемых параметров выступает количество эпох. Этот параметр задаёт количество проходов одной партии набора данных через архитектуру сети для извлечения необходимых признаков. Его значение равняется 1000.

Размер партии, коэффициент скорости обучения, цветность изображения, размер изображения – гиперпараметры.

Размер партии зависит от вычислительных возможностей машины, на которой будет проходить обучение. Это параметр влияет на скорость прохождения одной эпохи. Параметр был установлен в значение 16.

Коэффициент скорости обучения позволяет управлять величиной коррекции весов на каждой эпохе. Его значение зависит от размера партии. Чем больше размер партии, тем выше становится коэффициент обучения. Параметр был установлен в значение 0.001.

Цветность изображения определяет количество цветовых каналов подаваемого на вход изображения. Значение этого параметра было приравнено 3, то означает, что на вход подаётся цветное изображение.

Размер изображения представляет собой два параметра, отвечающих за высоту и ширину изображения. Значения обоих параметров должны быть кратны 32. В настоящей работе используется стандартный размер изображения для этой версии архитектуры – 608x608 [22].

Также стоит отметить, что фреймворк darknet автоматически выполняет изменение размера входящего изображения. Например: для распознавания было подано изображение размером 1080x1920. Его размер будет уменьшен до размера 608x608 [23].

Опытным путём было определено, что если в качестве детектора и классификатора использовать только одну сеть, то после обучения на 3000 изображений точность составит 91 %.

Для повышения точности, определение состояния сталь-ковша в захвате крана можно разделить на две задачи:

- определение захвата и сталь-ковша на кадре;
- классификация сталь-ковша.

Для повышения точности классификации были обучены на 3000 изображений сталь-ковшей и захватов мостового крана две нейронные сети, а затем выбрана лучшая из них.

Нейронные сети для классификации обучались для определения одного из следующих пяти классов:

- захват пустой;
- в захвате есть пустой малый сталь-ковш;
- в захвате есть полный малый сталь-ковш;
- в захвате есть пустой большой сталь-ковш;
- в захвате есть полный большой сталь-ковш.

В рамках проверки предположения о улучшении точности классификации, рассматривались две нейронные сети: MobileNetV2 [24] и EfficientNet-b1 [25]. Точность классификации для MobileNetV2 составила 98.03 %, для EfficientNet-b1 98,44 %. Исходя из этих показателей, был сделан выбор в пользу модели EfficientNet-b1.

EfficientNet – семейство моделей, цель разработки которых заключается в том, чтобы в процессе масштабирования сохранить высокое качество решения задачи и при этом повысить эффективность модели [25].

Для обучения выбранной модели в компании Малленом Системс также имеется готовый инструментарий, с помощью которого проводилось её обучение.

Для классификатора необходимо видоизменить выборку. А именно, из изначального изображения, которое размечали, вырезать содержимое внутри размеченной рамки детекции.

Таким образом, на вход классификатора будет подаваться не всё изображение, а только сталь-ковш и захват или пустой захват.

Аналогичным образом, готовые средства производят аугментацию и разделение выборки на 3 набора данных для обучения. В результате которого, готовые скрипты сохраняют наилучший файл весов и выводят значение метрики Ассигасу. Эта метрика больше подходит для проверки классификации, так как показывает общую точность предсказания модели по всем классам.



Для классификатора на вход также подаётся цветное изображение, но размер изображения составляет 240x240. Коэффициент обучения установлен в значение 0.0003. Размер партии для обучения, как и у детектора равняется 16. При этом, количество эпох также равняется 1000.

Сеть детекции, в свою очередь, была переучена для детекции одного класса – в кадре есть захват. В результате этого, метрика точности составила 99%, как и метрика полноты.

Таким образом, был получен новый алгоритм определения состояния сталь-ковша в захвате мостового крана. Он представлен на рис. 18.

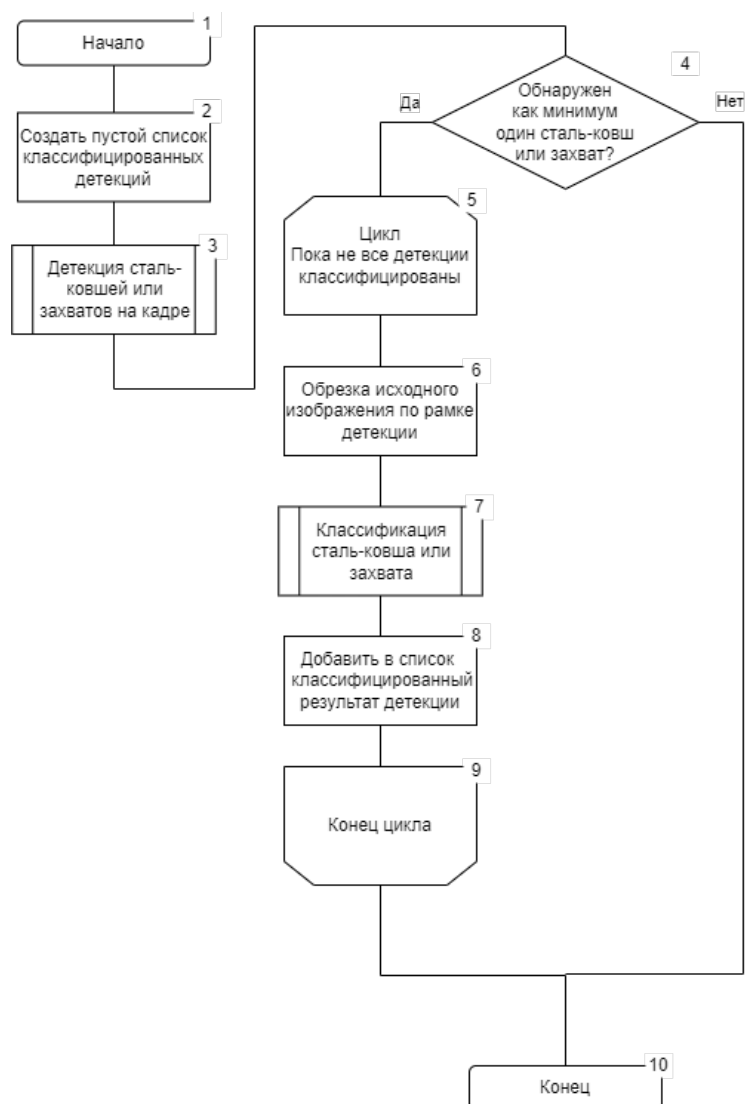


Рис. 18. Алгоритм определения состояния сталь-ковша в захвате мостового крана

Согласно представленному алгоритму, сначала создаётся пустой список классифицированных детекций и полученный кадр подаётся в сеть-детектор, которая может вернуть одну или несколько детекций. В противном случае, результатом выполнения алгоритма будет являться пустой список.

Если сеть-детектор в результате обработки кадра вернула одну или несколько детекций, то эти детекции перебираются в цикле до тех пор, пока каждая из них не будет обработана сетью-классификатором.

Для каждой детекции, иными словами сталь-ковша или захвата, по информации о его местоположении на изображении выполняется обрезка исходного кадра. Таким образом, в классификатор подаётся только изображение сталь-ковша или захвата.

Сеть-классификатор возвращает число обозначающее состояние захвата или сталь-ковша.

Полученный ответ классификатора комбинируется с информацией о местоположении детекции на исходном кадре и добавляется в список классифицированных детекций.

В таком случае, результатом работы алгоритма будет список детекций на текущем кадре каждая из которых обладает одним из пяти классов.

## 2.3 Алгоритм определения местоположения по наблюдаемой сцене

Как было сказано ранее, маркеры могут быть перекрыты другим мостовым краном. При этом, оба мостовых крана могут синхронно перемещаться в одну сторону.

В таком случае, когда в кадре снова появятся маркеры, то произойдёт скачок местоположения мостового крана. Для предотвращения подобной ситуации, необходимо иметь резервный способ определения позиции крана. На рис. 19 представлен алгоритм определения позиции по наблюдаемой сцене.

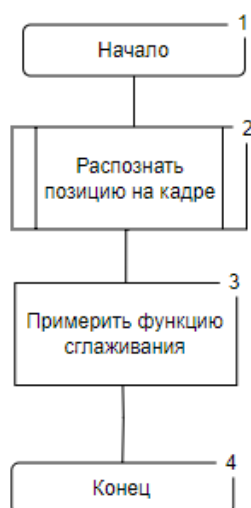


Рис. 19. Алгоритм определения позиции по наблюдаемой сцене

Согласно приведённому алгоритму, во втором блоке кадр отправляется в подпрограмму распознавания позиции. Эта подпрограмма представляет собой сервис исполнения нейронных сетей. По запросу на распознавание позиции, в этом сервисе запускается нейронная сеть, разработанная компанией Малленом Системс для подтверждения предположения о возможности определения позиции по наблюдаемой на кадре сцене. Обученная модель выполняет классификацию текущей позиции по кадру. Полученный ответ, представляющий из себя число, проходит через функцию сглаживания для устранения колебаний. В результате получается сглаженная позиция.

#### 2.4 Алгоритм локализации объектов на крановом пути

Имея информацию о позиции мостового крана и информацию о наличии сталь-ковша в захвате, можно произвести их объединение и анализ для получения единого пакета данных. Он будет содержать не только местоположение сталь-ковша и его состояние, но и признак наличия подъёма или опускания. Также будет присутствовать признак перемещения сталь-ковша вдоль кранового пути.

На рис. 20 представлен алгоритм локализации объектов на крановом пути.

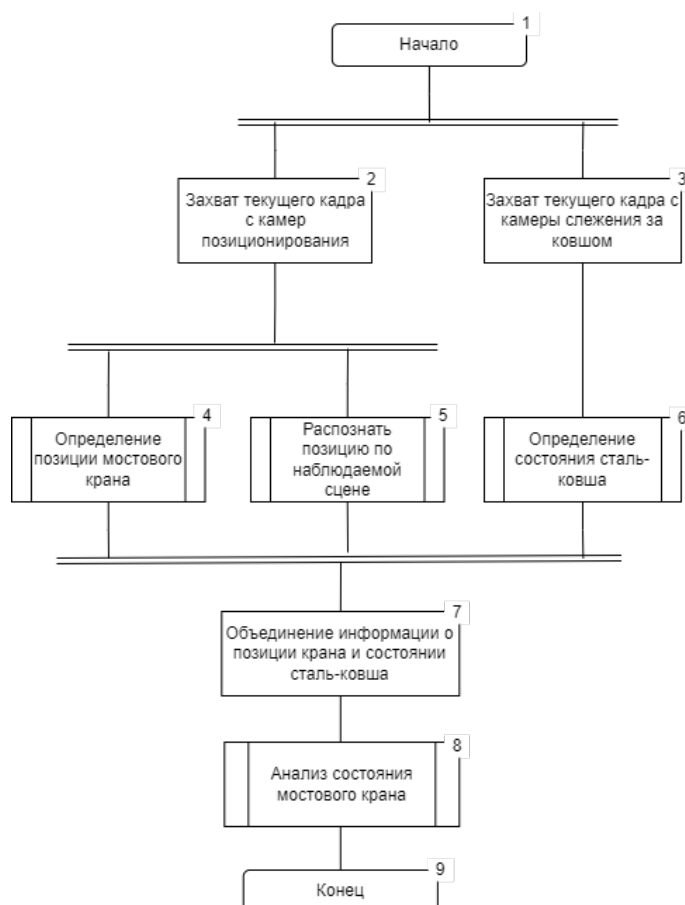


Рис. 20. Алгоритм локализации объектов на крановом пути

Согласно представленному алгоритму, параллельно с видеокамер захватывается по кадру.

Затем, кадры подаются на вход описанных выше алгоритмов. После чего результаты работы алгоритмов объединяются в 7 блоке в состояние мостового крана.

В дальнейшем, это состояние будет передано на анализ для определения наличия движения крана и определения действия захвата.

Как можно было заметить, помимо определения позиции по маркерам происходит определение позиции по наблюдаемой сцене. Это дополнительный вариант позиционирования.

После объединения результатов в состояние мостового крана, происходит его анализ.

## 2.5 Алгоритм анализа состояния мостового крана

Этот алгоритм (рис. 21) расширяет состояние мостового крана дополнительной информацией.

Согласно приведённому алгоритму, в подпрограмме второго блока определяется действие захвата крана. Его определение происходит на основании истории местоположения рамки детекции захвата и сталь-ковша на изображении. Из позиции по координате Y самого первого местоположения из истории вычитается текущее. Если модуль результата превысит установленный порог, то по знаку разницы определяется тип действия. Положительный – опускание захвата, а отрицательный – подъём. Если порог не был превышен, то действие не определено.

После этого, по наличию распознанных маркеров, выбирается источник информации о местоположении крана. Если есть хотя бы один распознанный маркер, то применяется местоположение по маркерам. В противном случае, по наблюдаемой сцене.

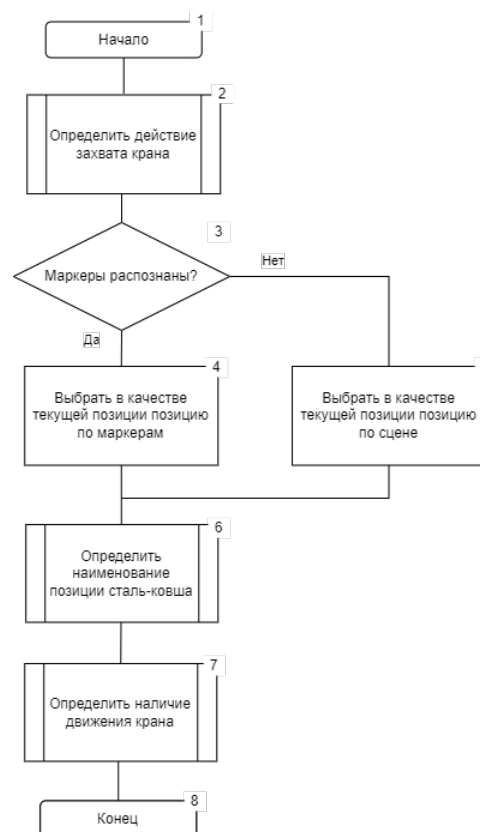


Рис. 21. Алгоритм подсистемы анализа данных крана

В подпрограмме шестого блока определяется наименование текущей позиции сталь-ковша исходя из его положения на кадре и местоположения на крановом пути.

В седьмом блоке определяется наличие движения крана путём анализа истории смены позиции. Если хотя бы одна разница текущей позиции и позиций из истории по модулю больше заданного порога, то кран движется. Иначе, движения нет или оно не значительное.

## 2.6 Выводы по разделу

В данном разделе был предложен метод определения позиции мостового крана по маркерам с нескольких камер. Был описан алгоритм определения состояния сталь-ковша в захвате мостового крана. Описан алгоритм контроля перемещения сталь-ковшей мостовыми кранами на основе методов машинного зрения.

Также было дано описание алгоритма определения местоположения по наблюдаемой сцене и алгоритма анализа данных о состоянии мостового крана.

Предложенный метод определения местоположения мостового крана по маркерам, позволяет решить проблему, возникающую при перекрытии какого-либо маркера. А также устраняет смещение рассчитанного местоположения, которое возникает в случае монтажа камеры на некотором расстоянии от мостового крана.

Благодаря использованию ансамбля нейронных сетей удалось повысить точность детекции до 99 %, а точность классификации при этом составила 98,44 %.

Применение дополнительного способа позиционирования, а именно позиционирование по наблюдаемой сцене, позволит решить проблему определения местоположения мостового крана, возникающую при отсутствии маркеров кадре.

При всём этом, контроль перемещения объекта, примером которого является сталь-ковш, становится возможным при совокупном учёте позиции мостового крана и состояния сталь-ковша в его захвате.

В итоге работы алгоритмов, формируется состояние мостового крана, по которому система производственного планирования будет способна определить как наличие сталь-ковша в захвате крана, так и его местоположение на крановом пути. При этом, становится доступным определение текущего производственного процесса. Например: смена состояния «ковш полный» на «ковш пустой», может дать вышестоящей системе информацию о том, что содержимое было перелито в агрегат.

### 3 Проектирование информационного и программного обеспечения системы контроля перемещения объектов мостовыми кранами

#### 3.1 Концептуальное моделирование программного обеспечения

##### 3.1.1 Модель «Черного ящика»

Модель системы «черный ящик» с известными выходными и входными параметрами и неизвестным внутренним устройством. Позволяет получить представление о внутреннем устройстве. Модель разрабатываемой системы представлен на рис. 22.



Рис. 22. Модель черного ящика

Описание входных и выходных параметров системы представлено в табл. 2.

Таблица 2

Параметры системы

| Наименование                       | Описание                                                                                     |
|------------------------------------|----------------------------------------------------------------------------------------------|
| 1                                  | 2                                                                                            |
| Видеопотоки камер позиционирования | Входящий параметр, представляет собой видеопотоки с камер позиционирования по протоколу RTSP |



Продолжение табл. 2

| 1                                                | 2                                                                                                                                                            |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Видеопотоки ковшевых камер                       | Входящий параметр, представляет собой видеопотоки с ковшевых камер по протоколу RTSP                                                                         |
| Конфигурационные файлы                           | Входящий параметр, представляет собой набор конфигурационных файлов, описывающих параметры работы системы                                                    |
| Обученная модель детектора                       | Входящий параметр, представляющий собой файл в формате ONNX для детектора                                                                                    |
| Обученная модель классификатора                  | Входящий параметр, представляющий собой файл в формате ONNX для классификатора                                                                               |
| Данные крановых весов                            | Входящий параметр, представляющий собой словарь типа ключ-значение. Ключом является номер крана, а значением вес в захвате крана, при наличии крановых весов |
| Обученная модель распознавателя позиции по сцене | Входящий параметр, представляющий собой файл в формате ONNX для распознавателя позиции по наблюдаемой сцене                                                  |
| Состояние кранов цеха                            | Результат работы системы, который запрашивается системой производственного планирования                                                                      |
| Детальное состояние кранов цеха                  | Результат работы системы, который запрашивается для визуализации данных                                                                                      |
| Текущий кадр видеоканала                         | Результат работы системы, представляющий собой последний обработанный кадр видеоканала                                                                       |

### 3.1.2 Модель состава

Модель «черного ящика» отражает внешние свойства системы. Но у любой системы есть свой состав, ее отдельные части. При более детальном рассмотрении некоторые части системы могут быть в свою очередь разбиты на составные части и т. д., те части системы, которые рассматриваются как неделимые, будут называться элементами. Части системы, состоящие более чем из одного элемента, называются подсистемами. В результате получается модель состава системы, описывающая, из каких подсистем и элементов она состоит [26].

Модель состава разрабатываемой системы представлена на рис. 23.



Рис. 22. Модель состава

Краткое описание подсистем проектируемой модели состава представлена в табл. 3. Таблица включает в себя указанные выше подсистемы и их краткое описание в рамках проектируемой системы.

## Описание подсистем

| Наименование                                           | Описание                                                                                                                                                                                                                                                      |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1                                                      | 2                                                                                                                                                                                                                                                             |
| Подсистема получения данных крановых весов             | Задача подсистемы периодически запрашивать значения крановых весов у АСУ ТП                                                                                                                                                                                   |
| Подсистема обработки видео                             | Задача подсистемы устанавливать подключение к видеокамере по протоколу RTSP и получать от неё видеокadres. Выполнять их обработку при указании такого требования в конфигурации. Хранить последний кадр, переданный видеоканалом на распознавание             |
| Подсистема расчёта позиции по маркерам                 | Задача подсистемы заключается в определении местоположения мостового крана по наблюдаемым маркерам                                                                                                                                                            |
| Подсистема определения позиции по наблюдаемой сцене    | Задача подсистемы заключается в дублировании определения местоположения мостового крана. Эта подсистема определяет местоположение по наблюдаемой сцене                                                                                                        |
| Подсистема определения состояния захвата и сталь-ковша | Задачей подсистемы является определение наличия захвата в кадре и определении его состояния или состояния сталь-ковша в нём                                                                                                                                   |
| Подсистема объединения данных                          | Задачей этой подсистемы является объединение данных от подсистемы получения данных с крановых весов, подсистемы определения состояния захвата и сталь-ковша, а также подсистем определения местоположения мостового крана. В итоге получается состояние крана |

Продолжение табл. 3

| 1                                  | 2                                                                                                                                                                                                                                                                                                       |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Подсистема агрегации данных кранов | Задача подсистемы состоит в сборе состояний с нескольких кранов цеха и предоставлении для системы производственного планирования единого среза состояний мостовых кранов в цехе. Также подсистема выполняет сбор детальных состояний кранов для предоставления их модулю визуализации данных по запросу |
| Подсистема анализа данных крана    | Задача подсистемы состоит в том, чтобы анализировать объединённые данные о кране и на основании результата анализа дополнять его состояние                                                                                                                                                              |

### 3.1.3 Модель структуры

После этапа построения модели состава системы следует этап построения структурной модели. Структурную модель системы еще называют структурной схемой. На структурной схеме отражается состав системы и ее внутренние связи. Наряду с термином «связь» нередко употребляют термин «отношение». Модель структуры представлена на рис. 24.

В табл. 4 представлено описание связей подсистем модели структуры



Рис. 24. Модель структуры системы

Таблица 4

## Описание связей подсистем

| Наименование                               | Описание                                                                                                                                                                                                                                                  |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1                                          | 2                                                                                                                                                                                                                                                         |
| Подсистема получения данных крановых весов | Подсистема производит периодический опрос АСУ ТП и получает словарь типа ключ-значение. Из словаря извлекается значение веса для текущего крана. Если в словаре нет искомого ключа, то в подсистему объединения данных передаётся неопределённое значение |
| Подсистема обработки видео                 | Подсистема извлекает и обрабатывает кадры из видеопотоков сконфигурированных камер крана. Кадры, в зависимости от типа видеоканала или от его идентификатора, передаются в ту или иную подсистему распознавания                                           |

Продолжение табл.4

| 1                                                      | 2                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Подсистема расчёта позиции по маркерам                 | Подсистема получает кадр с видеоканала, предназначенного для позиционирования. С помощью сервиса распознавания маркеров получает набор распознанных 2D кодов на кадре. Рассчитывает по ним позицию крана. Полученная позиция передаётся подсистеме объединения данных |
| Подсистема определения позиции по наблюдаемой сцене    | Подсистема получает кадр с определённого видеоканала, обрабатывает его, используя сервис запуска нейронных сетей, распознанную позицию подсистема передаёт подсистеме объединения данных                                                                              |
| Подсистема определения состояния захвата и сталь-ковша | Подсистема получает кадр с видеоканала, предназначенного для детекции сталь-ковша. С помощью сервиса запуска нейронных сетей выполняется детекция и классификация захвата и сталь-ковша. Полученное состояние передаётся подсистеме объединения данных                |
| Подсистема объединения данных                          | Подсистема получает: вес, состояние захвата и сталь-ковша, позицию по маркерам, позицию по сцене. Объединяет данные в единый отчёт, выбирая одну из полученных позиций. В результате получается состояние крана                                                       |

Продолжение табл.4

| 1                                  | 2                                                                                                                                                                                                                                                                                                                                          |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Подсистема анализа данных крана    | Подсистема получает результат объединения информации о кране в единый отчёт. Дополняет его такой информацией как: действие захвата крана, наименование позиции сталь-ковша, признак наличия движения крана. Также в подсистеме определяется применяемый метод позиционирования на основании признака наличия распознанных маркеров в кадре |
| Подсистема агрегации данных кранов | Подсистема собирает состояния нескольких кранов в перечень состояний. Он представляет собой текущее состояние всех кранов в цехе. Тип собираемого состояния задаётся в конфигурационном файле                                                                                                                                              |

### 3.1.4 Модель «белого ящика»

Построенные выше модели «черного ящика» и структуры системы позволяют построить модель «белого ящика» [27], объединив входы и выходы модели «черного ящика» и связи между подсистемами модели структуры системы.

Модель «белого ящика» системы представлена на рис. 25.

В представленной модели видеокадры сконфигурированных видеоканалов захватываются и обрабатываются в подсистеме обработки видео. Параллельно происходит периодический опрос АСУ ТП для получения значения весов для крана, указанного в конфигурации.

После обработки видеокадра конкретного видеоканала, он сохраняется в память как текущий кадр и передаётся одной из подсистем для распознавания. Выбор подсистемы зависит как от идентификатора видеоканала, так и от его типа.

Данные весов и данные после одной из подсистем для распознавания поступают в подсистему объединения данных, где формируется состояние мостового крана.

Далее состояние анализируется и дополняется новыми данными о наличии движения, текущем действии захвата, наименованием позиции сталь-ковша.

В цехе может находиться несколько мостовых кранов. Для формирования единого среза состояния кранов цеха, подсистемой агрегации формируется перечень состояний кранов. Этот перечень может быть в дальнейшем запрошен системой производственного планирования. Также перечень детальных состояний кранов может быть в дальнейшем запрошен модулем визуализации данных системы.

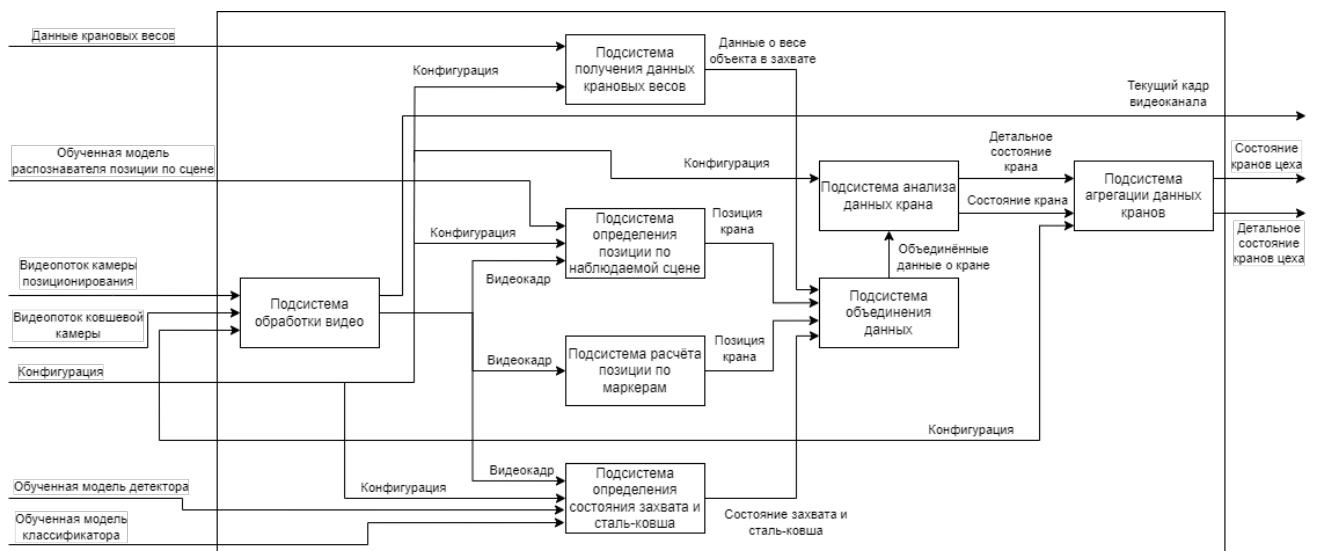


Рис. 25. Модель белого ящика

## 3.2 Проектирование информационного обеспечения

### 3.2.1 Моделирование потоков информации в проектируемой системе

Диаграмма потока данных — это графическое представление потока данных в информационной системе, обеспечивающее удобный способ описания передаваемой информации между частями системы [28].



На рис. 26 представлена обобщенная модель потоков данных, а на рис. 27 представлена её детализация.



Рис. 26. Обобщенная модель потоков данных

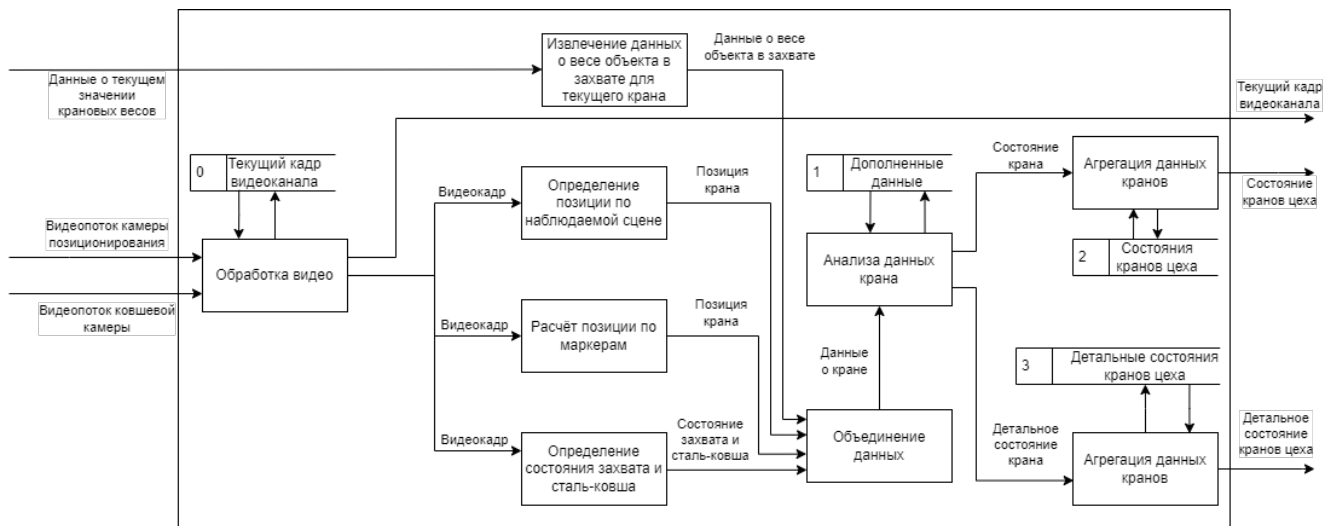


Рис. 27. Детализация диаграммы потоков данных

На вход системе подаются видеопотоки камер и текущие значения крановых весов.

Из значений крановых весов извлекается значение для крана, указанного в файле конфигурации. При невозможности получить значение, для объединения передаётся пустое значение.

После обработки кадра видеоканала, он сохраняется в память и отправляется на распознавание. Полученные после распознавания данные, передаются на объединение.

Объединённые данные анализируются, в следствие чего дополняются результатом их анализа. Сохраняются в памяти до следующей операции анализа. После чего они будут перезаписаны.

Агрегация состояний кранов и детальных состояний кранов происходит раздельно через равные периоды времени. Запущенные экземпляры подсистемы агрегации отличаются только конфигурацией. Для получения данных, отправляется запрос подсистеме анализа данных, которая, в свою очередь, извлекает из памяти результат последнего объединения и формирует либо обычное состояние, либо детальное. В зависимости от запроса.

После агрегации перечень полученных состояний помещается в память до последующего запроса.

Из памяти перечень извлекается при получении запроса. От системы производственного планирования, в случае состояний кранов цеха. Либо от модуля визуализации данных системы, в случае детальных состояний кранов цеха.

### 3.3 Проектирование программного обеспечения

#### 3.3.1 Выбор технологии проектирования, среды и языка программирования

##### 3.3.1.1 Выбор модели жизненного цикла

Жизненный цикл программного обеспечения (ПО) — период времени, который начинается с момента принятия решения о необходимости создания программного продукта и заканчивается в момент его полного изъятия из эксплуатации [29].

Среди моделей жизненного цикла программного обеспечения наиболее известны следующие:

- Каскадная модель;
- Итерационная модель;
- Инкрементная модель;
- Спиральная модель;
- Гибкая модель разработки Agile.

Исходя из требований заказчика к реализации программного продукта была выбрана гибкая модель разработки (рис. 28).

Agile — это итеративный подход к управлению проектами и разработке программного обеспечения, который помогает командам быстрее и с меньшими проблемами поставлять ценность клиентам. Вместо того чтобы выпускать весь продукт целиком, команда, следующая принципам Agile, выполняет работу в рамках небольших, но удобных инкрементов. Требования, планы и результаты оцениваются непрерывно, благодаря чему команды могут быстро реагировать на изменения [30].

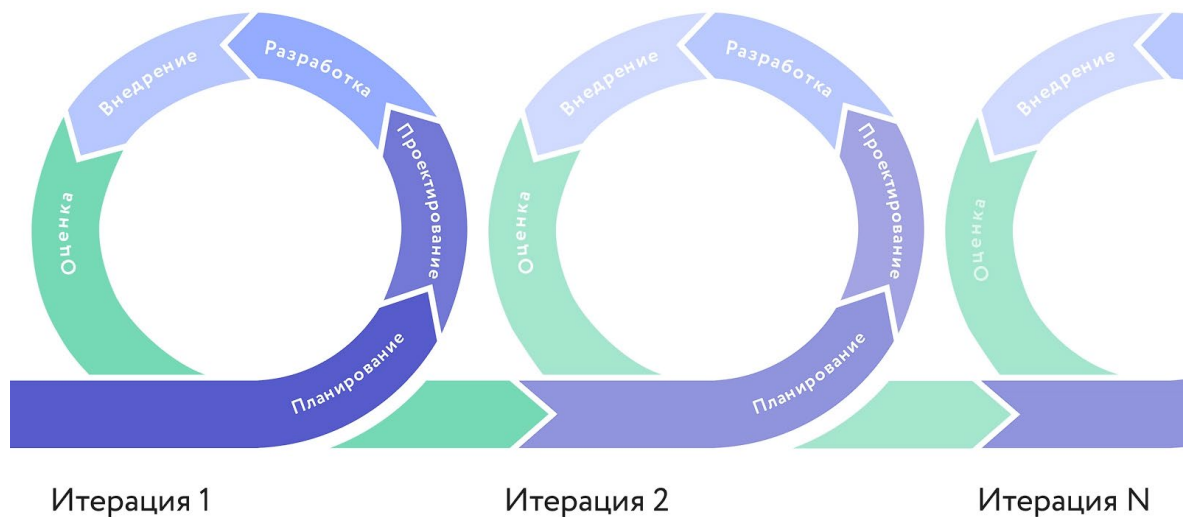


Рис. 28. Гибкая модель разработки

Выбор данной модели жизненного цикла для разработки системы обусловлен следующими причинами:

- Данный тип модели ЖЦ позволяет вносить изменения на каждом этапе проекта, адаптировать проект под требования владельца продукта, снижать финансовые риски и быстро запускать продукт.
- На каждом этапе разработки формируется законченный набор проектной документации, проверенный на полноту и согласованность.
- Гибкая модель предоставляет возможность начать разработку сразу после согласования бизнес-модели, общей стратегии и функциональных требований.

### 3.3.1.2 Выбор подхода к разработке

На текущий момент существует два основных подхода: структурный и объектно-ориентированный.

Сущность структурного подхода к разработке заключается в декомпозиции информационной системы на автоматизируемые функции: система разбивается на функциональные подсистемы, которые в свою очередь делятся на подфункции, подразделяемые на задачи и так далее. Процесс разбиения продолжается вплоть до конкретных процедур. При этом автоматизируемая система сохраняет целостное представление, в котором все составляющие компоненты взаимосвязаны [31].

Объектно-ориентированное программирование (ООП) — методология программирования, основанная на представлении программы в виде совокупности объектов, каждый из которых является экземпляром определённого класса, а классы образуют иерархию наследования [32].

В качестве подхода к разработке программного обеспечения было выбрано Объектно-ориентированное программирование (ООП). Выбор данного подхода обусловлен двумя факторами:

- относительная простота разработки программного обеспечения и дальнейшая модернизация,

- C# является объектно-ориентированным языком программирования, что позволит реализовать данный подход без проблем, также, данный подход позволит достаточно легко и точно декомпозировать объекты предметной области.

В качестве стандарта проектирования и написания программной документации и спецификации был выбран язык моделирования UML.

UML – унифицированный язык моделирования. Он представляет собой систему обозначений, которую можно применять для объектно-ориентированного анализа и проектирования [33].

Его можно использовать для визуализации, спецификации, конструирования и документирования программных систем.

UML обладает следующим преимуществами:

- UML объектно-ориентирован, в результате чего методы описания результатов анализа и проектирования семантически близки к методам программирования на современных объектно-ориентированных языках.
- UML позволяет описать систему практически со всех возможных точек зрения и в различных аспектах поведения системы.
- Диаграммы UML сравнительно просты для чтения после достаточно быстрого ознакомления с его синтаксисом.
- UML получил широкое распространение и быстро развивается.

### 3.3.1.3 Выбор среды и языка программирования

Исходя из требований и сформированных задач, выбранного подхода к разработке программного обеспечения необходимо выбрать набор инструментов, с помощью которых будет создаваться предлагаемая информационная система.

Для проектирования данной системы был выбран следующий стек технологий:

- для реализации программного обеспечения использовался язык программирования высокого уровня C#, платформа ASP.NET;
- пользовательский интерфейс реализован в рамках отдельного модуля;
- при проектировании ПО использовался объектный подход и язык UML;
- среда разработки Microsoft Visual Studio 2022;
- ввиду того, что за каждый кран отвечает один экземпляр системы, а для агрегации данных кранов используется два экземпляра подсистемы агрегации состояний, была выбрана система оркестрации – Nomad;
- для слежения за состоянием сервисов и обеспечения средства обнаружения сервисов, был выбран service discovery Consul;
- для сбора метрик был выбран сервис Prometheus;
- для визуализации метрик и настройки оповещений был выбран сервис Grafana;
- в качестве единой точки сбора логов множества сервисов, был выбран сервис Loki;
- в качестве прокси-сервиса был выбран Nginx;
- для автоматизации настройки Nginx была выбрана утилита Consul template.

Для реализации программного обеспечения использовался язык C#, работающий на платформе .Net. C# — объектно-ориентированный язык программирования [34].

Пользовательский интерфейс реализован в рамках отдельного программного модуля. Он выполнен в виде клиент-серверного приложения. Где в качестве сервера используется платформа ASP.NET, а в качестве клиента выступает среда выполнения .NET на основе WebAssembly – WASM Blazor [35].

Разработка производилась в среде Microsoft Visual Studio 2022. Microsoft Visual Studio — линейка продуктов компании Microsoft, включающих

интегрированную среду разработки программного обеспечения и ряд других инструментальных средств [36].

Ввиду множества сервисов, необходимо средство для управления их запуском, остановкой, перезапуском при сбое. Таким средством является оркестратор Nomad [37]. Развёртывание сервисов в контейнерах для данного проекта исключается ввиду того, что есть сервисы, которые зависят от аппаратной части сервера. Например, сервис запуска нейронных сетей требует наличия GPU.

Так как кранов в цехе может быть больше 10, то ручное задание сетевого порта каждому сервису крана не представляется целесообразным. Nomad имеет возможность задания динамического порта запускаемому им приложению. В таком случае, становится неочевидным, по какому порту должен обращаться агрегатор, чтобы получить состояние конкретного крана. Для того, чтобы найти все запущенные сервисы кранов, а следовательно, и их сетевые порты, необходимо использовать механизм обнаружения сервисов. Таким средством является Consul [38].

Ввиду того, что сигнал с камеры может оборваться, то необходимо продумать механизм оповещения ответственных за камеры лиц. Для этого, необходимо либо вручную реализовывать работу с оповещением и создавать целую подсистему, либо воспользоваться готовой.

Оповещения можно реализовать с помощью связки двух сервисов Grafana и Prometheus.

Prometheus – это система мониторинга с многомерной моделью данных, гибким языком запросов, эффективной базой данных временных рядов [39].

Grafana - это платформа с открытым исходным кодом для визуализации, анализа данных, оповещения о событиях на основании анализа данных [40].

Так как предполагается запуск множества сервисов кранов, а цехов может быть несколько, то необходим инструмент для сбора логируемой информации в единой точке. Таким инструментом является сервис Loki. Это система агрегации и хранения журналов работы сервисов [41].

Для упрощения маршрутизации запросов от модуля визуализации данных к серверам, обслуживающих цеха применяется Nginx. Это программное обеспечение с открытым исходным кодом для создания легкого и мощного веб-сервера. Также его можно использовать в качестве почтового или обратного прокси-сервера [42].

Ввиду динамического задания портов, необходимо при каждом перезапуске какого-либо сервиса обновлять конфигурацию Nginx. Это можно автоматизировать с помощью утилиты Consul template.

Так же для проектирования ПО было использовано следующее CASE-средство - Microsoft Visio. Это векторный графический редактор, редактор диаграмм и блок-схем для Windows.

Разработка программной документации проводилась в соответствии с Единой системой программной документации (ЕСПД), позволяющая облегчить процесс сопровождения программного решения как самим создателем ПО, так и сторонним лицам, сопровождающим разработанный программный продукт.

### 3.3.2 Разработка логической модели

#### 3.3.2.1 Диаграмма вариантов использования

Одним из стандартных элементов языка UML является диаграмма вариантов использования, которая позволяет наглядно представить ожидаемое поведение системы. Основными понятиями диаграмм вариантов использования являются: действующее лицо, вариант использования, связь. Диаграмма вариантов может дополняться пояснительным текстом, который раскрывает смысл или семантику составляющих ее компонентов. Такой пояснительный текст получил название примечания или сценария [43].

Отдельный вариант использования обозначается на диаграмме эллипсом, внутри которого содержится его краткое название или имя в форме глагола с пояснительными словами [44].



Действующее лицо (actor) – внешняя по отношению к разрабатываемому программному обеспечению сущность, которая взаимодействует с ним с целью получения или предоставления какой-либо информации. Действующими лицами могут быть пользователи, другое программное обеспечение или какие-либо технические средства, взаимодействующие с разрабатываемым программным обеспечением.

Вариант использования - некоторая очевидная для действующего лица процедура, решающая его конкретную задачу. Все варианты использования, так или иначе, связаны с требованиями к функциональности разрабатываемой системы и могут сильно отличаться по объему выполняемой работы.

Связь - взаимодействие действующих лиц и соответствующих вариантов использования.

Варианты использования также могут быть связаны между собой. При этом фиксируют связи использования и расширения.

Диаграмма вариантов использования для программного обеспечения системы контроля перемещения мостовых кранов представлена на рис. 29.

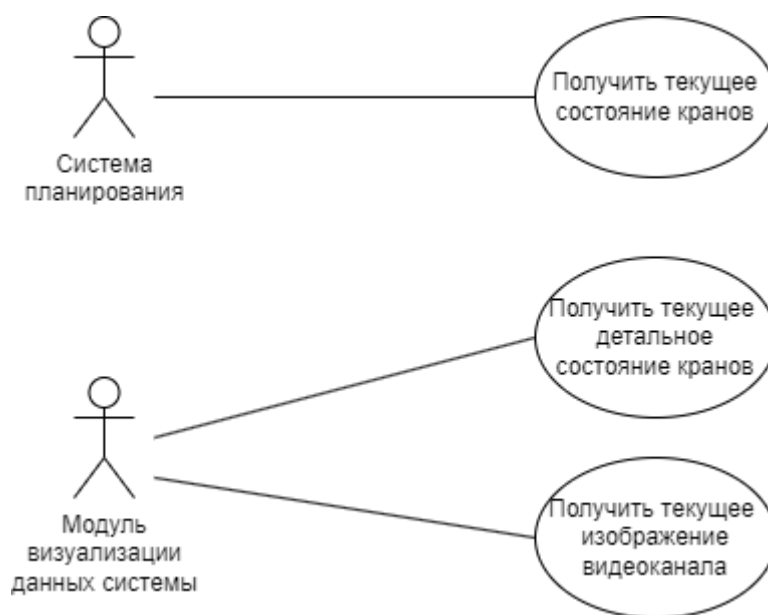


Рис. 29. Диаграмма вариантов использования

У данной диаграммы два действующих лица – «Система планирования» и «Модуль визуализации данных системы». Оба действующих лица получают состояния кранов с одним отличием – модуль получает детальные состояния кранов. Также модуль получает текущее изображение видеоканала.

В табл. 5 представлено краткое описание варианта «Получить текущее состояние кранов».

Таблица 5

Краткое описание варианта «Получить текущее состояние кранов»

|                  |                                                                                  |
|------------------|----------------------------------------------------------------------------------|
| Название         | Получить текущее состояние кранов                                                |
| Назначение       | Выполняет извлечение из памяти копии перечня состояний кранов цеха               |
| Действующие лица | Система планирования                                                             |
| Краткое описание | По запросу системы планирования отдаётся перечень состояний мостовых кранов цеха |

Типичный ход событий для данного варианта представлен в табл. 6.

Таблица 6

Типичный ход событий варианта «Получить текущее состояние кранов»

|                                                                                     |                                                                 |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Действие исполнителя                                                                | Отклик системы                                                  |
| 1. исполнение периодического запроса для получения текущего состояния кранов в цехе | 2. выгрузка из памяти и отдача текущего состояния кранов в цехе |

В табл. 7 представлено краткое описание варианта «Получить текущее детальное состояние кранов».

Таблица 7

Краткое описание варианта «Получить текущее детальное состояние кранов»

|                  |                                                                                           |
|------------------|-------------------------------------------------------------------------------------------|
| Название         | Получить текущее детальное состояние кранов                                               |
| Назначение       | Выполняет извлечение из памяти копии перечня детальных состояний кранов цеха              |
| Действующие лица | Модуль визуализации данных системы                                                        |
| Краткое описание | По запросу модуля визуализации отдаётся перечень детальных состояний мостовых кранов цеха |

Типичный ход событий для данного варианта представлен в табл. 8.

Таблица 8

Типичный ход событий варианта «Получить текущее детальное состояние кранов»

|                                                                                                                              |                                                                            |
|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Действие исполнителя                                                                                                         | Отклик системы                                                             |
| 1. в модуле визуализации выбран цех, запущен периодический запроса для получения текущего детального состояния кранов в цехе | 2. выгрузка из памяти и отдача текущего детального состояния кранов в цехе |

В табл. 9 представлено краткое описание варианта «Получить текущее изображение видеоканала».

Альтернатива:

Если в модуле визуализации цех не выбран, то в модуле не будет запущен механизм периодического запроса детального состояния.

Таблица 9

Краткое описание варианта «Получить текущее изображение  
видеоканала»

|                  |                                                                                          |
|------------------|------------------------------------------------------------------------------------------|
| Название         | Получить текущее изображение видеоканала                                                 |
| Назначение       | Выполняет извлечение из памяти копии текущего кадра и отдаёт его в теле ответа на запрос |
| Действующие лица | Модуль визуализации данных системы                                                       |
| Краткое описание | По запросу модуля визуализации отдаётся текущий кадр указанного в запросе видеоканала    |

Типичный ход событий для данного варианта представлен в табл. 10.

Таблица 10

Типичный ход событий варианта «Получить текущее изображение  
видеоканала»

|                                                                                       |                                            |
|---------------------------------------------------------------------------------------|--------------------------------------------|
| Действие исполнителя                                                                  | Отклик системы                             |
| 1. выбран конкретный кран, запущен механизм получения кадров определённой видеокамеры | 2. выдача кадра указанной в запросе камеры |
| 3. исполнение периодического запроса для получения текущего кадра камеры              | 4. выдача кадра указанной в запросе камеры |

Альтернатива:

Если в модуле визуализации не выбран определённый кран, то в модуле не будет запущен механизм получения кадров.

### 3.3.2.2 Построение контекстной диаграммы классов

Диаграммы классов - центральное звено объектно-ориентированных методов разработки программного обеспечения, поэтому все существующие методы используют диаграммы классов в одной из известных нотаций. Однако в основном диаграммы классов в этих методах применяют на этапе проектирования, для того чтобы показать особенности построения конкретных классов [44].

Концептуальные модели в соответствии с определением оперируют понятиями предметной области, атрибутами этих понятий и отношениями между ними. Понятию в предметной области разрабатываемого программного обеспечения могут соответствовать как материальные предметы, так и абстракции, которые применяют специалисты предметной области.

Основным понятиям в модели ставятся в соответствие классы. Класс при этом традиционно понимают, как совокупность общих признаков заданной группы объектов предметной области. В соответствии с этим определением на диаграмме классов каждому классу соответствует группа объектов, общие признаки которых и фиксирует класс.

Контекстная диаграмма классов программного обеспечения системы контроля перемещения объектов мостовыми кранами представлена на рис. 30.

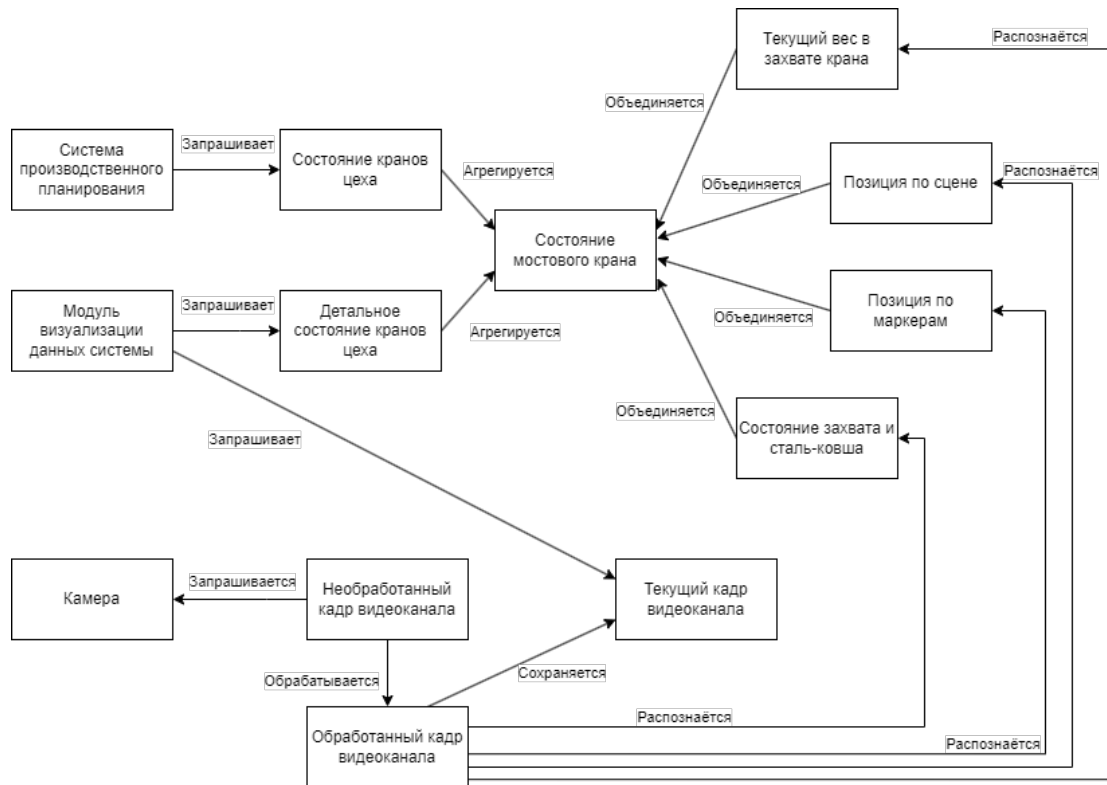


Рис. 30. Контекстная диаграмма классов

### 3.3.2.3 Диаграмма пакетов

Пакетом при объектном подходе называется совокупность описаний классов и других программных ресурсов, в том числе и самих пакетов. Объединение в пакеты используют для удобства создания больших проектов, количество классов в которых велико. При этом в один пакет обычно собирают классы и другие ресурсы одинакового назначения [44].

Диаграмма пакетов показывает, из каких частей состоит проектируемая программная система, и как эти части связаны друг с другом.

Связь между пакетами определяется внешними связями классов и других ресурсов, объединенных в пакет.

Спецификация языка UML не налагает жёстких ограничений на то, как модель разбивается на пакеты, существует множество способов организации по функциональности, виду модели или по любому другому признаку. Допускается также иерархическое вкладывание одних UML-пакетов в другие,

при этом вложенный пакет имеет полный доступ к содержимому своего контейнера и считается его частью [45].

В разрабатываемом программном продукте имеется подсистема агрегации состояний кранов. Она выполнена в виде отдельного приложения ввиду того, что количество подсистем, обслуживающих мостовые краны цеха может масштабироваться.

Описание пакетов для подсистемы агрегации данных представлено в табл. 11. Диаграмма пакетов представлена на рис. 31.

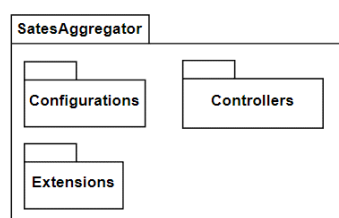


Рис. 31. Диаграмма пакетов сервиса агрегации состояний

Таблица 11

Описание пакетов сервиса агрегации состояний

| Пакет          | Описание                                                                                                                                                                      |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1              | 2                                                                                                                                                                             |
| Extensions     | Пакет, содержащий статический класс, который содержит методы расширения. Они позволяют добавлять новые методы в уже существующие типы без создания нового производного класса |
| Controllers    | Пакет, содержащий обработчик HTTP запросов на получения состояния кранов цеха                                                                                                 |
| Configurations | Пакет, содержащий класс, описывающий структуру конфигурационного файла приложения                                                                                             |

Продолжение табл. 11

| 1               | 2                                                                                     |
|-----------------|---------------------------------------------------------------------------------------|
| SatesAggregator | Основной пакет, содержащий классы, которые реализуют основную логику работы программы |

Пакет «Extensions» содержит один статический класс «IServiceCollectionExtensions», который добавляет новые методы для реализации интерфейса «IServiceCollection».

Пакет «Controllers» содержит один класс «StatesController». Этот класс реализует логику обработки запроса на получение текущего состояния мостовых кранов в цехе.

Пакет «Configurations» также содержит один класс «AggregationConfiguration». Этот класс описывает файл конфигурации приложения.

Пакет «SatesAggregator» содержит классы, реализующие основную логику работы приложения. Детальная диаграмма пакета представлена на рис. 32. В табл. 12 представлено описание классов.

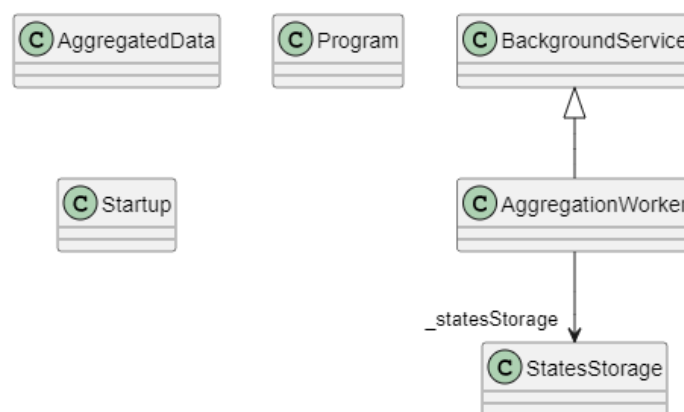


Рис. 32. Диаграмма классов пакета «SatesAggregator»



Описание классов пакета «SatesAggregator»

| Класс             | Описание                                                                                                             |
|-------------------|----------------------------------------------------------------------------------------------------------------------|
| AggregatedData    | Класс, представляющий собой информацию о запросе состояния крана и состояние крана в формате JSON                    |
| AggregationWorker | Класс, реализующий фоновый процесс, который периодически опрашивает сервисы кранов цеха, зарегистрированные в Consul |
| Program           | Класс, представляющий собой точку входа в приложение                                                                 |
| Startup           | Класс, производящий настройку сервисов в контейнере внедрения зависимостей ASP.NET приложения                        |
| StatesStorage     | Класс, реализующий сохранение и извлечение из памяти перечня состояний кранов цеха                                   |

Как было сказано ранее, количество подсистем, обслуживающих мостовые краны цеха может масштабироваться.

Ввиду этого, описанные ранее подсистемы, за исключением подсистемы «Подсистема агрегации данных кранов» были объединены в сервис крана.

Этот сервис масштабируемый. Другими словами, на один мостовой кран запускается по одному сервису. Один сервис отвечает только за один кран.

Описание пакетов для сервиса крана представлено в табл. 13. Диаграмма пакетов представлена на рис. 33.

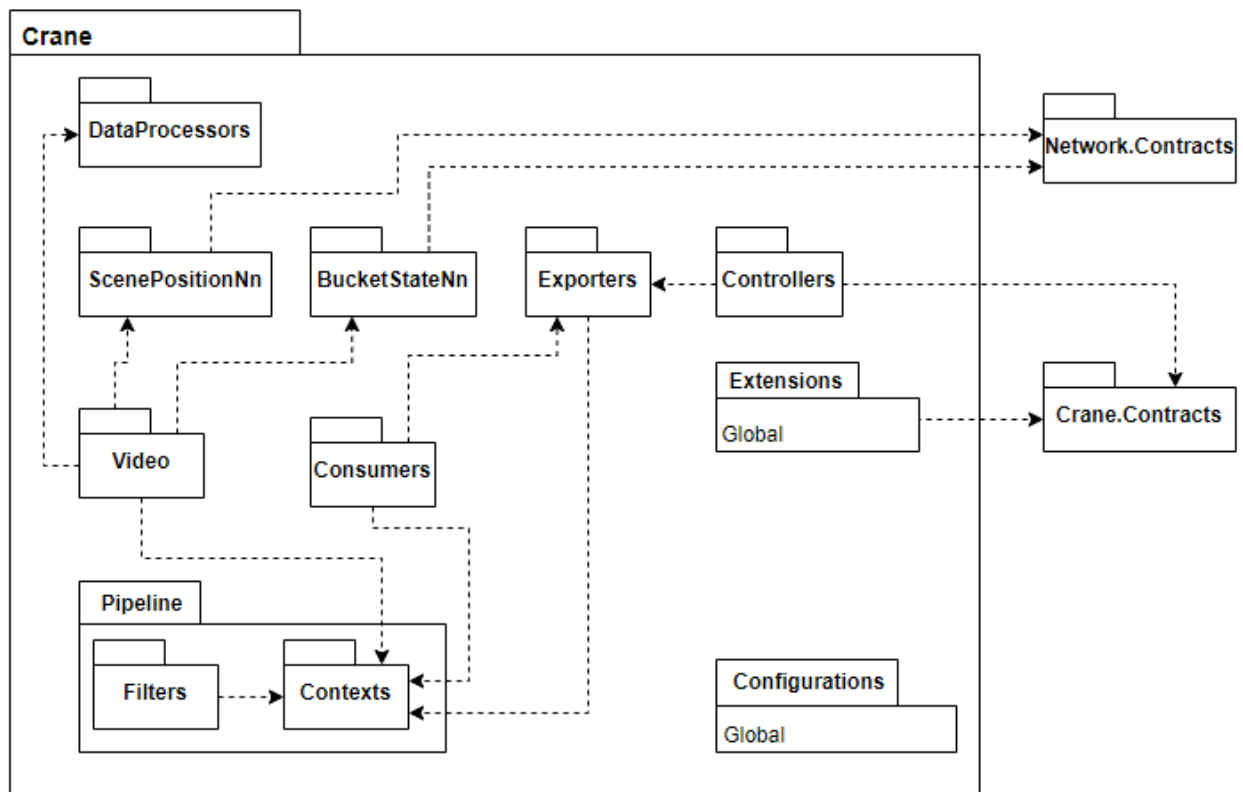


Рис. 33. Диаграмма пакетов сервиса крана

Таблица 13

## Описание пакетов сервиса крана

| Пакет           | Описание                                                                                      |
|-----------------|-----------------------------------------------------------------------------------------------|
| 1               | 2                                                                                             |
| Crane           | Основной пакет сервиса крана                                                                  |
| DataProcessors  | Пакет, реализующий классы, которые интерпретируют получаемые данные после распознавания кадра |
| ScenePositionNn | Пакет, классы которого, реализуют клиент распознавателя позиции по наблюдаемой сцене          |
| BucketStateNn   | Пакет, классы которого, реализуют клиент распознавателя захвата, сталь-ковша, их состояния    |

Продолжение табл. 13

| 1                 | 2                                                                                                                                                                              |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Network.Contracts | Пакет, классы которого описывают запрос к сервису запуска нейронной сети или его ответ                                                                                         |
| Video             | Пакет, классы которого реализуют работу с получаемыми от видеоканала кадрами                                                                                                   |
| Consumers         | Пакет, классы которого являются получателями сообщения, распространяемого в сервисе с помощью паттерна «Медиатор»                                                              |
| Pipeline.Filters  | Подпакет, классы которого реализуют фильтры для паттерна «Каналы и фильтры»                                                                                                    |
| Pipeline.Contexts | Подпакет, интерфейсы которого описывают контексты сообщений как для паттерна «Каналы и фильтры», так и для паттерна «Медиатор»                                                 |
| Exporters         | Пакет, классы которого предназначены для экспорта данных                                                                                                                       |
| Controllers       | Пакет, классы которого отвечают на HTTP запросы                                                                                                                                |
| Extensions        | Пакет, содержащий статические классы, которые содержат методы расширения. Они позволяют добавлять новые методы в уже существующие типы без создания нового производного класса |
| Configurations    | Пакет, классы которого описывают конфигурацию сервиса крана                                                                                                                    |
| Crane.Contracts   | Пакет, классы которого описывают ответ на запрос состояния или детального состояния крана                                                                                      |

Пакет «Crane» содержит классы, связанные как с запуском ASP.NET платформы, так и вспомогательные. Детальная диаграмма пакета представлена на рис. 34. В табл. 14 представлено описание классов.

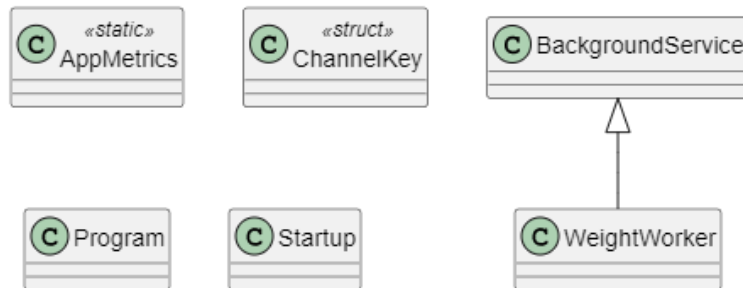


Рис. 34. Диаграмма классов пакета «Crane»

Таблица 14

Описание классов пакета «Crane»

| Класс        | Описание                                                                                                           |
|--------------|--------------------------------------------------------------------------------------------------------------------|
| AppMetrics   | Статический класс, содержащий набор статических полей, обозначающих ту или иную метрику приложения                 |
| ChannelKey   | Структура, идентифицирующая видеоканал                                                                             |
| Program      | Точка входа в программу                                                                                            |
| Startup      | Класс, производящий настройку сервисов в контейнере внедрения зависимостей ASP.NET приложения                      |
| WeightWorker | Класс, реализующий фоновый процесс, который периодически опрашивает АСУ ТП и извлекает вес объекта в захвате крана |

Пакет «DataProcessors» содержит классы, которые интерпретируют получаемые данные после распознавания кадра распознавателем позиции по маркерам или распознавателем состояния захвата и сталь-ковша. Детальная

диаграмма пакета представлена на рис. 35. В табл. 15 представлено описание классов.

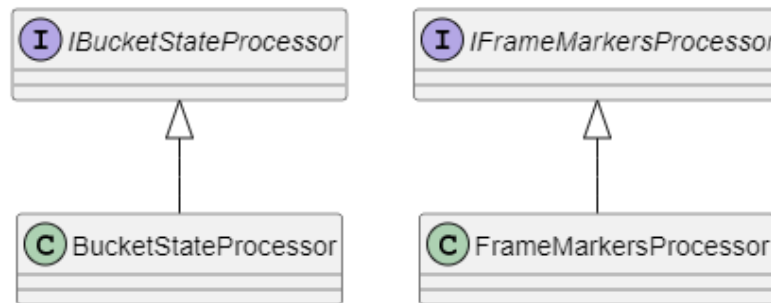


Рис. 35. Диаграмма классов пакета «DataProcessors»

Таблица 15

Описание классов пакета «DataProcessors»

| Класс                  | Описание                                                                              |
|------------------------|---------------------------------------------------------------------------------------|
| IBucketStateProcessor  | Интерфейс, описывающий обработчик кадров камеры слежения за сталь-ковшом              |
| BucketStateProcessor   | Класс, реализующий IBucketStateProcessor. Текст этого класса представлен на рис. П2.3 |
| IFrameMarkersProcessor | Интерфейс, описывающий обработчик кадров камеры позиционирования                      |
| FrameMarkersProcessor  | Класс, реализующий IFrameMarkersProcessor. Его текст представлен на рис. П2.2         |

Пакет «ScenePositionNn» содержит набор классов и интерфейсов, реализующих клиент сервиса запуска нейронных сетей, который выполняет распознавание позиции по наблюдаемой сцене. Диаграмма классов пакета представлена на рис. 36. В табл. 16 представлено описание классов.

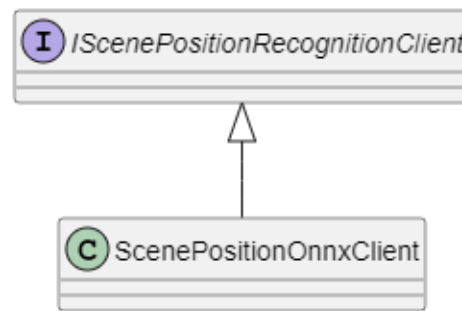


Рис. 36. Диаграмма классов пакета «ScenePositionNn»

Таблица 16

Описание классов пакета «ScenePositionNn»

| Класс                           | Описание                                                       |
|---------------------------------|----------------------------------------------------------------|
| IScenePositionRecognitionClient | Интерфейс, клиента распознавателя позиции по наблюдаемой сцене |
| ScenePositionOnnxClient         | Класс, реализующий IScenePositionRecognitionClient             |

Пакет «BucketStateNn» содержит набор классов и интерфейсов, реализующих клиент сервиса запуска нейронных сетей, который выполняет детекцию и классификацию сталь-ковша на кадре. Диаграмма классов пакета представлена на рис. 37. В табл. 17 представлено описание классов.

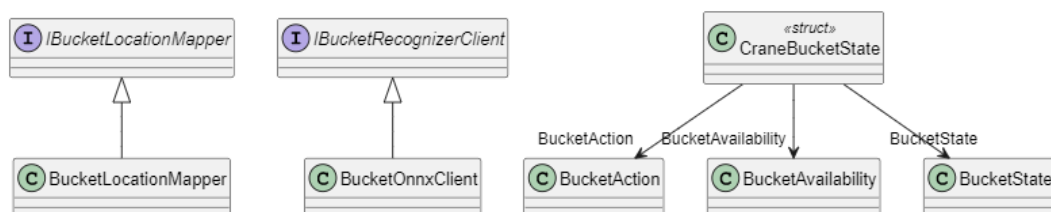


Рис. 37. Диаграмма классов пакета «BucketStateNn»

Пакет «Network.Contracts» содержит набор классов, описывающих протокол общения между сервисом крана и сервисом запуска нейронной сети. Диаграмма классов пакета представлена на рис. 38. В табл. 18 представлено описание классов.

Описание классов пакета «DataProcessors»

| Класс                   | Описание                                                                              |
|-------------------------|---------------------------------------------------------------------------------------|
| IBucketLocationMapper   | Интерфейс, описывающий обработчик наименования текущей позиции ковша на крановом пути |
| BucketLocationMapper    | Класс, реализующий IBucketLocationMapper                                              |
| IBucketRecognizerClient | Интерфейс, клиента распознавателя                                                     |
| BucketOnnxClient        | Класс, реализующий IBucketRecognizerClient                                            |
| CraneBucketState        | Структура, описывающая состояние сталь-ковша                                          |

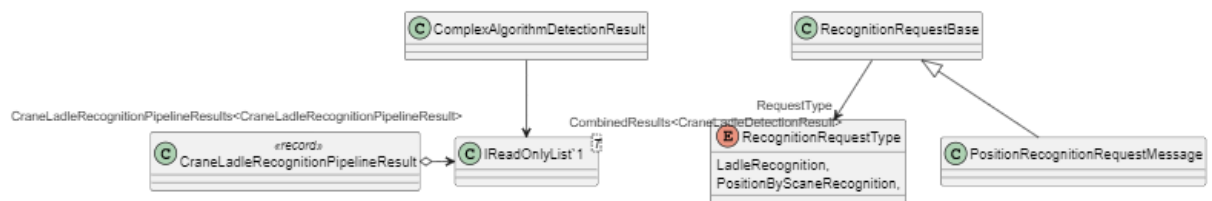


Рис. 38. Диаграмма классов пакета «Network.Contracts»

Описание классов пакета «Network.Contracts»

| Класс                             | Описание                                                                                                                         |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| 1                                 | 2                                                                                                                                |
| RecognitionRequestType            | Тип сообщения на распознавание. Предоставляет два возможных значения: распознавание сталь-ковша и распознавание позиции по сцене |
| ComplexAlgorithmDetectionResult   | Класс, реализующий ответ, содержащий несколько распознанных сталь-ковшей                                                         |
| PositionRecognitionRequestMessage | Класс, реализующий сообщение на распознавание позиции по наблюдаемой сцене                                                       |

| 1                                   | 2                                                     |
|-------------------------------------|-------------------------------------------------------|
| CraneLadleRecognitionPipelineResult | Класс, реализующий распознанный стальной ковш         |
| RecognitionRequestBase              | Базовый класс, реализующий сообщение на распознавание |

Пакет «Video» содержит набор классов, реализующий работу с кадрами. Диаграмма классов пакета представлена на рис. 39. В табл. 19 представлено описание классов.

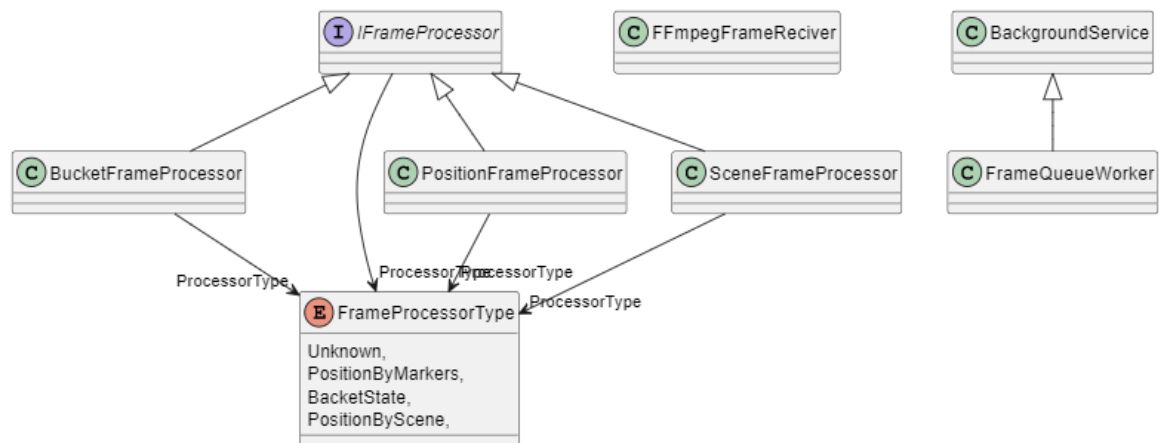


Рис. 39. Диаграмма классов пакета «Video»

Таблица 19

Описание классов пакета «Network.Contracts»

| Класс                | Описание                                                                                                                                |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 1                    | 2                                                                                                                                       |
| IFrameProcessor      | Интерфейс, описывающий обработчик кадра                                                                                                 |
| BucketFrameProcessor | Класс, реализующий обработчик кадра, который распознаёт состояние захвата и стальной ковша. Текст этого класса представлен на рис. П2.6 |



Продолжение табл. 19

| 1                      | 2                                                                                                                                                                                                           |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PositionFrameProcessor | Класс, реализующий обработчик кадра, который распознаёт позицию по маркерам. Текст этого класса представлен на рис. П2.5                                                                                    |
| SceneFrameProcessor    | Класс, реализующий обработчик кадра, который распознаёт позицию по наблюдаемой сцене                                                                                                                        |
| FrameProcessorType     | Тип обработчика кадров                                                                                                                                                                                      |
| FFmpegFrameReciver     | Класс, реализующий получение кадров от камер. И скалывающий их в очередь, в которой слишком старые кадры удаляются                                                                                          |
| FrameQueueWorker       | Класс, реализующий фоновый процесс, который запускает несколько параллельных подпроцессов. Каждый подпроцесс периодически забирает кадр из очереди класса FFmpegFrameReciver и рассылает его через медиатор |

Пакет «Consumers» содержит набор классов, реализующий работу с кадрами. Диаграмма классов пакета представлена на рис. 40. В табл. 20 представлено описание классов.

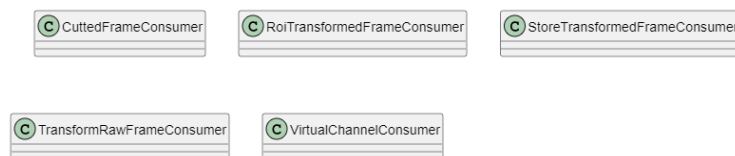


Рис. 40. Диаграмма классов пакета «Consumers»

Описание классов пакета «Consumers»

| Класс                         | Описание                                                                                                                                                                           |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CuttedFrameConsumer           | Класс, реализующий получение обрезанного кадра. В зависимости от типа и идентификатора канала, кадр передаётся определённому обработчику кадров                                    |
| RoiTransformedFrameConsumer   | Класс, реализующий получение преобразованного кадра и обрезание его по области ROI. Далее кадр рассылается через медиатор                                                          |
| StoreTransformedFrameConsumer | Класс, реализующий получение преобразованного кадра. Полученный кадр сохраняется в память                                                                                          |
| TransformRawFrameConsumer     | Класс, реализующий получение кадра от FrameQueueWorker. Над кадром выполняются преобразования. Например, убираются геометрические искажения. Далее кадр рассылается через медиатор |
| VirtualChannelConsumer        | Класс, реализующий получение преобразованного кадра и формирующий виртуальный видеоканал. Далее кадр рассылается через медиатор                                                    |

Подпакет «Pipeline.Contexts» содержит набор интерфейсов, которые описывают контексты сообщений как для паттерна «Каналы и фильтры», так и для паттерна «Медиатор». Диаграмма классов подпакета представлена на рис. 41. В табл. 21 представлено описание классов.

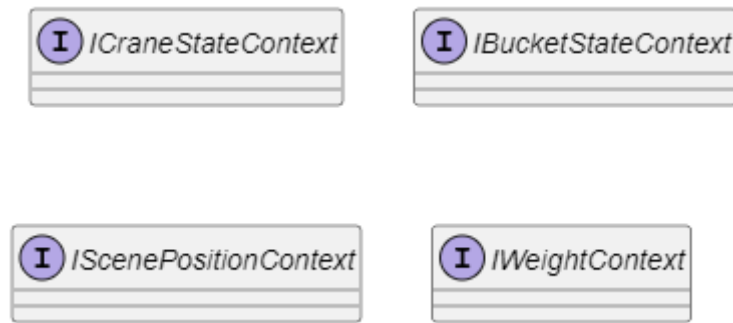


Рис. 41. Диаграмма классов подпакета «Pipeline.Contexts»

Таблица 21

Описание классов пакета «Consumers»

| Класс                 | Описание                                                        |
|-----------------------|-----------------------------------------------------------------|
| ICraneStateContext    | Интерфейс, описывающий контекст состояния крана                 |
| IBucketStateContext   | Интерфейс, описывающий контекст состояния сталь-ковша и захвата |
| IScenePositionContext | Интерфейс, описывающий контекст позиции по наблюдаемой сцене    |
| IWeightContext        | Интерфейс, описывающий контекст веса объекта в захвате крана    |

Подпакет «Pipeline.Filters» содержит классы которые реализуют фильтры для паттерна «Каналы и фильтры». Диаграмма классов подпакета представлена на рис. 42. В табл. 22 представлено описание классов.



Рис. 42. Диаграмма классов подпакета «Pipeline.Filters»

Описание классов пакета «Pipeline.Filters»

| Класс                    | Описание                                                                                                                                                                                                 |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MergePipesContextsFilter | Класс, реализующий фильтр, который объединяет такие контексты как: IBucketStateContext, IScenePositionContext, IWeightContext в контекст ICraneStateContext. Текст этого класса представлне на рис. П2.4 |
| OutputDataFilter         | Класс, получающий сообщения с типом контекста ICraneStateContext и дополняющий этот контекст новыми данными, на основании анализа полученных                                                             |
| MovingFilter             | Класс, реализующий фильтр контекста ICraneStateContext. Этот фильтр определяет наличие движения мостового крана                                                                                          |

Пакет классов «Exporters» содержит классы, выполняющие сохранение состояния крана или изображения камеры. Диаграмма классов пакета представлена на рис. 43. В табл. 23 представлено описание классов.



Рис. 43. Диаграмма классов пакета «Exporters»

Описание классов пакета «Exporters»

| Класс                      | Описание                                                            |
|----------------------------|---------------------------------------------------------------------|
| InMemoryCraneStateExporter | Класс, реализующий хранилище текущего состояния мостового крана     |
| BucketImageStorage         | Класс, реализующий хранилище текущего кадра ковшевой камеры         |
| PosImageStorage            | Класс, реализующий хранилище текущего кадра камеры позиционирования |

Пакет классов «Controllers» содержит классы, отвечающие на HTTP запросы. Диаграмма классов пакета представлена на рис. 44. В табл. 24 представлено описание классов.

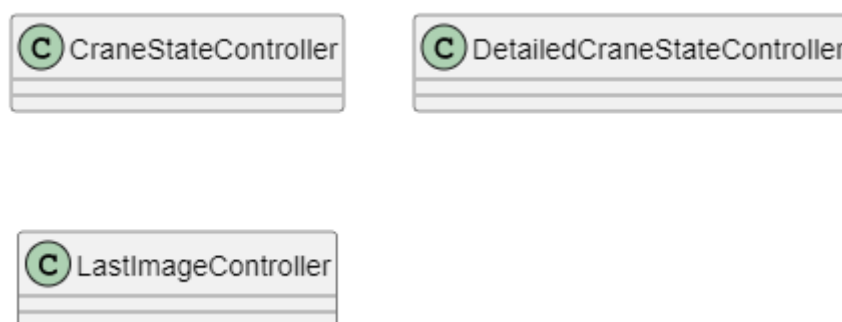


Рис. 44. Диаграмма классов пакета «Controllers»

Описание классов пакета «Controllers»

| Класс                        | Описание                                                                            |
|------------------------------|-------------------------------------------------------------------------------------|
| 1                            | 2                                                                                   |
| CraneStateController         | Класс, реализующий обработчик HTTP запросов на получение состояния крана            |
| DetailedCraneStateController | Класс, реализующий обработчик HTTP запросов на получение детального состояния крана |

Продолжение табл. 24

| 1                   | 2                                                                                         |
|---------------------|-------------------------------------------------------------------------------------------|
| LastImageController | Класс, реализующий обработчик HTTP запросов на получение текущего изображения видеоканала |

Глобальный пакет классов «Extensions» содержит классы, расширяющие функционал существующих типов. Диаграмма классов пакета представлена на рис. 45. В табл. 25 представлено описание классов.

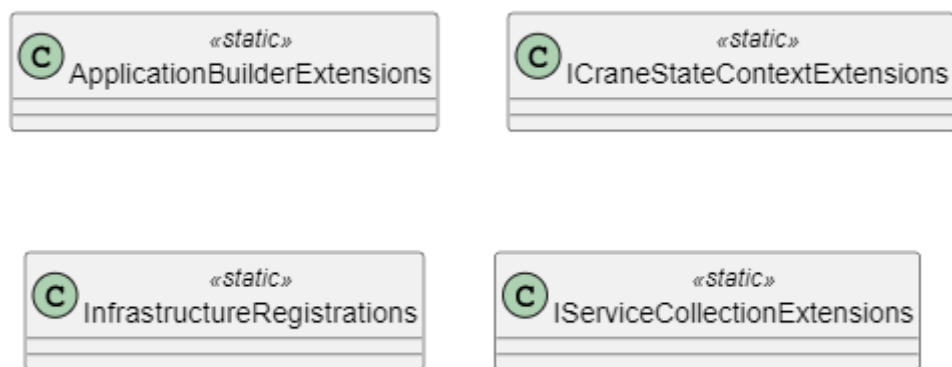


Рис. 45. Диаграмма классов пакета «Extensions»

Таблица 25

Описание классов пакета «Extensions»

| Класс                        | Описание                                                                                                                                                     |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1                            | 2                                                                                                                                                            |
| ApplicationBuilderExtensions | Статический класс, расширяющий тип ApplicationBuilder                                                                                                        |
| ICraneStateContextExtensions | Статический класс, расширяющий тип ICraneStateContext                                                                                                        |
| InfrastructureRegistrations  | Статический класс, расширяющий тип IServiceCollection. Расширение специализировано на регистрации в контейнере внедрения зависимостей инфраструктуры сервиса |

Продолжение табл. 25

| 1                            | 2                                                     |
|------------------------------|-------------------------------------------------------|
| IServiceCollectionExtensions | Статический класс, расширяющий тип IServiceCollection |

Глобальный пакет классов «Configurations» содержит классы, описывающие конфигурацию. Диаграмма классов пакета представлена на рис. 46. В табл. 26 представлено описание классов.

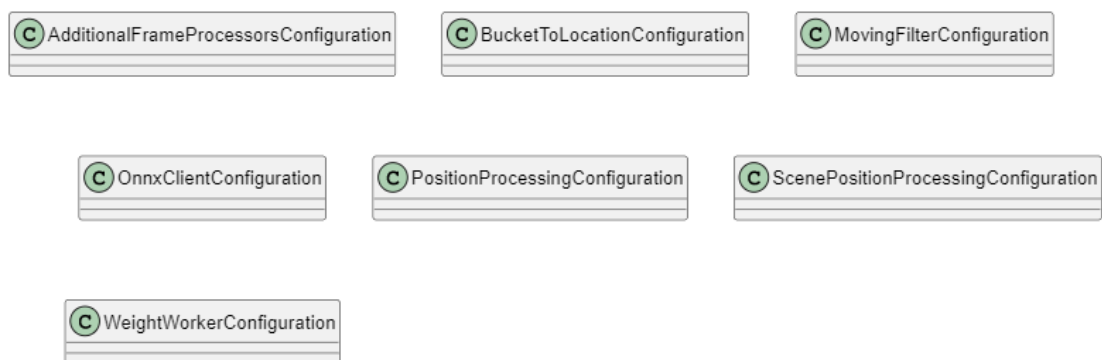


Рис. 46. Диаграмма классов пакета «Configurations»

Таблица 26

Описание классов пакета «Configurations»

| Класс                                  | Описание                                                                                                    |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 1                                      | 2                                                                                                           |
| AdditionalFrameProcessorsConfiguration | Класс, реализующий конфигурацию дополнительного обработчика кадров                                          |
| BucketToLocationConfiguration          | Класс, реализующий конфигурацию привязки местоположения сталь-ковша на крановом пути к наименованию позиции |
| MovingFilterConfiguration              | Класс, реализующий конфигурацию фильтра, определяющего наличие движения крана                               |
| OnnxClientConfiguration                | Класс, реализующий конфигурацию клиента сервиса запуска нейронных сетей                                     |

| 1                                    | 2                                                                                                                        |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| PositionProcessingConfiguration      | Класс, реализующий конфигурацию обработчика кадра камеры позиционирования, который получает позицию по маркерам          |
| ScenePositionProcessingConfiguration | Класс, реализующий конфигурацию обработчика кадра камеры позиционирования, которые получает позицию по наблюдаемой сцене |
| WeightWorkerConfiguration            | Класс, реализующий конфигурацию фонового процесса, получающего с АСУ ТП значения крановых весов                          |

Пакет классов «Crane.Contracts» содержит классы, которые описывают ответ на запрос состояния или детального состояния крана. Диаграмма классов пакета представлена на рис. 47. В табл. 27 представлено описание классов.

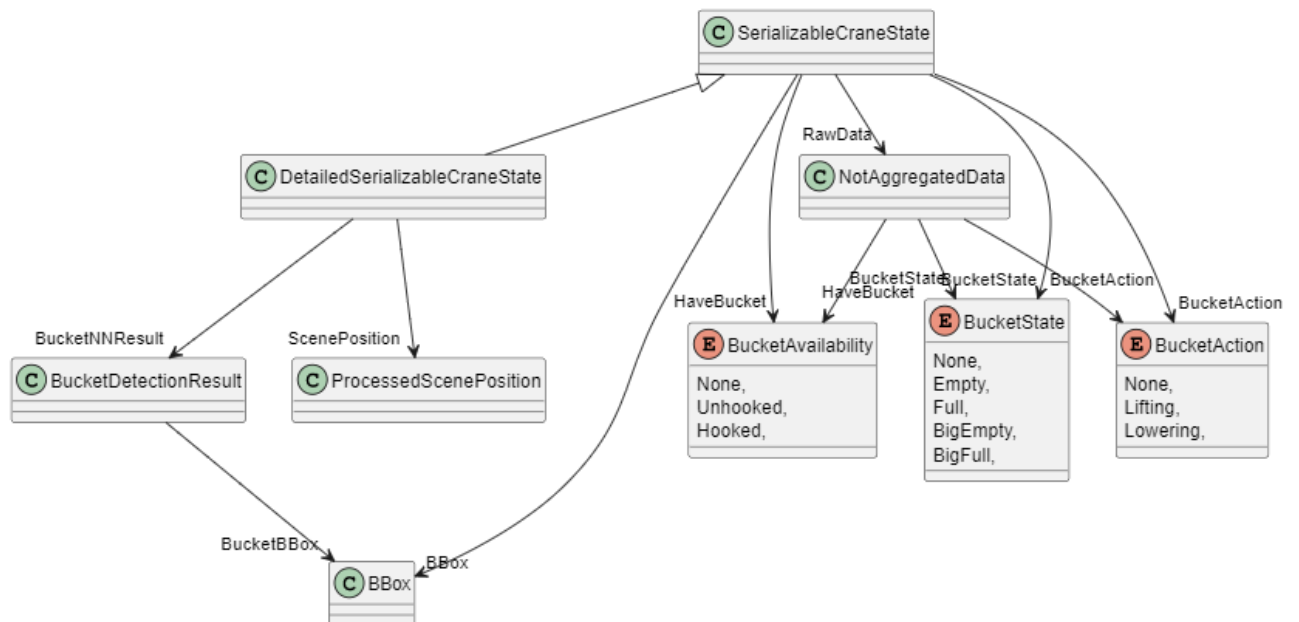


Рис. 47. Диаграмма классов пакета «Crane.Contracts»



Описание классов пакета «Crane.Contracts»

| Класс                          | Описание                                                       |
|--------------------------------|----------------------------------------------------------------|
| BBox                           | Класс, реализующий рамку детекции                              |
| BucketDetectionResult          | Класс, реализующий результат детекции сталь-ковша              |
| DetailedSerializableCraneState | Класс, реализующий детальное состояние крана                   |
| NotAggregatedData              | Класс, реализующий состояние крана до фильтра OutputDataFilter |
| SerializableCraneState         | Класс, реализующий состояние крана                             |
| BucketAction                   | Действие, производимое над ковшом                              |
| BucketAvailability             | Наличие ковша в захвате крана                                  |
| BucketState                    | Состояние сталь-ковша в захвате крана                          |

### 3.3.3 Разработка физической модели

#### 3.3.3.1 Диаграмма компонентов

Диаграммы компонентов применяют при проектировании физической структуры разрабатываемого программного обеспечения. Эти диаграммы показывают, как выглядит программное обеспечение на физическом уровне, т. е. из каких частей оно состоит и как эти части связаны между собой [46].

Диаграммы компонентов оперируют понятиями компонент и зависимость. Под компонентами при этом понимают физические заменяемые части программного обеспечения, которые соответствуют некоторому набору интерфейсов и обеспечивают их реализацию. По сути дела, это отдельные файлы различных типов.

Диаграмма компонентов системы представлена на рис. 48.

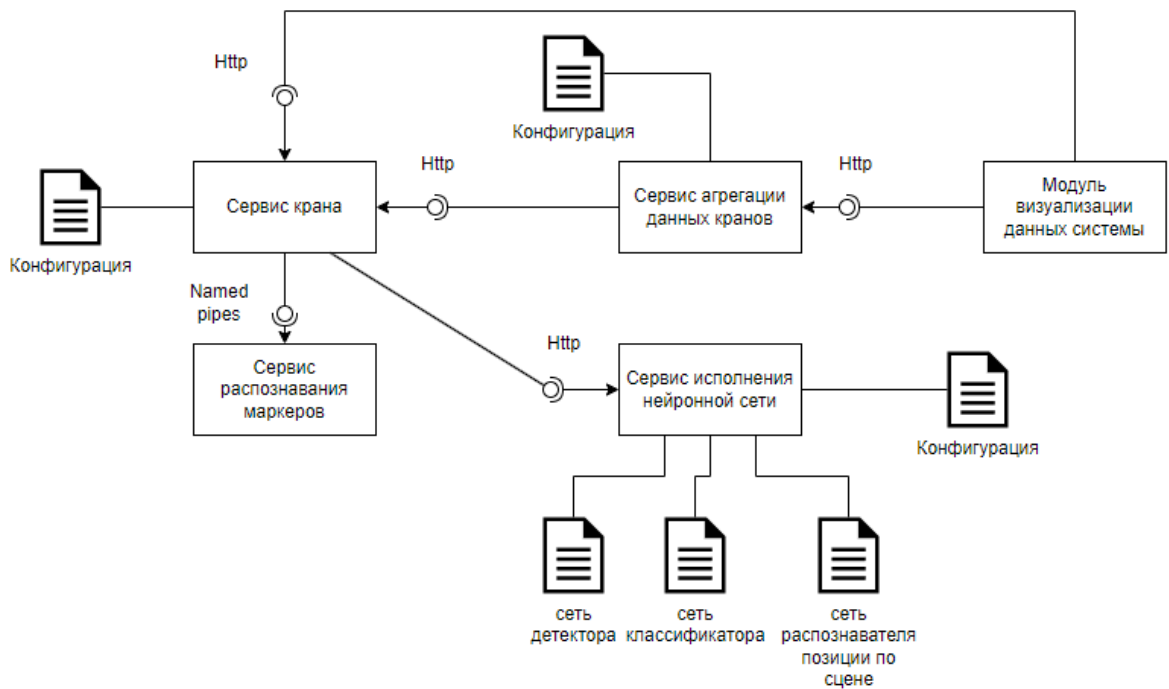


Рис. 48. Диаграмма компонентов

### 3.3.3.2 Построение диаграммы размещения

При физическом проектировании распределенных программных систем необходимо определить наиболее оптимальный вариант размещения программных компонентов на реальном оборудовании в локальной или глобальной сетях. Для этого используют специальную модель UML - диаграмму размещения [44].

Диаграмма размещения отражает физические взаимосвязи между программными и аппаратными компонентами системы. Каждой части аппаратных средств системы, например, компьютеру или датчику, на диаграмме размещения соответствует узел.

Диаграмма размещения проектируемой системы представлена на рис. 49.

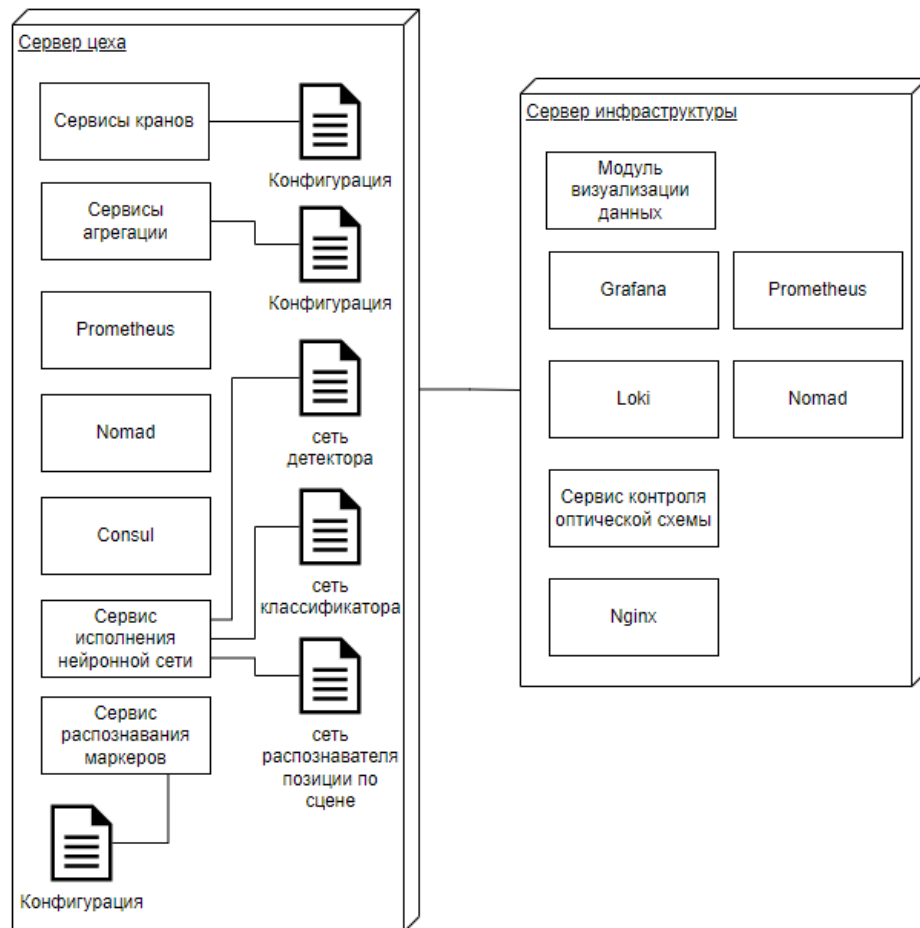


Рис. 49. Диаграмма размещения

Согласно диаграмме, имеется несколько серверов: сервер инфраструктуры и сервера, обслуживающие цеха.

На сервере инфраструктуры размещаются такие сервисы как:

- Модуль визуализации данных;
- Grafana для настройки уведомлений о пропаже сигнала какой-либо камеры;
- Prometheus. Для сбора метрик с серверов цехов;
- Loki. Для сбора журналов работы приложений;
- Nomad. Для управления и автоматического запуска сервисов. Он объединяется в кластер с другими серверами, где также запущен Nomad;
- Nginx. Для маршрутизации запросов;

- Сервис контроля оптической схемы. Он необходим для предотвращения сдвига камеры из её Web-интерфейса, а также он отслеживает сам факт сдвига камеры;

На каждом сервере цеха разворачиваются следующие сервисы:

- Сервисы кранов. Для каждого крана запускается отдельный сервис;
- Сервисы агрегации. На каждый сервер запускается по 2 сервиса. Один для детальных состояний кранов, а другой для обычных;
- Prometheus. Для сбора метрик;
- Nomad. Для оркестрации;
- Consul. Для предоставления функционала service discovery;
- Сервис исполнения нейронной сети. Этот сервис запускает нейронные сети. Согласно конфигурации, собирает из сетей пайплайн или несколько пайплайнов обработки. Выполняет распознавание по запросу;
- Сервис распознавания маркеров.

### 3.3.4 Разработка интерфейсов

Пользовательский интерфейс системы выполнен в виде отдельного модуля по причине того, что заказчику требуются только состояния мостовых кранов цеха. Таким образом, пользовательский интерфейс носит информационный характер для разработчика системы.

Система также имеет графический интерфейс набора инструментов для разработчиков API. Он позволяет ознакомиться с предоставляемым сервисом REST API. За реализацию этого интерфейса отвечает библиотека Swagger.

Интерфейс интегрированной утилиты swagger представлен на рис. П4.1.

Так как он оформлен в виде отдельного модуля, то в данной работе он рассматриваться не будет. На рис. 50 представлен пользовательский интерфейс программного обеспечения контроля перемещения объектов мостовыми кранами.

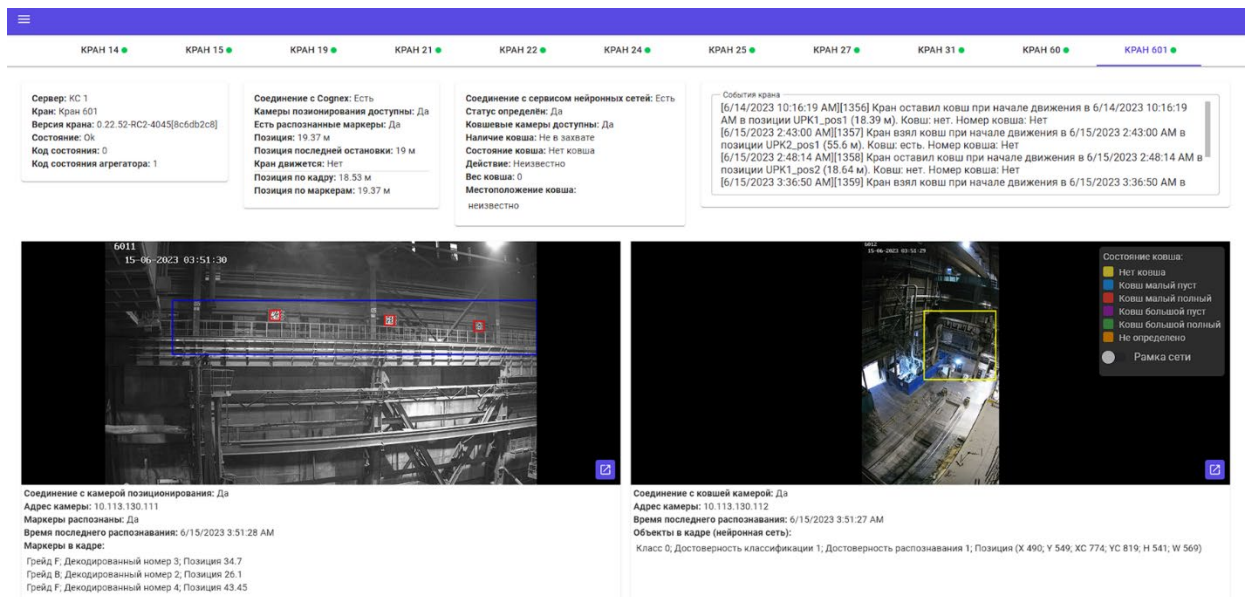


Рис. 50. Пользовательский интерфейс системы

### 3.4 Выводы по разделу 3

Были выполнены все этапы проектирования разрабатываемого программного обеспечения. Объектно-ориентированный подход был выбран для анализа и проектирования системы. В качестве средства реализации был выбран объектно-ориентированный язык программирования C# и среда разработки приложений Visual Studio.

На этапе концептуального моделирования разработаны модели «черного ящика», состава, структуры «белого ящика» системы.

На этапе проектирования информационного обеспечения был произведен выбор подхода к разработке и модели жизненного цикла программного обеспечения, а также используемых инструментов.

Были разработаны диаграммы потоков данных в системе. С использованием унифицированного языка моделирования UML разработаны логическая модель системы: диаграмма использования, контекстная диаграмма классов, диаграммы пакетов, диаграммы классов пакетов и физическая модели системы.

#### 4 Результаты экспериментальной проверки алгоритмического обеспечения системы контроля перемещения сталь-ковшей мостовыми кранами

##### 4.1 Описание эксперимента и экспериментальных данных проверки алгоритмов позиционирования, обнаружения и классификации сталь-ковша

Для проверки правильности определения состояния мостового крана, необходимо выполнить проверку правильности работы каждой из описанных ранее подсистем. За исключением подсистемы сбора данных о весе объекта в захвате. Так как её работа полностью зависит от данных, предоставляемых АСУ ТП.

Для проверки подсистемы определения местоположения по маркерам необходимо выполнить следующие действия:

- Составить перечень известных местоположений крана. Например: конвертерный путь 1 для всех кранов цеха находится на позиции 108-110 метров.
- Сформировать набор изображений с камеры позиционирования за сутки. К каждому изображению в его названии должна быть записана распознанная по маркерам позиция.
- Из сформированного набора изображений оставить только те, которые по распознанной позиции попадают в одну из позиций перечня.
- Просмотреть полученный набор и отфильтровать на кадры по принципу: запечатлённая фактическая позиция крана соответствует хотя бы одной из перечня хорошо известных позиций.
- Подсчитать процент точности позиционирования по представленной ниже формуле.

$$x = \frac{gi}{ti} * 100, \quad (5)$$

где:

$t_i$  – сколько изображений учувствовало в проверке,

gi – сколько изображений прошли проверку.

Точность позиционирования должна составлять не менее 98 %.

Для проверки работы подсистемы определения состояния захвата крана, необходимо произвести описанные ниже действия:

- Сформировать набор изображений с ковшевой камеры за сутки. К каждому изображению в его названии должен быть записан результат распознавания: наличие и состояние сталь-ковша.

- Просмотреть полученный набор кадров и отфильтровать по принципу: фактическая ситуация на изображении соответствует распознанной.

- Подсчитать процент правильных изображений по формуле 5.

Достоверность определения наличия и состояния должна быть не менее 97 %.

Для проверки правильности определения текущего действия захвата крана необходимо:

- Записать видео с ковшевой камеры за сутки.

- Собрать набор отчётов о работе системы.

- Построить график о производимом действии захватом по набору отчётов.

- Сопоставить действие на графике и отрывок видео за этот период.

- Записать количество верно определённых действий и количество действий, запечатлённых на видеоролике.

- Подсчитать процент правильных действий по формуле 5.

Достоверность определения действия захвата должна быть не менее 97 %.

Для того, чтобы проверить правильность определения наличия движения крана необходимо будет:

- Собрать набор отчётов о работе системы.

- Построить график движения крана и наличия движения по набору отчётов.

- Записать количество верно определённых состояний движения и количество перемещений крана за сутки.

- Подсчитать процент правильных определений по формуле 5.

Точность определения движения должна составлять не менее 99 %.

Для проверки подсистемы определения местоположения по наблюдаемой сцене, необходимо:

- Собрать набор отчётов о работе системы.
- Построить график разницы между движением по распознаванию сцены от движения крана по маркерам.

Разница местоположения по сцене от местоположения по маркерам не должна превышать 2 метров.

Для проверки подсистемы определения местоположения по маркерам была сформирована выборка из 10000 изображений за сутки.

Каждое изображение имеет следующий формат наименования: [дата\_и\_время][позиция\_по\_маркерам]-идентификатор.jpeg

Также были определены контрольные точки для проведения проверки. В табл. 28 представлены хорошо известные позиции.

Таблица 28

Хорошо известные позиции конвертерного цеха 1

| Ось  | Позиция, м |
|------|------------|
| КП1  | 108-110    |
| КП2  | 132-134    |
| КП3  | 156-158    |
| АЦВ  | 82-84      |
| УПК1 | 18-20      |
| УПК2 | 55-57      |

Для проверки работы подсистемы определения состояния захвата крана, за сутки была собрана выборка из 10000 изображений.



Каждое изображение имеет следующий формат наименования:  
[дата\_и\_время][наличие\_ковша][номер\_состояния]-идентификатор.jpg

Параметр «наличие\_ковша» представляет собой число, которое может принимать три значения: 0 – неизвестно, 1 – ковша нет, 2 – ковш есть.

Параметр «номер\_состояния» представляет собой число, которое может быть расшифровано следующим образом: 0 – неизвестно, 1 – ковш малый пустой, 2 – ковш малый полный, 3 – ковш большой пустой, 4 – ковш большой полный.

Для проверки правильности определения текущего действия захвата:

- было записано видео с ковшевой камеры за сутки работы системы;
- видео было описано в документе ПО «MS Excel» по принципу:  
таймкод\_начала – таймкод\_окончания – наблюдаемое\_действие;
- собран набор отчётов о работе системы за сутки.

Для проверки правильности определения наличия движения крана достаточно собранного ранее набора отчётов о работе системы за сутки.

Этого же набора достаточно для проверки подсистемы определения местоположения по наблюдаемой сцене.

## 4.2 Описание методики настройки алгоритмического и программного обеспечения системы контроля перемещения объектов мостовыми кранами

### 4.2.1 Настройка алгоритма позиционирования по маркерам

Предположим, что необходимо произвести настройку позиционирования крана, имеющего проблему с частичным перекрытием маркеров. Ввиду этого, для его позиционирования по маркерам будет применяться два видеоканала: позиционирования и слежения за сталь-ковшом.

Для настройки алгоритма позиционирования, необходимо определить местоположение маркеров на крановом пути. Места расположения маркеров на крановом пути для настраиваемого крана представлены в табл. 29.

Таблица 29

Привязка маркеров к реальным координатам

|               |       |       |      |      |      |      |      |      |       |     |
|---------------|-------|-------|------|------|------|------|------|------|-------|-----|
| Номер         | 001   | 002   | 003  | 004  | 005  | 006  | 007  | 008  | 009   | 010 |
| Позиция,<br>м | 104,3 | 99,59 | 86,9 | 74,5 | 63,7 | 51,9 | 34,2 | 27,4 | 14,47 | 4,4 |

Также известно, что для настраиваемого крана, видеокамера позиционирования имеет смещение в 3 метра.

Далее, по описанному в пункте 2.1 способу необходимо получить перечень смещений каждого маркера на изображении, которое используется для настройки. В табл. 30 представлены смещения маркеров для камеры позиционирования, а в табл. 31 для камеры слежения за сталь-ковшом.

Таблица 30

Смещения маркеров для камеры позиционирования

|                     |     |       |     |      |      |      |      |      |
|---------------------|-----|-------|-----|------|------|------|------|------|
| Столбец,<br>пиксель | 0   | 223   | 283 | 1008 | 1807 | 2430 | 2507 | 2560 |
| Смещение,<br>м      | -29 | -22,8 | -21 | -9   | 3    | 15   | 17,5 | 19   |

Таблица 31

Смещения маркеров для камеры слежения за сталь-ковшом

|                     |       |     |      |     |      |
|---------------------|-------|-----|------|-----|------|
| Столбец,<br>пиксель | 0     | 392 | 645  | 890 | 1060 |
| Смещение,<br>м      | -26,5 | -15 | -3,8 | 7,5 | 15   |

Полученные привязки маркеров к реальным позициям в цехе и смещения маркеров вносятся в конфигурационный файл. Пример конфигурации алгоритма позиционирования по маркерам представлен на рис. П5.

#### 4.2.2 Настройка алгоритма определения состояния сталь-ковша в захвате мостового крана

Определением состояния сталь-ковша в захвате мостового крана занимается ансамбль нейронных сетей. Он выполняется в отдельном сервисе. Этот сервис регистрируется при запуске сервисом Nomad в Consul с тегом «nnSharp». Таким образом, в конфигурационном файле сервиса крана для параметра «NeuroNetSdZmqTag» необходимо в качестве значения указать этот тег.

Этот же параметр используется для поиска сервиса распознавания во время определения местоположения по наблюдаемой сцене.

В настройках сервиса исполнения нейронных сетей необходимо в словаре «Networks» добавить или изменить существующий дочерний словарь. Значением элемента дочернего словаря является набор параметров для исполнения сети, а именно: путь к ONNX файлу; флаг цветности входящего на распознавание изображения; количество классов, если применяемая нейронная сеть является классификатором. Пример конфигурации сервиса исполнения нейронных сетей представлен рис. П6.

#### 4.2.3 Настройка алгоритма определения начала движения

Для настройки этого алгоритма, необходимо по предоставляемой сервисом мостового крана метрике «crane\_processing\_position» построить график движения. Он представлен на рис. 51.



Рис. 51. График движения крана

На представленном графике наблюдается колебание позиции крана в 3 метра во время остановки с 4:31 по 4:44. При проверке других периодов остановки крана, величина колебания не превышала ранее установленную. Для того, чтобы такие перепады не влияли на определение наличия движения, необходимо для алгоритма установить порог, равный 3.

Таким образом, значение параметра конфигурации «PositionChangeSensety» будет равняться найденному значению.

#### 4.2.4 Настройка алгоритма определения действия захвата

Для настройки алгоритма, необходимо определить разницу между максимальной и минимальной координатой Y рамки детекции захвата во время перерыва между подъёмами и опусканиями сталь-ковша. Проще всего это сделать по метрике «crane\_processing\_bucket\_pos\_y», которая предоставляется сервисом крана. На рис. 52 представлен график колебания высоты захвата на кадре во время перерыва между его действиями.

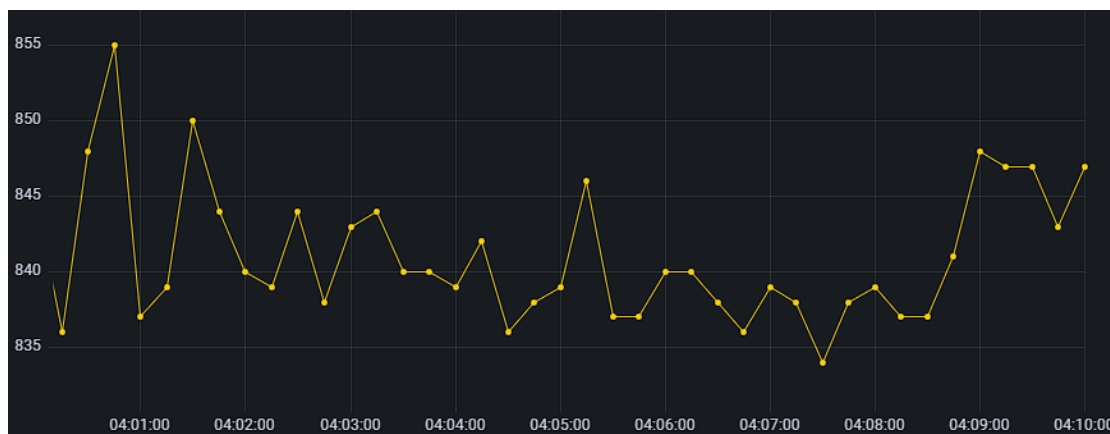


Рис. 52. График колебания высоты захвата крана на кадре

По представленному графику, максимальное значение координаты Y составляет 855 пикселей, а минимальное 834. Разница между двумя значениями составляет 21 пиксель. При проверке других периодов отсутствия действия захвата, было определено, что ни один из результатов подсчёта не превышает 21 пиксель, а это значит, что значение можно считать порогом определения наличия действия захвата. Следовательно, параметр «PositionChangeSensety» конфигурации сервиса крана будет равняться 21.

#### 4.2.5 Настройка сервиса агрегации состояний кранов цеха

Для настройки сервиса агрегации состояний кранов цеха, необходимо указать путь запроса состояния крана. Для агрегатора, предоставляющего информацию системе производственного планирования, адресом является значение параметра «TargetEndpoint» равное «/api/v1/state». Для агрегатора детальных состояний – «/api/v1/detailedCraneState».

Система производственного планирования производит опрос агрегатора не чаще одного раза в секунду. Ввиду этого, необходимо установить более короткий период запроса состояний у сервисов кранов. Это необходимо для того, чтобы система производственного планирования получала актуальную информацию. Таким образом, значение параметра «SyncTimeout» можно

установить в любое значение, меньшее чем 1000 миллисекунд. Значением было выбрано 500 миллисекунд.

Так как сервис агрегации получает IP-адрес и порт сервиса крана от сервиса обнаружения других сервисов, то необходимо указать признак, по которому будут искаться необходимые для опроса сервисы. Таким признаком в сервисе Consul выступает ключевое слово, иными словами, «тег». У сервиса крана, при его запуске через Nomad, устанавливается тег «CraneAggregation». Это значение и будет являться значением параметра «ServiceTag».

#### 4.2.6 Настройка уведомлений о проблемах с видеопотоком

Каждый видеоканал сервиса крана предоставляет метрику количества кадров в секунду (далее FPS). Для сбора метрик используется сервис Prometheus. Он может получать список сервисов, с которых необходимо запросить метрики, от сервиса Consul по указанному тегу.

Для сбора метрик с сервисов кранов, в разделе «scrape\_configs» конфигурационного файла сервиса Prometheus, описывается задача с параметром «consul\_sd\_configs», который имеет в перечислении «tags» тег «Monitoring». Этот тег присваивается приложению крана сервисом Nomad во время автоматической регистрации приложения в Consul.

Далее, в сервисе Grafana, Prometheus подключается в качестве источника данных [47]. И создаётся новое правило уведомления [48].

В качестве выражения уведомления выступает условие: средний FPS видеоканала за последние 15 минут ниже 1 кадра с секунду. Блок настройки условия представлен на рис. 53.

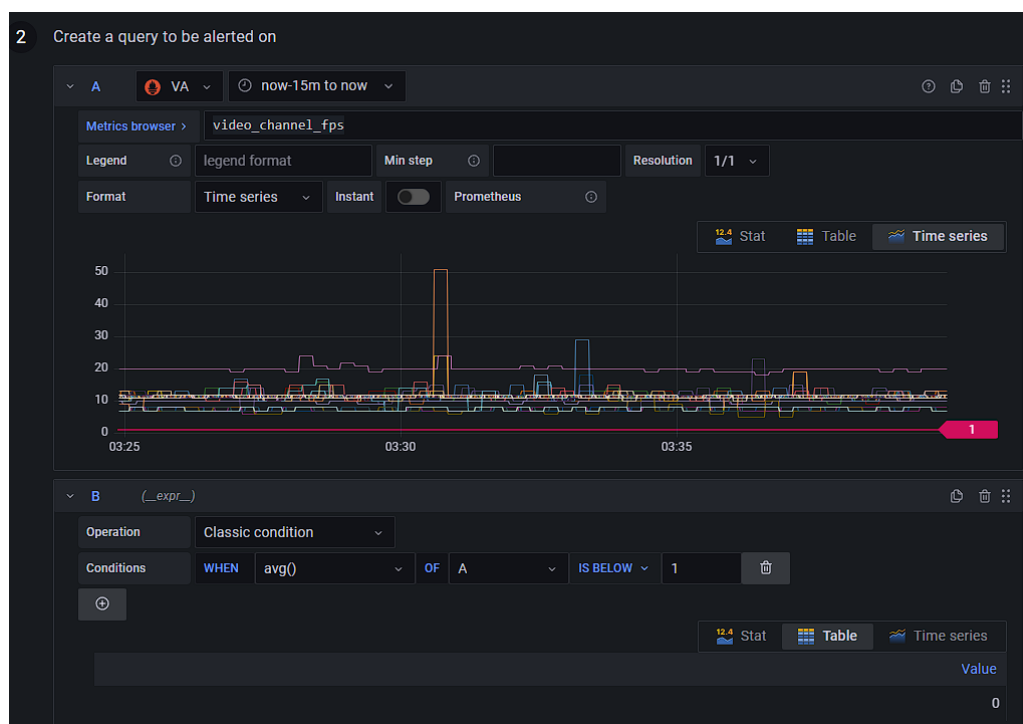


Рис. 53. Блок настройки условия уведомления

#### 4.2.7 Настройка сервиса Nginx

Для упрощения доступа модуля визуализации данных системы контроля перемещения объектов мостовыми кранами применяется прокси сервер Nginx.

Ввиду того, что сервисы кранов имеют динамический порт, для конфигурирования Nginx применяется утилита Consul template.

Эта утилита с определённой периодичностью проверяет наличие изменений в составе зарегистрированных сервисов в Consul. При появлении изменений, в шаблон конфигурации Nginx подставляются данные из Consul. Шаблон конфигурации представлен на рис. П7.

Конечная цель применения прокси сервиса – предоставление возможности запроса изображения с определенного крана по запросу «{IP-адрес сервера цеха}:80/crane{номер крана}/{дополнительные параметры запроса}» вместо «{IP-адрес крана цеха}:{порт крана цеха}/{дополнительные параметры запроса}».

#### 4.2.8 Настройка задач сервиса Nomad

Все сервисы описываемой системы запускаются с помощью оркестратора Nomad. Исключением является:

- Сервис распознавания маркеров на изображении. Этот сервис не поддерживает подобный вид запуска. В связи с этим, он регистрируется как служба windows.

- Consul.
- Prometheus.
- Grafana.

Сервисы системы поделены на рабочие нагрузки, относительно терминологии применяемого оркестратора. Каждая рабочая нагрузка состоит из групп. Каждая группа содержит перечень задач.

Группа задач позволяет единожды описать необходимые для запуска какого-либо сервиса действия, а затем масштабировать количество сервисов.

Таким образом, может быть создана группа задач необходимых для запуска сервиса крана, а затем количество запускаемых сервисов может быть установлено с помощью параметра «count» группы, содержащей эти задачи.

На рис. П8 представлен пример рабочей нагрузки оркестратора.

### 4.3 Описание результатов проверки алгоритмов системы контроля перемещения объектов мостовыми кранами

#### 4.3.1 Результаты экспериментальной проверки метода определения местоположения по маркерам

Из первоначальной выборки в 10000 было убрано 3053 изображения по причине несоответствия позиции в описании ни одной из хорошо известных позиций крана в цехе. В большинстве случаев, на этих изображениях кран находится в движении.



После просмотра получившейся выборки, было выявлено, что содержимое 48 изображений не соответствует описанию в наименовании их файла.

Дополнительно был собран набор отчётов о работе системы и построен график перемещения мостового крана. Пример графика перемещения крана в течении часа представлен на рис. 54.

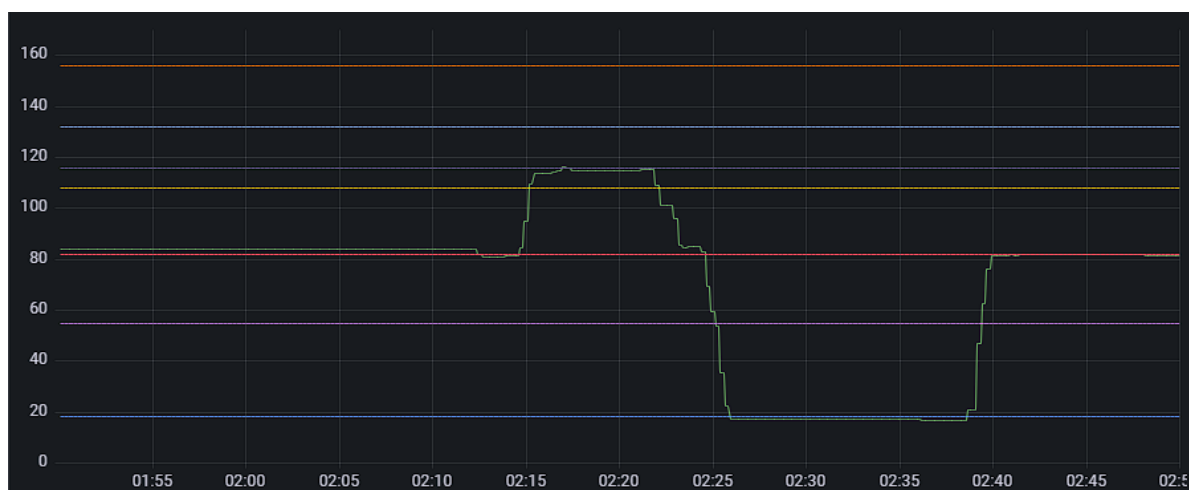


Рис. 54. Перемещение мостового крана на протяжении одного часа

Как можно наблюдать на графике, места остановки крана практически соответствуют горизонтальным линиям. Эти линии заданы на основании координат контрольных точек, используемых для тестирования.

По итогу расчётов, точность позиционирования мостового крана составила 99,3 %.

#### 4.3.2 Результаты экспериментальной проверки алгоритма определения состояния захвата крана

При просмотре суточной выборки из 10000 были отфильтрованы 182 изображения из-за несоответствия содержимого на кадре описанию файла. Таким образом, достоверность определения наличия и состояния сталь-ковша составила 98,2 %.

Также был использован набор отчётов о работе системы для построения графика смены наличия сталь-ковша в зависимости от местоположения крана. Он представлен на рис. 55.



Рис. 55. График смены наличия сталь-ковша в зависимости от местоположения крана

На представленном графике видно, в какие моменты кран ставит сталь-ковш и берёт его в захват. Жёлтая линия на графике – позиция крана. Оранжевая – наличие сталь-ковша в захвате.

#### 4.3.3 Результаты экспериментальной проверки алгоритма определения текущего действия захвата крана

Для проверки был построен график подъёма и опускания захвата, который представлен на рис. 56.

По графику была проведена сверка с составленным описанием к видеоряду.



Рис. 56. График подъёма и опускания захвата крана

В результате сверки, достоверность определения действия захвата составила 98 %.

#### 4.3.4 Результаты экспериментальной проверки алгоритма определения наличия движения крана

Был построен график передвижения крана и факта наличия движения. Пример графика за 15 минут представлен на рис. 57.



Рис. 57. График передвижения крана и факта наличия его движения

На представленном выше графике жёлтым выделяется факт наличия движения. Он соответствует увеличению и уменьшению позиции крана.

Были подсчитаны правильные определения наличия движения на графике за весь день.

Точность определения наличия движения крана составила 99,7%

#### 4.3.5 Результаты экспериментальной проверки алгоритма определения местоположения по наблюдаемой сцене

Был построен график разницы между позицией, подсчитанной по маркерам и позицией, распознанной по наблюдаемой сцене. Он представлен на рис. 58.



Рис. 58. График разницы между двумя типами определения наличия движения

Учитывая, что точность определения позиции по маркерам составляет 99,3%, а разброс разницы в пике составляет 6-8 метров, то можно утверждать, что нейронная сеть определения местоположения по наблюдаемой сцене требует дополнительного обучения и на текущий момент, может применяться только в том случае, когда нет маркеров в кадре.

#### 4.4 Результаты экспериментальной проверки модулей и классов программного обеспечения системы контроля перемещения объектов мостовыми кранами

Для проверки программной документации применялся метод ручного контроля. Как правило, этот метод используется на ранних этапах разработки, так как с его помощью можно находить от 30 до 70 % ошибок логического проектирования и кодирования.

В качестве метода ручного контроля выбран метод проверки за столом. Проверка исходного текста проводится одним человеком, который читает и проверяет текст на наличие возможных проблем по списку наиболее часто встречающихся ошибок.

В результате проверки программной документации методом ручного контроля, были исправлены найденные пунктуационные и грамматические ошибки. Проверено соответствие документации требованиям ЕСПД. Схемы и диаграммы проверены на соответствие ГОСТ.

Для тестирования основных компонентов программного продукта применялось модульное тестирование. Модульное функциональное тестирование предполагает изолированное тестирование компонентов ПО в искусственно созданной среде.

Результаты тестирования представлены в табл. 32.

В результате проверки основных классов программного продукта ошибок не было найдено. Таким образом, можно сделать вывод, что программный продукт способен исполнять основные функции.

## Результаты тестирования

| Дата       | Тестируемый модуль или подпрограмма | Кто проводил тестирование | Описание теста                                                                                                                                                                                         | Результат |
|------------|-------------------------------------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 1          | 2                                   | 3                         | 4                                                                                                                                                                                                      | 5         |
| 02.05.2023 | PositionFrameProcessor.cs           | Разработчик               | Подан кадр с камеры позиционирования. Должны быть найдены маркеры, рассчитана позиция                                                                                                                  | Успех     |
| 02.05.2023 | BucketFrameProcessor.cs             | Разработчик               | Подан кадр с ковшевой камеры. Должен быть найден ковш, определено его состояние                                                                                                                        | Успех     |
| 02.05.2023 | MergePipesContextsFilter.cs         | Разработчик               | На вход периодически подаются контексты IBucketStateContext, IScenePositionContext, IWeightContext. На выходе должен быть контекст ICraneStateContext, содержащий данные из ранее поданных контекстов. | Успех     |

Продолжение табл. 32

| 1          | 2                   | 3           | 4                                                                                                                                                                                                                                                            | 5     |
|------------|---------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 02.05.2023 | OutputDataFilter.cs | Разработчик | На вход подаётся контекст ICraneStateContext. Результатом работы должен быть дополненный контекст ICraneStateContext информацией о: действии захвата, наличии движения, наименовании позиции сталь-ковша. Должен быть выбран правильный тип позиционирования | Успех |
| 02.05.2023 | MovingFilter.cs     | Разработчик | На вход подаётся несколько контекстов ICraneStateContext. Эти контексты содержат меняющуюся позицию. Результатом должен быть признак наличия движения крана                                                                                                  | Успех |

Продолжение табл. 32

| 1          | 2                        | 3           | 4                                                                                                                                                                                            | 5     |
|------------|--------------------------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 02.05.2023 | BucketLocationMapper.cs  | Разработчик | На вход подаётся позиция конкретного агрегата и описание положения сталь-ковша на кадре (кран ставит сталь-ковш на агрегат). Результатом работы должно быть наименование позиции сталь-ковша | Успех |
| 02.05.2023 | FrameMarkersProcessor.cs | Разработчик | На вход подаётся перечень маркеров. Результатом работы должна быть позиция                                                                                                                   | Успех |
| 02.05.2023 | BucketStateProcessor.cs  | Разработчик | На вход подаётся ответ сервиса запуска НС. Результатом должно быть состояние сталь-ковша                                                                                                     | Успех |

Также было проведено интеграционное тестирование между сервисом мостового крана, сервисом агрегации состояний мостовых кранов, и модулем визуализации данных системы.

В результате тестирования, было определено, что состояние мостового запрашивается сервисом агрегации и сохраняется в память этого сервиса. Сохранённое состояние запрашивается модулем визуализации данных и отображается в пользовательском интерфейсе.



До появления первого состояния у сервиса крана, сервис агрегации отдаёт ответ с числовым кодом, обозначающим, что ещё не было получено ни одного состояния. Эта ситуация корректно обрабатывается модулем визуализации.

#### 4.5 Перспективы применения предложенных технических решений

Предложенные решения, позволяют в реальном времени определять состояние мостового крана.

Получаемые состояния кранов цеха могут использоваться системами верхнего уровня как дополнительный источник информации при планировании производства. Что, в свою очередь повысит оперативность планирования и позволит пересчитывать производственный план при выходе реальной ситуации за рамки первоначального.

Это позволит экономить на операции дополнительного разогрева стали в сталь-ковше.

В данной работе, контроль перемещения объектов был описан на примере сталь-ковше в металлургическом производстве. При дополнительной доработке системы, объектами контроля перемещения могут выступать не только сталеплавильные ковши, но и иные объекты. Например: лоток с металлоломом.

Также, появляется возможность фиксировать действия машиниста крана, и возможность просмотра с удаленного рабочего места текущих кадров с камер крана. При необходимости, в кабине машиниста мостового крана можно установить монитор, на который выводится изображение с камеры слежения за ковшом для минимизации слепых зон.

Появляется возможность интеграции с системами промышленной безопасности. Например: при наличии сталь-ковша в захвате крана и нахождении сотрудника металлургической компании в определённой зоне, которая в данный момент может считаться опасной, выдавать машинисту мостового крана сигнал тревоги.

#### 4.6 Выводы по разделу 4

В рамках данного раздела опытным путём было проведено тестирование позиционирования мостового крана в условиях реального производства. Точность позиционирования составила 99 %.

Также была проверена и достоверность определения состояния стальной ковша в захвате мостового крана. Она составила 98 %.

Были рассмотрены перспективы применения системы контроля перемещения объектов мостовыми кранами.

## Заключение

Результатом выпускной квалификационной работы является разработанное программное обеспечение контроля перемещения объектов мостовыми кранами.

В результате выполнения работы получены следующие результаты:

- произведенный анализ имеющихся решений выявил необходимость разработки программного обеспечения;
- дано описание метода позиционирования, отличающегося использованием маркеров на нескольких кадрах, что позволяет выполнить локализацию мостового крана в условиях перекрытия одной или нескольких меток позиционирования;
- дано описание алгоритма локализации объектов на крановом пути, отличающегося возможностью определения текущего состояния сталь-ковша в захвате мостового крана;
- в результате экспериментальной проверки установлено, что детекция и классификация сталь-ковша на видеокадре имеют точность более 98%;
- в результате экспериментальной проверки установлено, что определение местоположения крана по маркерам на кадре имеет точность более 99 %.

Разработанное программное обеспечение готово к внедрению на металлургическом производстве и дальнейшему тиражированию компанией ООО "Малленом Системс", о чём свидетельствует акт внедрения, представленный на рис. П9.1.

За счет оптимального планирования движения кранов и стелевозов в конвертерном цехе НЛМК г. Липецк на первых стадиях проекта «Оптимизации, диспетчеризации и логистики в конвертерном производстве», частью которого является представленное решение, был показан подтвержденный экономический эффект 75 млн. рублей в год [49].

Кроме того, в результате выполнения диссертационной работы были полученные следующие профессиональные компетенции:

- способность осуществлять критический анализ проблемных ситуаций на основе системного подхода, вырабатывать стратегию действий;
- способность управлять проектом на всех этапах его жизненного цикла;
- способность организовывать и руководить работой команды, вырабатывая командную стратегию для достижения поставленной цели;
- способность применять современные коммуникативные технологии, в том числе на иностранном(ых) языке(ах), для академического и профессионального взаимодействия;
- способность анализировать и учитывать разнообразие культур в процессе межкультурного взаимодействия;
- способность определять и реализовывать приоритеты собственной деятельности и способы ее совершенствования на основе самооценки;
- способность самостоятельно приобретать, развивать и применять математические, естественнонаучные, социально экономические и профессиональные знания для решения нестандартных задач, в том числе, в новой или незнакомой среде и в междисциплинарном контексте;
- способность разрабатывать оригинальные алгоритмы и программные средства, в том числе с использованием современных интеллектуальных технологий, для решения профессиональных задач;
- способность анализировать профессиональную информацию выделять в ней главное, структурировать, оформлять и представлять в виде аналитических обзоров с обоснованными выводами и рекомендациями;
- способность применять на практике новые научные принципы и методы исследований;
- способность разрабатывать и модернизировать программное и аппаратное обеспечение информационных и автоматизированных систем;

- способность самостоятельно приобретать с помощью информационных технологий и использовать в практической деятельности новые знания и умения, в том числе в новых областях знаний, непосредственно не связанных со сферой деятельности;
- способность применять при решении профессиональных задач методы и средства получения, хранения, переработки и трансляции информации посредством современных компьютерных технологий, в том числе, в глобальных компьютерных сетях;
- способность осуществлять эффективное управление разработкой программных средств и проектов;
- управлять программно-техническими, технологическими и человеческими ресурсами.

## Список литературы

1. Трофименко, Я. М. Модель идентификации стальной в сталеплавильном производстве / Я. М. Трофименко, Е. В. Ершов // Вестник Череповецкого государственного университета. – 2021. – № 1(100). – С. 68-77. – DOI 10.23859/1994-0637-2021-1-100-5. – EDN DHYBUS
2. Шабалина Наталья Анатольевна Электромагнитные помехи в промышленности: принципы возникновения и влияние на электронное оборудование // Инновации в науке. 2012. №13-1
3. Google patents. Способ определения координат мостового крана - [Электронный ресурс] – Режим доступа: <https://patents.google.com/patent/RU2767796C1/ru> (Дата обращения: 14.02.2022)
4. Сенсор 365. Принцип действия магнитострикционных датчиков линейных перемещений Micropulse и Temposonics - [Электронный ресурс] – Режим доступа: <https://sensor365.ru/stati/princip-dejstviya-magnitostriktionnyh-datchikov-linejnyh-peremewenij/> (Дата обращения: 14.02.2022)
5. Google patents. Control system and method for an overhead bridge crane. - [Электронный ресурс] – Режим доступа: <https://patents.google.com/patent/US7190146> (Дата обращения: 14.02.2022)
6. Датчики для автоматизации, комплексное оснащение. Система позиционирования WCS. - [Электронный ресурс] – Режим доступа: <https://compeq-pf.ru/promyshlennye-datchiki/sistema-kodirovaniya-polozheniya-WCS> (Дата обращения: 14.02.2022)
7. Ростехнадзор. Постановка на учёт подъёмных сооружений - [Электронный ресурс] – Режим доступа: [http://svost.gosnadzor.ru/activity/control/gos\\_kontrol\\_i\\_nadzor\\_v\\_sfere\\_prombezo\\_pasnosti/postanovka-na-uchyet-podyemnykh-sooruzheniy-.php](http://svost.gosnadzor.ru/activity/control/gos_kontrol_i_nadzor_v_sfere_prombezo_pasnosti/postanovka-na-uchyet-podyemnykh-sooruzheniy-.php) (Дата обращения: 16.02.2022)

8. Ростехнадзор. Приказ № 494 от 25.11.2016. «Об утверждении административного регламента по предоставлению федеральной службой по экологическому, технологическому и атомному надзору государственной услуги по регистрации опасных производственных объектов в государственном реестре опасных производственных объектов»

9. Symeo. LRP 2D - [Электронный ресурс] – Режим доступа: <https://www.symeo.com/en/products/position-detection/lpr-2d/index.html> (Дата обращения: 10.03.2022)

10. Склад & Техника. Умный склад на основе MAGNETTRACK - [Электронный ресурс] – Режим доступа: <https://sitmag.ru/article/26073-umniy-sklad-na-osnove-magnettrack> (Дата обращения: 10.03.2022)

11. Авторское свидетельство № 838391 А1 СССР, МПК G01G 19/14. Крановые весы: № 2821083: заявл. 28.09.1979: опубл. 15.06.1981 / А. С. Кукуй, Б. В. Верницкий; заявитель Запорожский филиал всесоюзного научно-исследовательского и конструкторского института "Цветметавтоматика"

12. Варим суп из стали: оптимизация логистики ковшей и как устроен цех КЦ № 2 - [Электронный ресурс] – Режим доступа: <https://habr.com/ru/companies/nlmk/articles/580270/> (Дата обращения: 16.03.2022)

13. Prometheus. Alerting overview - [Электронный ресурс] – Режим доступа: <https://prometheus.io/docs/alerting/latest/overview/> (Дата обращения: 21.03.2022)

14. MERTECH. Всё о сканерах штрих кода. Datamatrix - [Электронный ресурс] – Режим доступа: <https://mertech.ru/blog/vse-o-skanerah-shtrih-koda/datamatrix-kod-markirovki> (Дата обращения: 21.03.2022)

15. Патент № 2770940 С1 Российская Федерация, МПК В66С 13/18. Система и способ автоматического определения положения мостового крана вдоль пути движения: № 2021138785: заявл. 24.12.2021: опубл. 25.04.2022 / Е. Н. Веснин, Ю. Н. Бахвалов; заявитель общество с ограниченной ответственностью "Малленом Системс"

16. Zhi-Hua Zhou. Ensemble Learning – [Электронный ресурс] – Режим доступа: <https://cs.nju.edu.cn/zhoush/zhoush.files/publication/springerEBR09.pdf> (Дата обращения: 24.06.2022)
17. Nick Ferrando. FFMPEG From Zero to Hero – [Электронный ресурс] – Режим доступа: [https://ffmpegfromzerotohero.com/free-sample/FFMPEG-From\\_Zero\\_to\\_Hero-Free\\_Sample.pdf](https://ffmpegfromzerotohero.com/free-sample/FFMPEG-From_Zero_to_Hero-Free_Sample.pdf) (Дата обращения: 24.06.2022)
18. CVAT. Open Data Annotation Platform Hero – [Электронный ресурс] – Режим доступа: <https://www.cvat.ai/> (Дата обращения: 14.09.2022)
19. Arxiv. YOLOv4: Optimal Speed and Accuracy of Object Detection - [Электронный ресурс] – Режим доступа: <https://arxiv.org/abs/2004.10934> (Дата обращения: 14.09.2022)
20. ONNX. Open Neural Network Exchange – [Электронный ресурс] – Режим доступа: <https://onnx.ai/> (Дата обращения: 05.10.2022)
21. Bradski G., Kaehler A. OpenCV // Dr. Dobb's journal of software tools. – 2000. – С. 122-125
22. Github user content. yolov4.cfg – [Электронный ресурс] – Режим доступа: <https://raw.githubusercontent.com/AlexeyAB/darknet/master/cfg/yolov4.cfg> (Дата обращения: 11.10.2022)
23. Github. network.c – [Электронный ресурс] – Режим доступа: <https://github.com/AlexeyAB/darknet/blob/103d301ccbc19e47e002005bdfdbaf07a92cd880/src/network.c#L1051> (Дата обращения: 11.10.2022)
24. Arxiv. MobileNetV2: Inverted Residuals and Linear Bottlenecks - [Электронный ресурс] – Режим доступа: <https://arxiv.org/abs/1801.04381> (Дата обращения: 17.10.2022)
25. Huggingface. EfficientNet (b1 model) - [Электронный ресурс] – Режим доступа: <https://huggingface.co/google/efficientnet-b1> (Дата обращения: 17.10.2022)



26. Алексеев Г.В. Основы системного анализа в пищевой промышленности: монография / Алексеев Г.В., Холявин И.И. — Саратов: Вузовское образование, 2017. — 112 с
27. Цветков В.Я. Модели в информационном поле. Palmarium Academic Publishing, 2018. 129 с.
28. Киселев Д.Ю., Киселев Ю.В., Макарьев В.Д. Структурный анализ потоков данных (Data Flow Diagrams – DFD). Метод. указания - Самара: СГАУ, 2014. – 12 с.
29. 610.12-1990 - IEEE Standard Glossary of Software Engineering Terminology
30. Agile – [Электронный ресурс] – Режим доступа: <https://www.atlassian.com/ru/agile> (Дата обращения: 05.12.2022)
31. Вендров А.М. CASE-технологии. Современные методы и средства проектирования информационных систем. – Москва: Финансы и статистика, 1998. – 176 с
32. Иванова Г.С. Технология программирования: Учебник для вузов. - М.: Изд-во МГТУ им. Н.Э. Баумана, 2002. -320 с.: ил. (Сер. Информатика в техническом университете.)
33. Лешек А. Мацяшек. Анализ и проектирование информационных систем с помощью UML 2.0. - Вильямс, 2008. – 816 с
34. Назаров С.В. Архитектура и проектирование программных систем: Монография. / С.В. Назаров. - М.: ИНФРА-М, 2013. - 351 с.
35. Microsoft. Размещение и развертывание ASP.NET Core Blazor WebAssembly – [Электронный ресурс] – Режим доступа: <https://learn.microsoft.com/ru-ru/aspnet/core/blazor/host-and-deploy/webassembly> (Дата обращения: 24.12.2022)
36. Гниденко, И. Г. Технология разработки программного обеспечения: учебное пособие для среднего профессионального образования / И. Г. Гниденко, Ф. Ф. Павлов, Д. Ю. Федоров. — Москва: Издательство Юрайт, 2022. — 235 с.

37. Nomad – [Электронный ресурс] – Режим доступа: <https://www.nomadproject.io/> (Дата обращения: 20.02.2023)
38. Consul – [Электронный ресурс] – Режим доступа: <https://www.consul.io/> (Дата обращения: 20.02.2023)
39. Prometheus – [Электронный ресурс] – Режим доступа: <https://prometheus.io/> (Дата обращения: 20.02.2023)
40. Grafana – [Электронный ресурс] – Режим доступа: <https://grafana.com/> (Дата обращения: 20.02.2023)
41. Loki OSS – [Электронный ресурс] – Режим доступа: <https://grafana.com/oss/loki/> (Дата обращения: 20.02.2023)
42. Администрирование сервера NGINX. – М.: ДМК Пресс, 2018. – 288 с.: ил
43. Леоненков А.В. «Самоучитель UML – 2-е изд.» – Санкт-Петербург.: БХВ-Петербург, 2013. - 432 с.
44. Леоненков А.В. «Самоучитель UML – 2-е изд.» – Санкт-Петербург.: БХВ-Петербург, 2013. - 432 с.
45. Буч, Якобсон, Рамбо, Видимость – М.: ДМК Пресс, 2006. - с. 123
46. S. Rielau. DB2 9.7: Run Oracle applications on DB2 9.7 for Linux, Unix, and Windows (англ.). Developer Works. IBM (Дата обращения: 20.04.2023)
47. Grafana. Get started Grafana Prometheus – [Электронный ресурс] – Режим доступа: <https://grafana.com/docs/grafana/latest/getting-started/get-started-grafana-prometheus/> (Дата обращения: 20.04.2023)
48. Grafana. Alerting – [Электронный ресурс] – Режим доступа: <https://grafana.com/docs/grafana/latest/alerting/> (Дата обращения: 20.04.2023)
49. Уникальный проект на НЛМК. ДАТА-ЦЕНТР Автоматика – [Электронный ресурс] – Режим доступа: <https://www.dc.ru/news/2021/02/12/77/> (Дата обращения: 25.05.2023)

## **Устав проекта**

### **1. Общие положения**

Данный документ является соглашением между основными участниками проекта, определяющими цели и содержание проекта, обязательства его участников. Оно включает основные этапы выполнения работ по его реализации.

Соблюдение положений устава является обязательным для всех участников проекта.

Данный документ может меняться в ходе реализации проекта. Внесение изменений и дополнений в устав проекта инициализируется руководителем проекта и утверждается заказчиком.

Настоящий устав действителен, начиная с момента подписания договора на проектные работы и до окончания сроков реализации проекта (до подписания акта приема-передачи всех работ по проекту)

### **2. Требования, удовлетворяющие потребности, пожелания и ожидания заказчика, спонсора и других участников проекта**

Программное обеспечение системы контроля перемещения объектов мостовыми кранами должно выполнять следующие задачи:

#### **1. Задача интеграционная**

Проектируемое программное обеспечение должно иметь возможность экспортировать по запросу внешней системы состояния мостовых кранов. При этом, состоянием считается его текущее местоположение, а также состояние ковшевого захвата.

## **2. Задача повышения отказоустойчивости**

Программное обеспечение должно диагностировать отказ оборудования, отказ программных компонентов системы и оповещать об этом. По возможности производить восстановление работоспособности программных компонентов.

## **3. Задача накопления исторических данных**

Для выполнения анализа правильности позиционирования мостового крана, необходимо накапливать данные о позиционировании. Также необходимо накапливать данные о детектированном состоянии захвата крана. Для диагностирования состояния оборудования необходимо, помимо прочего, накапливать информацию о его работоспособности.

## **3. Производственная необходимость**

Перед компанией ООО "Малленом Системс" поставлена задача: разработать систему видеоаналитики сталь-ковшей (далее ковш) для проекта "Цифровой сервис ГЕФЕСТ: внутрицеховая логистика и диспетчеризация для сталеплавильного производства". Программное обеспечение этого проекта производит построение расписания работы оборудования в рамках всего предприятия. Полученные таким образом частные расписания производственных подразделений являются взаимосвязанными с точки зрения изделия и его операций.

Видеоаналитика является неотъемлемой частью такого проекта. Так как получение среза текущего состояния оборудования традиционными методами зачастую является невозможным, а решение задачи оптимального планирования без фактических данных требует построение алгоритмов и математических моделей высокой сложности или вовсе является невыполнимой.

Система видеоаналитики подразделяется на два независимых программных продукта:

- программное обеспечение системы контроля перемещения объектов мостовыми кранами;
- программное обеспечение системы контроля перемещения объектов стелевозами.

Такое разделение связано с тем, что ковши перемещаются по цеху с помощью стелевозов и мостовых кранов. Стелевозы и мостовые краны перемещаются перпендикулярно друг другу. Это делает невозможным определение позиции и состояния ковшей в цехе только с кранов и наоборот.

Обе части видеоаналитики должны определять наличие ковша, но есть и принципиальные различия. Так, наземные камеры могут видеть номер ковша на одной из его сторон, а камеры на кране нет. Но при этом, только камеры крана способны определить наполненность ковша.

#### **4. Цель проекта**

Повышение оперативности контроля за перемещением объектов мостовыми кранами.

#### **5. Критерии оценки успешной реализации проекта**

- правильное определение позиции мостового крана на крановом пути цеха;
- правильное определение наличия ковша;
- правильное определение наполненности ковша;
- правильное определение факта подъема/опускания ковша.

#### **6. Расписание контрольных событий**

- 1 этап: Анализ предметной области, постановка задачи    ноябрь - декабрь 2021  
(разработка требований)
- 2 этап: Выбор и описание подхода и инструментальных    январь - февраль 2022  
средств разработки программного обеспечения
- 3 этап: Выбор и описание математического аппарата,    февраль - март 2022

используемого в проекте

4 этап: Проектирование программного обеспечения: апрель - июнь 2022  
разработка логической и физической модели

5 этап: Разработка программного обеспечения июль - декабрь 2022

6 этап: Экспериментальная проверка программного января - март 2023  
обеспечения, оформление результатов проекта

## 7. Информация об участниках проекта

### Заказчик проекта:

- Организация: ООО «ДАТА-ЦЕНТР Автоматика»
- Полномочия:
- формирование общих требований к результатам проекта;
  - контроль хода выполнения работ по проекту;
  - приемка результатов проекта.
- Ответственность:
- предоставление доступа к серверам размещения системы;
  - уведомление о возникающих инцидентах во время работы системы в составе программного продукта «ГЕФЕСТ».

### Исполнитель проекта:

- Организация: ООО "Малленом Системс"
- ФИО, должность: Веснин Евгений Николаевич, технический директор
- Контактная информация: e-mail: vesnin@mallenom.ru
- Полномочия:
- планирование, организация и контроль выполнения работ по достижению целей

проекта с требуемыми затратами,  
качеством и в заданный срок;

- управление командой разработки;
- координация взаимодействия команды и заказчика;
- управление ресурсами на разных этапах проекта;
- управление рисками проекта;
- разработка отчётных документов по проекту.

- Ответственность:
- исполнение этапов проекта в установленные сроки;
  - соответствие разработанного программного продукта сформированным требованиям

### **Исполнитель работ по проекту:**

Организация: ООО "Малленом Системс"

ФИО, должность: Адамов Даниил Сергеевич, студент группы  
направления подготовки 09.04.04 Программная  
инженерия, инженер - программист I  
категории

Контактная информация: e-mail: danith@mallenom.ru

- Полномочия:
- выполнение работ согласно этапам проекта;
  - взаимодействие с заказчиком при необходимости.

- Ответственность:
- выполнение работ согласно этапам проекта в установленные сроки;

- разрабатываемый программный продукт должен соответствовать сформированным требованиям

## **8. Функциональные организации и их участие**

Малленом Системс – ведущая российская компания в области разработки и внедрения систем видеоаналитики и промышленного контроля на основе технологий машинного зрения и искусственного интеллекта.

Компания «Малленом Системс» создана в 2011 г. на базе коллектива ученых, инженеров–программистов и ИТ-менеджеров. Сфера деятельности: искусственный интеллект, системы машинного (технического) зрения, компьютерного моделирования и машинного обучения.

Эта компания имеет большой опыт успешной реализации наукоемких ИТ-проектов в сфере транспорта, машиностроения, нефтегазовой, металлургической, пищевой, фармацевтической, алмазодобывающей, атомной и других отраслях промышленности.

Её продукция представлена в большинстве регионов РФ, странах СНГ и ЕС. Уникальный опыт внедрений и ноу-хау компании обеспечивают быструю разработку новых высокотехнологичных продуктов и решений.

Помимо выше сказанного, в 2019 году Малленом Системс вошла в национальный рейтинг российских быстрорастущих технологических компаний «ТехУспех», разработанный РВК. А в 2020-2021 году Малленом Системс реализует грант Фонда содействия инновациям на разработку технологии конвейерного производства систем машинного зрения для решения задач промышленного контроля на рынке промышленного интернета вещей.

Компанией «Малленом Системс» оказывается широкий спектр услуг в сфере идентификации наземных транспортных средств и контроля их передвижения:



- разработка систем идентификации и контроля передвижения автотранспорта и ж/д транспорта;
- интеграция систем распознавания регистрационных номеров автотранспорта и ж/д транспорта с комплексными системами контроля и управления доступом (СКУД), интегрированными системами безопасности (ИСБ), интеграция с автомобильными и железнодорожными весами и с другими автоматизированными системами с функцией контроля и учета грузоперевозок;
- адаптация программного обеспечения идентификации и контроля передвижения автотранспорта и ж/д транспорта под требования заказчика с учетом условий эксплуатации;
- визуальный контроль продукции на производстве;
- прослеживание продукции в алкогольной, фармацевтической и других отраслях промышленности;
- транспортное моделирование и управление дорожным движением;
- поставка и внедрение продукции Cognex (умные камеры, оптические датчики, сканеры кодов и маркировок) в РФ и СНГ;
- пуско-наладочные работы и консультационные услуги при внедрении продукции;
- обучение специалистов компаний-партнеров;
- обучение эксплуатирующего и обслуживающего персонала.

Заказчиком видеоаналитики, а в частности и программного обеспечения контроля перемещения объектов мостовыми кранами является ООО «ДАТА-ЦЕНТР Автоматика».

Эта компания является ведущим системным интегратором в области автоматизации технологических процессов, управления производством, предприятием, отраслью.

Она предлагает и создаёт интегрированные отраслевые решения, объединяющие процессы производства управления, не только для отдельных

предприятий, но и целых отраслей промышленности на базе высокоинтеллектуальных систем автоматизации и виртуального моделирования.

Основа бизнеса компании "ДАТА-ЦЕНТР Автоматика" - интеграция технологий, которая позволяет разрабатывать и внедрять решения по автоматизации, как для отдельного технологического процесса предприятия, так и для целой отрасли, включая все уровни управления.

Компания "ДАТА-ЦЕНТР Автоматика" имеет следующие направления деятельности:

- контроль материальных потоков;
- аналитика больших данных и методов видеоанализа;
- промышленная аналитика;
- планирование и сопровождение ж/д перевозок;
- автоматизированные системы управления технологическими процессами;
- автоматизированные системы визуального контроля;
- виртуальное моделирование.

## **9. Допущения относительно организации и окружения, а также внешние допущения**

1. работником заключается трудовой договор с организацией, выполняющей заказ;
2. согласно трудовому договору и ТК РФ работнику устанавливается пятидневная рабочая неделя;
3. согласно ТК РФ работнику предоставляется ежегодный оплачиваемый отпуск продолжительностью 28 календарных дней;
4. продолжительность рабочего дня работника, согласно ТК РФ, устанавливается равной 8 часов;
5. с согласия работодателя, по письменному заявлению, работнику может быть предоставлен отпуск без сохранения заработной платы;
6. компетенции команды разработки должно быть достаточно для реализации проекта

## **10. Ограничения относительно организации и окружения, а также внешние ограничения**

1. Сообщать руководителю проекта об имеющихся недостатках в работе и принимаемых мерах по их устранению.
2. Соблюдать правила противопожарной безопасности.
3. соблюдать правила охраны труда;
4. соблюдать технику безопасности;
5. соблюдать правила санитарной безопасности;
6. соблюдать продиктованные трудовым договором обязанности;
7. соблюдать правила внутреннего трудового распорядка;
8. соблюдать сроки контрольных событий.

## **11. Бюджет проекта**

Программное обеспечение системы контроля перемещения объектов мостовыми кранами является самостоятельным программным продуктом, несмотря на наличие второго компонента в видеоаналитике.

Программное обеспечение системы контроля перемещения объектов мостовыми кранами может быть адаптировано под требования различных предприятий. Например: разгрузка подвижного состава мостовым краном.

Для подобной задачи неизменным остаётся определение местоположения мостового крана и перевозимого им груза. Единственным изменением в программном обеспечении будет компонент, который определяет наличие груза и классифицирует его на основании полученного видеоизображения.

Исходя из того, что программный продукт является тиражируемым, закупка необходимых технических средств для обеспечения работы программного продукта может быть выполнена заказчиком. При этом, программный продукт обязательно должен иметь перечень рекомендуемого для использования оборудования.

Таким образом, для расчётов бюджета проекта закупка технических средств не будет учитываться.

Компанией ООО «Малленом Системс» предоставляется рабочий компьютер. Стоимость компьютера составляет 67 684 руб.

### **Трудозатраты на разработку и отладку программы**

Расчет экономической эффективности и срока окупаемости проектируемой программы для ЭВМ начинается с расчета трудовых затрат.

Затраты на оплату труда при разработке программного продукта вычисляются по формуле:

$$З_{тр} = (З_{общ} + Отч) * Т_{Н},$$

где  $З_{общ}$  – общая зарплата работника за час;

Отч – отчисления с зарплаты, %;

$Т_{Н}$  – время написания программы.

Заработная плата программиста за час определяется по следующей формуле:

$$З_{пр} = \frac{Ст_{пр}}{\Phi_{вм}},$$

где  $Ст_{пр}$  – ставка программиста;

$\Phi_{вм}$  – фонд рабочего времени в месяц, ч.

Заработная плата дополнительная определяется по следующей формуле:

$$З_{доп} = \frac{З_{пр} * Н_{доп}}{100},$$

где  $Z_{\text{пр}}$  – заработная плата программиста;

$N_{\text{доп}}$  – норма отчислений на дополнительную зарплату (10%).

Зарплата общая вычисляется по следующей формуле:

$$Z_{\text{общ}} = Z_{\text{пр}} + Z_{\text{доп}},$$

Отчисления на соцстрах, фонд занятости и пенсионный фонд вычисляются по следующей формуле:

$$\text{Отч} = O_{\text{сс}} + O_{\text{фз}} + O_{\text{пф}},$$

где  $O_{\text{сс}}$  - отчисления на социальное страхование (0,5% от  $Z_{\text{общ}}$ );

$O_{\text{фз}}$  - отчисления в фонд занятости (0,5% от  $Z_{\text{общ}}$ );

$O_{\text{пф}}$  - отчисления в пенсионный фонд (2% от  $Z_{\text{общ}}$ )

В табл. П1.1 представлены данные по заработной плате.

Таблица П1.1

Данные по заработной плате

| Должность   | Время работы, мес. | Ст <sub>пр</sub> , руб. | З <sub>пр</sub> , руб. | З <sub>доп</sub> , руб. | З <sub>общ</sub> , руб. | Отч, руб. | З <sub>тр</sub> , руб. |
|-------------|--------------------|-------------------------|------------------------|-------------------------|-------------------------|-----------|------------------------|
| Программист | 11                 | 49 845                  | 303,93                 | 30,39                   | 334,32                  | 10,03     | 621218,2               |

Затраты на использование машинного времени вычисляются по формуле:

$$Z_{\text{м.вр}} = C_{\text{м.вр}} * \text{Вр}_{\text{в.т}},$$

где  $Z_{\text{м.вр}}$  - затраты на использование машинного времени, руб;

$C_{\text{м.вр}}$  - стоимость одного часа машинного времени, руб/ч;

$V_{\text{в.т}}$  - время использования вычислительной техники, ч.

Стоимость одного часа машинного времени рассчитывается по формуле:

$$C_{\text{м.вр}} = \frac{C_{\text{к}}}{C_{\text{сп.к}} * K_{\text{р.д}} * V_{\text{р.с}}} + C_{\text{т.э}} * M_{\text{вс}},$$

где  $C_{\text{к}}$  - покупная цена компьютера;

$C_{\text{сп.к}}$  - срок службы компьютера, год;

$K_{\text{р.д}}$  - количество рабочих дней в году;

$V_{\text{р.с}}$  - время работы компьютера в течение суток, ч;

$C_{\text{т.э}}$  - стоимость одного кВт/ч электроэнергии, руб;

$M_{\text{вс}}$  - мощность вычислительной системы, кВт.

$$C_{\text{м.вр}} = \frac{67\,684}{4 * 250 * 8} + 5,85 * 0,5 = 10,26 \text{ руб/ч}$$

Время использования вычислительной техники ( $V_{\text{в.т}}$ ) рассчитывается по следующей формуле:

$$V_{\text{в.т}} = K_{\text{д.р}} * V_{\text{р.с}},$$

где  $K_{\text{д.р}}$  - количество дней разработки ПО;

$V_{\text{р.с}}$  - время работы компьютера в течение суток, ч.

$$Bp_{в.т} = 8 * 328 = 2624 \text{ ч.}$$

$$З_{м.вр} = 10,26 * 2624 = 26922,24 \text{ руб.}$$

Затраты на носители информации принимаются в размере 2% от цены вычислительной техники.

$$З_{н.и} = Ц_{к} * 2\%$$

$$З_{н.и} = 67\,684 * 2\% = 1353,68 \text{ руб.}$$

Затраты на текущий и профилактический ремонт принимаются в размере 4% от цены вычислительной техники.

$$З_{рем} = Ц_{к} * 4\%$$

$$З_{рем} = 67\,684 * 4\% = 2707,36 \text{ руб.}$$

Прочие эксплуатационные расходы включают в себя затраты на освещение, отопление, охрану, уборку и текущий ремонт помещений. Они принимаются в размере 10% от стоимости помещения (или его аренды), где происходит разработка программного продукта.

$$З_{пр} = C_{мп} * 10\%,$$

$$З_{пр} = 18000 * 10\% = 1800 \text{ руб.}$$

где  $C_{мп}$  - стоимость помещения.

Себестоимость разработки определяется по следующей формуле:

$$C_{п.п} = З_{тр} + З_{м.вр} + З_{н.и} + З_{рем} + З_{пр}$$

$$C_{п.п} = 621218,2 + 26922,24 + 1353,68 + 2707,36 + 1800 = 654001,52 \text{ руб.}$$

### Расчет цены программного продукта

Установление определенной цены на программный продукт служит для последующей его продажи и получения прибыли. Очень важно назначить цену таким образом, чтобы она не оказалась слишком высокой или слишком низкой. Для определения минимальной цены, ниже которой разработчику будет невыгодно продавать программный продукт, используется следующая формула:

$$Ц_{п.п} = C_{п.п} * (1 + N_{пр}),$$

где  $N_{пр}$  - норматив прибыли (20%).

$$Ц_{п.п} = 654001,52 * (1 + 0,2) = 784801,82 \text{ руб.}$$

## 12. Риски проекта

Риски проекта описаны в табл. П1.2

Таблица П1.2

Риски проекта

| №<br>п/п | Риски проекта                                                                                                                                                   | Мероприятия по<br>управлению рисками                              |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| 1        | Риски, связанные с выполнением участниками своих обязательств в рамках проекта по причине форс-мажорных обстоятельств (не зависящих от воли участников проекта) | Замена участников проекта.<br>Пересмотр сроков реализации проекта |
| 2        | Риски, связанные с недостаточным финансированием проекта                                                                                                        | Приостановление проекта /<br>Пересмотр сроков реализации проекта  |



## Текст программы

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Metallurgical.Crane.Contracts.Interfaces;
namespace Metallurgical.Crane.PositionProcessing;
/// <summary>Алгоритм определения позиции по всем меткам в
кадре.</summary>
///
/// <seealso
cref="Metallurgical.Crane.PositionProcessing.ICraneDistanceMeterAlg" />
public class CraneDistanceMeterAlgByAllMarkers :
ICraneDistanceMeterAlg
{
    private readonly
    IReadOnlyList<CraneDistanceMeterAlgParameter> _parameters;
    /// <summary>Создание <see
cref="CraneDistanceMeterAlgByAllMarkers"/>.</summary>
    /// <param name="parameters">Параметры
алгоритма.</param>
    public
    CraneDistanceMeterAlgByAllMarkers(IReadOnlyList<CraneDistanceMeterAlgParameter> parameters)
    {
        _parameters = parameters;
    }
    /// <inheritdoc />
    public float Process(IReadOnlyList<IMarker> markers, int
channelId, bool isVirtual, double xOffset, float posOffset)
    {
        var notMappedBindings = _parameters
            .Where(x => !x.IsVirtual && x.ForMarker is null)
            .ToList();
        var mappedBindings = _parameters
            .Where(x => !x.IsVirtual && x.ForMarker is not null)
            .ToList();
        if(isVirtual)
        {
            notMappedBindings = _parameters
                .Where(x =>
                    x.IsVirtual
                    && x.ForMarker is null
                    && x.ChannelId == channelId)
                .ToList();
        }
        mappedBindings = _parameters
            .Where(x =>
                x.IsVirtual
                && x.ForMarker is not null
                && x.ChannelId == channelId)
            .ToList();
    }
    var fx = -1000f;
    var fxm = new float[markers.Count];
    var dfm = new float[markers.Count];
    var calcedPosCount = 0;
    for(int i = 0; i < markers.Count; i++)
    {
        var count = notMappedBindings.Count - 1;
        var markerFramePos = markers[i].MarkerPositionOnFrame.X + xOffset;
        var markerRealPos = markers[i].PositionMappedToMarker;
        var ownRxDx = mappedBindings
            .Where(x => markers[i].DecodedMarkerNumber ==
x.ForMarker)
            .ToList();
        var hasOwnRxDx = ownRxDx.Count > 0;
        if(hasOwnRxDx)
        {
            count = ownRxDx.Count - 1;
        }
        for(int j = 0; j < count; j++)
        {
            float currentRX, currentDX, nextRX, nextDX = default;
            if(hasOwnRxDx)
            {
                currentRX = ownRxDx[j].RX;
                currentDX = ownRxDx[j].DX;
                nextRX = ownRxDx[j + 1].RX;
                nextDX = ownRxDx[j + 1].DX;
            }
            else
            {
                currentRX = notMappedBindings[j].RX;
                currentDX = notMappedBindings[j].DX;
                nextRX = notMappedBindings[j + 1].RX;
                nextDX = notMappedBindings[j + 1].DX;
            }
        }
    }
}

```

Рис. П2.1. Модуль «CraneDistanceMeterAlgByAllMarkers.cs»

## П2.1 Продолжение

```

        if(currentRX <= markerFramePos && markerFramePos
<= nextRX)
        {
            var hx1 = markerFramePos - currentRX;
            var hx2 = nextRX - markerFramePos;
            var ds = hx1 * nextDX + hx2 * currentDX;
            var df = ds / (hx1 + hx2);
            fxm[i] = markerRealPos - (float)df; // позиция крана
вычисленная на основе текущего маркера
            calcedPosCount++;
        }
    }
}
if(calcedPosCount > 0)
{
    fx = fxm.Sum() / calcedPosCount;
}
return fx + posOffset;
}

public Task<float> ProcessAsync(
    IReadOnlyList<IMarker> markers,
    int channelId,
    bool isVirtual,
    double xOffset = 0,
    float posOffset = 0) => Task.Run(() => Process(markers,
channelId, isVirtual, xOffset, posOffset));
}

```

Рис. П2.1. Модуль «CraneDistanceMeterAlgByAllMarkers.cs»

```

using GreenPipes;
using Mallenom.FFmpeg;
using Mallenom.Processing.Cranes;
using MassTransit;
using Metallurgical.Crane.Configurations;
using Metallurgical.Crane.Contracts.Interfaces;
using Metallurgical.Crane.Framework;
using Metallurgical.Crane.Pipeline;
using Metallurgical.Crane.PositionProcessing;
using Metallurgical.Crane.Video;
using Metallurgical.Crane.Video.Pipeline;
using Metallurgical.Crane.Video.VideoChannels.Configurations;
using Microsoft.AspNetCore.DataProtection.KeyManagement;
using Microsoft.Extensions.Options;
using Serilog;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
namespace Metallurgical.Crane.DataProcessors
{
    public class FrameMarkersProcessor
        : IFrameMarkersProcessor,
        IConsumer<IAppVideoChannelStateContext>
    {
        {
            public class CognexLogContext { }
            private object _syncRoot = new();
            private readonly ILogger _cognexDataLogger;
            private readonly ILogger _logger;
            private readonly IOOptions<PositionProcessingConfiguration>
_posProcConfiguration;
            private readonly IOOptions<AppDescriptionConfiguration>
_appDescription;
            private readonly IOOptions<VideoChannelsConfiguration>
_channelsCfg;
            private readonly Dictionary<ChannelKey,
(IReadOnlyList<IMarker> Markers, double Pos)>
_channelsMarkers;
            private readonly IPositionProcessor _positionProcessor;
            public FrameMarkersProcessor(
                IOOptions<AppDescriptionConfiguration> appDescription,
                IOOptions<PositionProcessingConfiguration>
posProcConfiguration,
                IOOptions<VideoChannelsConfiguration> channelsCfg,
                IPositionProcessor positionProcessor)
            {
                _posProcConfiguration = posProcConfiguration;
            }
        }
    }
}

```

Рис. П2.2. Модуль «FrameMarkersProcessor.cs»

```

        _appDescription = appDescription;
        _channelsCfg = channelsCfg;
        _positionProcessor = positionProcessor;
        _cognexDataLogger =
Log.ForContext<CognexLogContext>();
        _logger = Log.ForContext<PositionFrameProcessor>();
        _channelsMarkers = new Dictionary<ChannelKey,
(IReadOnlyList<IMarker> Markers, double Pos)>>();
    }
    public IReadOnlyList<IMarker> ProcessRawMarkers(Guid
frameId, ChannelKey key, PositionMarker[] positionMarkers)
    {
        var posProcCfg = _posProcConfiguration.Value;
        var bindedMarkers = GetBindedMarkers(
            positionMarkers,
            key.ChannelId,
            key.IsVirtual,
            posProcCfg);
        if (bindedMarkers is null)
        {
            _cognexDataLogger.Debug(
                "Cognex recognition details: " +
                "ChannelId={ChannelId}" +
                "IsVirtual={IsVirtual}" +
                $"FrameId={frameId} " +
                "Position={Position:N1} " +
                "HasMarkerProcessError={IsMarkerProcErr} " +
                "FromData={@CognexRecognitionResult}",
                key.ChannelId,
                key.IsVirtual,
                null,
                false,
                positionMarkers
            );
            lock (_syncRoot)
            {
                _channelsMarkers.Remove(key);
            }
            if (_channelsMarkers.All(x => x.Value.Markers.Count
== 0))
            {
                AppMetrics
                    .ProcessedPositionGauge
                    .WithLabels(
                        _appDescription.Value.CraneId.ToString(),
                        key.ChannelId.ToString(),
                        key.IsVirtual.ToString())
                    .Set(0);
            }
        }
    }

```

```

    }
    return null;
}
return bindedMarkers;
}
public async Task<double> ProcessMappedMarkersAsync(
    Guid frameId,
    ChannelKey key,
    PositionMarker[] positionMarkers,
    IReadOnlyList<IMarker> bindedMarkers)
{
    var posProcCfg = _posProcConfiguration.Value;
    var position = await ProcessPosition(
        bindedMarkers,
        key.ChannelId,
        key.IsVirtual,
        posProcCfg)
        .ConfigureAwait(false);
    lock (_syncRoot)
    {
        _channelsMarkers[key] = (bindedMarkers, position);
        position = _channelsMarkers.Sum(x
=> x.Value.Markers.Count * x.Value.Pos) / _channelsMarkers.Sum(x
=> x.Value.Markers.Count);
    }
    _cognexDataLogger.Debug(
        "Cognex recognition details: " +
        "ChannelId={ChannelId}" +
        "IsVirtual={IsVirtual}" +
        $"FrameId={frameId} " +
        "Position={Position:N1} " +
        "HasMarkerProcessError={IsMarkerProcErr} " +
        "FromData={@CognexRecognitionResult}",
        key.ChannelId,
        key.IsVirtual,
        (int)position,
        false,
        positionMarkers
    );
    _logger.Debug(
        "ChannelId {ChannelId} processed position:
{Position:N1}. Channel {IsVirtual}",
        key.ChannelId,
        (int)position,
        key.IsVirtual
    );
    AppMetrics
        .ProcessedPositionGauge

```

```

        .WithLabels(
            _appDescription.Value.CraneId.ToString(),
            key.ChannelId.ToString(),
            key.IsVirtual.ToString())
        .Set(position);
        return position;
    }

    public IProcessedPositionContext GetContext(Guid frameId,
        double position)
    {
        var posMarkers = new List<IMarker>();
        foreach (var channel in _channelsMarkers)
        {
            posMarkers.AddRange(channel.Value.Markers);
        }
        var context = new ProcessedPositionContext(new
        ProcessedPosition(frameId, position, posMarkers));
        return context;
    }

    public IProcessedPositionContext GetContext(Guid frameId)
    {
        double? position = default;
        var posMarkers = new List<IMarker>();
        lock(_syncRoot)
        {
            if(_channelsMarkers.Any())
            {
                position = _channelsMarkers.Sum(x =>
x.Value.Markers.Count * x.Value.Pos) / _channelsMarkers.Sum(x
=> x.Value.Markers.Count);
                foreach(var channel in _channelsMarkers)
                {
                    posMarkers.AddRange(channel.Value.Markers);
                }
            }
        }
        var context = new ProcessedPositionContext(new
        ProcessedPosition(frameId, position, posMarkers));
        return context;
    }

    public double? GetCurrentPosition()
    {
        double? position = default;
        lock(_syncRoot)
        {
            position = _channelsMarkers.Sum(x =>
x.Value.Markers.Count * x.Value.Pos) / _channelsMarkers.Sum(x
=> x.Value.Markers.Count);
        }
    }

    }

    return position;
}

public IProcessedPositionContext GetContext(Guid frameId,
double position, DateTime dateTime)
{
    var posMarkers = new List<IMarker>();
    foreach(var channel in _channelsMarkers)
    {
        posMarkers.AddRange(channel.Value.Markers);
    }
    var context = new ProcessedPositionContext(new
    ProcessedPosition(frameId, position, posMarkers), dateTime);
    return context;
}

private IReadOnlyList<IMarker> GetBindedMarkers(
    PositionMarker[] rawMarkers,
    int channelId,
    bool isVirtual,
    PositionProcessingConfiguration posProcCfg)
{
    if (rawMarkers.Length == 0)
    {
        return null;
    }
    try
    {
        var markers = new List<IMarker>();
        foreach (var rawMarker in rawMarkers)
        {
            _logger.Debug(
                "ChannelId: {ChannelId} IsVirtual: {IsVirtual} " +
                "Code: {Code} Grade: {Grade} UEC:
{UnusedErrorCorrection}",
                channelId,
                isVirtual,
                rawMarker.Code,
                rawMarker.Grade,
                rawMarker.UnusedErrorCorrection
            );
            AppMetrics
                .RecognizedMarkerUecGauge
                .WithLabels(
                    _appDescription.Value.CraneId.ToString(),
                    rawMarker.Code,
                    channelId.ToString(),
                    isVirtual.ToString())
                .Set(rawMarker.UnusedErrorCorrection);
        }
    }
    catch { }
}

```

```

AppMetrics
.RecognizedMarkerGradeGauge
.WithLabels(
    _appDescription.Value.CraneId.ToString(),
    rawMarker.Code,
    channelId.ToString(),
    isVirtual.ToString())
.Set(((int)rawMarker.Grade);
var craneSpan = rawMarker.Code.Substring(0, 2);
var pos = rawMarker.Code.Substring(2);
if (int.TryParse(pos, out var markerNumber))
{
    AppMetrics
    .RecognizedCodesCount
    .WithLabels(
        _appDescription.Value.CraneId.ToString(),
        rawMarker.Code,
        channelId.ToString(),
        isVirtual.ToString())
    .Inc();
}
if
(!string.IsNullOrEmpty(posProcCfg.CraneSpan))
{
    if (craneSpan != posProcCfg.CraneSpan)
    {
        _logger.Warning($"Crane span not match. (Got
{craneSpan} Expect {posProcCfg.CraneSpan}).");
        return null;
    }
}
_logger.Verbose($" {CraneSpan}           {MappedPos}
{MarkerPosX} {MarkerPosY}",
    craneSpan,
    pos,
    rawMarker.X,
    rawMarker.Y);
if (!posProcCfg.Markers.Any(x => x.Code == pos))
{
    _logger.Warning($"Unable to map marker {pos}
with position.");
    return null;
}
var          markerMapping          =
posProcCfg.Markers.FirstOrDefault(x => x.Code == pos);
var point = new Point(rawMarker.X, rawMarker.Y);
var  grade  = Enum.GetName(typeof(Grade),
rawMarker.Grade);

var marker = new Marker(channelId, isVirtual, point,
markerMapping.Position, markerNumber, grade);
markers.Add(marker);
}
return markers;
}
catch (Exception exc)
{
    _logger.Warning(exc, "Error occurred due markers
processing.");
    return null;
}
}
private async Task<double> ProcessPosition(
    IReadOnlyList<IMarker> markers,
    int channelId,
    bool isVirtual,
    PositionProcessingConfiguration posProcCfg)
{
    var chnlsCfg = _channelsCfg.Value;
    var roiXOffset = .0;
    ROI roi = default;
    if (isVirtual)
    {
        var ch = chnlsCfg.VirtualChannels.FirstOrDefault(x =>
x.ChannelId == channelId);
        roi = ch?.ROI;
    }
    else
    {
        var ch = chnlsCfg.VideoChannels.FirstOrDefault(x =>
x.ChannelId == channelId);
        roi = ch?.ROI;
    }
    if (roi is not null)
    {
        roiXOffset = roi.X;
    }
    var position = await _positionProcessor
        .ProcessAsync(markers, channelId, isVirtual, roiXOffset,
posProcCfg.PositionOffset)
        .ConfigureAwait(continueOnCapturedContext: false);
    return position;
}
public void OnNoRecognitions(Guid frameId, ChannelKey
key)
{

```

```

        _logger.Debug($"No recognitions for {frameId} on Ch
        {{@ChKey}}", key);
        lock(_syncRoot)
        {
            _channelsMarkers.Remove(key);
        }
    }
    public Task
    Consume(ConsumeContext<IAppVideoChannelStateContext>
    context)
    {
        var msg = context.Message;
        var key = new ChannelKey(msg.Id, msg.IsVirtual,
        msg.Type);
        if(!msg.IsAvailable)
        {
            lock(_syncRoot)
            {
                _channelsMarkers.Remove(key);
            }
        }
        return Task.CompletedTask;
    }
}

```

Рис. П2.2. Модуль «FrameMarkersProcessor.cs»

```

using System.IO;
using Metallurgical.Crane.BucketStateNn;
using Metallurgical.Crane.Contracts.Enums;
using Metallurgical.Crane.Contracts.Interfaces;
namespace Metallurgical.Crane.DataProcessors;
/// <summary>Обработчик детекций ковша.</summary>
/// <seealso cref="IBucketStateProcessor" />
public class BucketStateProcessor : IBucketStateProcessor
{
    public CraneBucketState Process(IBucketDetection detection)
    {
        if (detection is null)
        {
            return new CraneBucketState();
        }
        var availability = detection.ClassId switch
        {
            0 => BucketAvailability.Unhooked,
            1 => BucketAvailability.Hooked,
            2 => BucketAvailability.Hooked,
            3 => BucketAvailability.Hooked,
            4 => BucketAvailability.Hooked,
            _ => throw new InvalidDataException(),
        };
        var state = detection.ClassId switch
        {
            0 => BucketState.None,
            1 => BucketState.Empty,
            2 => BucketState.Full,
            3 => BucketState.BigEmpty,
            4 => BucketState.BigFull,
            _ => throw new InvalidDataException(),
        };
        return new CraneBucketState(BucketAction.None,
        availability, state);
    }
}

```

Рис. П2.3. Модуль «BucketStateProcessor.cs»

```

using System.Threading.Tasks;
using GreenPipes;
using Mallenom.Framework;
using Metallurgical.Crane.Configurations;
using Metallurgical.Crane.Contracts.Enums;
using Metallurgical.Crane.Contracts.Interfaces;
using Metallurgical.Crane.Framework;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.Options;
namespace Metallurgical.Crane.Pipeline;
/// <summary>Фильтр объединения контекстов разных каналов
распознавания.</summary>
/// <seealso cref="IProcessedPositionContext" />
/// <seealso cref="IBucketStateContext" />
public class MergePipesContextsFilter :
IFilter<IProcessedPositionContext>,
    IFilter<IBucketStateContext>,
    IFilter<IWeightContext>,
    IFilter<IScenePositionContext>
{
    private IProcessedPositionContext _positionContext;
    private IBucketStateContext _bucketStateContext;
    private IWeightContext _weightContext;
    private IScenePositionContext _sceneContext;
    private readonly SingleConcurrencyBlock _syncRoot = new
SingleConcurrencyBlock();
    private readonly IPipe<ICraneStateContext> _craneStatePipe;
    private readonly long _craneId;
    private readonly string _port;
    public MergePipesContextsFilter(IPipe<ICraneStateContext>
craneStatePipe,
        IConfiguration config,
        IOptions<AppDescriptionConfiguration> appDescription)
    {
        _craneStatePipe = craneStatePipe;
        _craneId = appDescription.Value.CraneId;
        _port = config.GetValue<string>("servicePort");
    }
    public async Task Send(IProcessedPositionContext context,
IPipe<IProcessedPositionContext> next)
    {
        using(_syncRoot.Enter())
        {
            _positionContext = context;
            CraneStateContext stateContext = new
CraneStateContext(_craneId,
                context.FrameId,
                    null,
                    _port,
                    BucketAvailability.None,
                    BucketState.None,
                    BucketAction.None,
                    null,
                    context.Position,
                    false,
                    context.Position.HasDetections,
                    context.Timestamp,
                    null,
                    null,
                    null,
                    null,
                    null,
                    null);
            if(_bucketStateContext != default)
            {
                stateContext.ApplyContext(_bucketStateContext);
            }
            if(_weightContext != default)
            {
                stateContext.ApplyContext(_weightContext);
            }
            if(_sceneContext != default)
            {
                stateContext.ApplyContext(_sceneContext);
            }
            await
_craneStatePipe.Send(stateContext).ConfigureAwait(continueOnC
apturedContext: false);
        }
        await
next.Send(context).ConfigureAwait(continueOnCapturedContext:
false);
    }
    public void Probe(ProbeContext context)
    {
        context.CreateScope(nameof(MergePipesContextsFilter));
    }
    public async Task Send(IBucketStateContext context,
IPipe<IBucketStateContext> next)
    {
        using(_syncRoot.Enter())
        {

```

Рис. П2.4. Модуль «MergePipesContextsFilter.cs»

```

        _bucketStateContext = context;
        CraneStateContext stateContext = new
        CraneStateContext(_craneId,
            null,
            context.FrameId,
            _port,
            context.HaveBucket,
            context.BucketState,
            context.BucketAction,
            context.BBox,
            null,
            context.HasDetections,
            false,
            null,
            context.Timestamp,
            null,
            null,
            context.BucketNNResult,
            null,
            null,
            null);
        if(_positionContext != default)
        {
            stateContext.ApplyContext(_positionContext);
        }
        if(_weightContext != default)
        {
            stateContext.ApplyContext(_weightContext);
        }
        if(_sceneContext != default)
        {
            stateContext.ApplyContext(_sceneContext);
        }
        await
        _craneStatePipe.Send(stateContext).ConfigureAwait(continueOnC
        apturedContext: false);
    }
    await
    next.Send(context).ConfigureAwait(continueOnCapturedContext:
    false);
}
public async Task Send(IWeightContext context,
IPipe<IWeightContext> next)
{
    using(_syncRoot.Enter())
    {
        _weightContext = context;
        CraneStateContext stateContext = new
        CraneStateContext(_craneId,
            null,
            null,
            _port,
            BucketAvailability.None,
            BucketState.None,
            BucketAction.None,
            null,
            null,
            false,
            false,
            null,
            null,
            null,
            null,
            null,
            _weightContext.Weight,
            _weightContext.RequestId);
        if(_positionContext != default)
        {
            stateContext.ApplyContext(_positionContext);
        }
        if(_bucketStateContext != default)
        {
            stateContext.ApplyContext(_bucketStateContext);
        }
        if(_sceneContext != default)
        {
            stateContext.ApplyContext(_sceneContext);
        }
        await
        _craneStatePipe.Send(stateContext).ConfigureAwait(continueOnC
        apturedContext: false);
    }
    await
    next.Send(context).ConfigureAwait(continueOnCapturedContext:
    false);
}
public async Task Send(IScenePositionContext context,
IPipe<IScenePositionContext> next)
{
    using(_syncRoot.Enter())
    {
        _sceneContext = context;
        CraneStateContext stateContext = new
        CraneStateContext(_craneId,

```



```

    null,
    null,
    _port,
    BucketAvailability.None,
    BucketState.None,
    BucketAction.None,
    null,
    null,
    false,
    false,
    null,
    null,
    null,
    null,
    null,
    null,
    null,
    null,
    context.Position);
if(_positionContext != default)
{
    stateContext.ApplyContext(_positionContext);
}
if(_bucketStateContext != default)
{
    stateContext.ApplyContext(_bucketStateContext);
}
if(_weightContext != default)
{
    stateContext.ApplyContext(_weightContext);
}
await
_craneStatePipe.Send(stateContext).ConfigureAwait(continueOnC
apturedContext: false);
}
await
next.Send(context).ConfigureAwait(continueOnCapturedContext:
false);
}

```

```

using GreenPipes;
using Mallenom.FFmpeg;
using Mallenom.Framework;
using Mallenom.Imaging;
using Mallenom.Processing.Cranes;
using Mallenom.Processing.Imaging;
using Metallurgical.Crane.Configurations;
using Metallurgical.Crane.Contracts.Interfaces;
using Metallurgical.Crane.DataProcessors;
using Metallurgical.Crane.Exporters;
using Metallurgical.Crane.Framework;
using Metallurgical.Crane.Infrastructure;
using Metallurgical.Crane.Pipeline;
using Metallurgical.Crane.Video.Pipeline;
using Metallurgical.Crane.Video.VideoChannels.Configurations;
using Microsoft.Extensions.Options;
using Serilog;
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.IO;
using System.Threading;
using System.Threading.Tasks;
namespace Metallurgical.Crane.Video;
public class PositionFrameProcessor : IFrameProcessor
{
    private static TimeSpan RATE_FILTER_TIMEOUT =
TimeSpan.FromMilliseconds(800);
    private readonly ILogger _logger;
    private readonly IImageProcessingServiceClient
_imageProcessor;
    private readonly IPosImageExportingService
_imageExportingService;
    private readonly Reference<ImageDataAllocator> _allocator;
    private readonly IOptions<PositionProcessingConfiguration>
_posProcConfiguration;
    private readonly IOptions<AppDescriptionConfiguration>
_appId;
    private readonly IFrameMarkersProcessor _markersProcessor;
    private readonly IPipe<IProcessedPositionContext>
_processedPosPipe;
    private readonly RecognitionRpmWatcher _fps;
    private Dictionary<ChannelKey,
NoWaitRateLimitFilter<ICutedVideoFrameContext>.Implementati
on> _rateFiltersImpl;
    public VideoChannelType ChannelType =>
VideoChannelType.Position;
    public FrameProcessorType ProcessorType =>
FrameProcessorType.PositionByMarkers;

    public int InitializationPriority =>
InitializationPriorities.AutoActivated;
    public bool IsInitialized { get; private set; }
    public PositionFrameProcessor(
        IImageProcessingServiceClient imageProcessor,
        IPosImageExportingService imageExportingService,
        Reference<ImageDataAllocator> allocator,
        IFrameMarkersProcessor markersProcessor,
        IPipe<IProcessedPositionContext> processedPosPipe,
        IOptions<AppDescriptionConfiguration> appDescription,
        IOptions<PositionProcessingConfiguration>
posProcConfiguration)
    {
        _logger = Log.ForContext<PositionFrameProcessor>();
        _posProcConfiguration = posProcConfiguration;
        _appId = appDescription;
        _imageProcessor = imageProcessor;
        _imageExportingService = imageExportingService;
        _allocator = allocator;
        _processedPosPipe = processedPosPipe;
        _markersProcessor = markersProcessor;
        _fps = new RecognitionRpmWatcher(appDescription,
"position");
        _rateFiltersImpl = new();
    }
    public async Task ProcessAsync(ICutedVideoFrameContext
videoFrame)
    {
        var key = new ChannelKey(videoFrame.VideoChannelId,
videoFrame.IsVirtual, videoFrame.ChannelType);
        if(!_rateFiltersImpl.ContainsKey(key))
        {
            _rateFiltersImpl[key] = new(RATE_FILTER_TIMEOUT);
        }
        var rateFilter = _rateFiltersImpl[key];
        var isCanGoNext = rateFilter.CanGoNext();
        if(!isCanGoNext)
        {
            return;
        }
        var sw = new Stopwatch();
        sw.Start();
        var posProcCfg = _posProcConfiguration.Value;
        var vf = videoFrame.FrameImageDataReference.Value;
        using var tmp = new
Mallenom.Framework.Reference<ImageData>();
        if(_allocator.Value.TryAllocate(tmp,
ImageDataFormat.BGR24, vf.Width, vf.Height))

```

Рис. П2.5. Модуль «PositionFrameProcessor.cs»

```

{
    ColorSpaceConverter.Convert(vf, tmp.Value);
}
else
{
    _markersProcessor.OnNoRecognitions(videoFrame.FrameId, key);
    _logger.Warning("Unable to allocate for position frame");
    return;
}

var rawMarkers = await _imageProcessor
    .ProcessImageAsFileAsync<PositionMarker[]>(
        posProcCfg.JobName,
        tmp)
    .ConfigureAwait(continueOnCapturedContext: false);
sw.Stop();
if(!key.IsVirtual)
{
    _fps.Update();
}

AppMetrics
    .PositionRecognitionDurationGauge
    .WithLabels(
        _appId.Value.CraneId.ToString(),
        key.ChannelId.ToString(),
        key.IsVirtual.ToString())
    .Set(sw.ElapsedMilliseconds);

AppMetrics
    .PositionRecognitionsCountGauge
    .WithLabels(
        _appId.Value.CraneId.ToString(),
        key.ChannelId.ToString(),
        key.IsVirtual.ToString())
    .Inc();

AppMetrics
    .RecognizedPositionPointsCountGauge
    .WithLabels(
        _appId.Value.CraneId.ToString(),
        key.ChannelId.ToString(),
        videoFrame.IsVirtual.ToString())
    .Set(rawMarkers.Length);
_logger.Debug($"Cognex recognized {rawMarkers.Length}
markers on channel {key.ChannelId}.");
if(rawMarkers.Length == 0)
{
    _markersProcessor.OnNoRecognitions(videoFrame.FrameId, key);
    await NextWithoutPosition(videoFrame.FrameId)
        .ConfigureAwait(false);
    return;
}

var
    bindedMarkers =
_markersProcessor.ProcessRawMarkers(videoFrame.FrameId, key,
rawMarkers);
if(bindedMarkers is null)
{
    await _imageExportingService
        .TakeShapshot(PositionSnapshotReason.UnknownMarker)
        .ConfigureAwait(false);
    _markersProcessor.OnNoRecognitions(videoFrame.FrameId, key);
    await NextWithoutPosition(videoFrame.FrameId)
        .ConfigureAwait(false);
    return;
}

var position = await _markersProcessor
    .ProcessMappedMarkersAsync(videoFrame.FrameId, key,
rawMarkers, bindedMarkers)
    .ConfigureAwait(false);
_imageExportingService.UpdateLastPos(position);
if(position < 0)
{
    await _imageExportingService
        .TakeShapshot(PositionSnapshotReason.Algorithm)
        .ConfigureAwait(false);
}

var
    context =
_markersProcessor.GetContext(videoFrame.FrameId, position);
try
{
    await _processedPosPipe
        .Send(context)
        .ConfigureAwait(continueOnCapturedContext: false);
}
catch(Exception exc)
{
    _logger.Warning(exc, "Unable to export crane position.");
}

private async Task NextWithoutPosition(Guid frameId)
{
    var context = _markersProcessor.GetContext(frameId);
    try
    {
        await _processedPosPipe

```

```

        .Send(context)
        .ConfigureAwait(continueOnCapturedContext: false);
    }
    catch(Exception exc)
    {
        _logger.Warning(exc, "Unable to export crane without
position.");
    }
}

public async Task InitializeAsync(CancellationToken
cancellationToken)
{
    const int WARM_UP_COUNT = 3;
    for(int i = 0; i < WARM_UP_COUNT; i++)
    {
        _logger.Debug("Warm up Cognex {Now}/{Total}.", i + 1,
WARM_UP_COUNT);
        using var img = new Reference<ImageData>();
        var isAllocated = _allocator.Value.TryAllocate(img,
ImageDataFormat.YUV410P, 10, 10);
        if(!isAllocated)
        {
            throw new Exception("Unable to allocate memory.");
        }
        try
        {
            await _imageProcessor

.ProcessImageAsFileAsync<PositionMarker[]>(_posProcConfigur
ation.Value.JobName, img)
            .ConfigureAwait(continueOnCapturedContext: false);
            _logger.Debug("Warm up Cognex {Now}/{Total}.
Done.", i + 1, WARM_UP_COUNT);
        }
        catch(Exception exc)
        {
            _logger.Debug(exc, "Warm up Cognex {Now}/{Total}.
Failed.", i + 1, WARM_UP_COUNT);
        }
    }
    await _fps
        .InitializeAsync(cancellationToken)
        .ConfigureAwait(false);
    IsInitialized = true;
}
}

```

```

using GreenPipes;
using Metallurgical.Crane.BucketStateNn;
using Metallurgical.Crane.Contracts;
using Metallurgical.Crane.DataProcessors;
using Metallurgical.Crane.Framework;
using Metallurgical.Crane.Infrastructure;
using Metallurgical.Crane.Pipeline;
using Metallurgical.Crane.Video.Pipeline;
using Metallurgical.Crane.Video.VideoChannels.Configurations;
using Microsoft.Extensions.Options;
using Serilog;
using System;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;
namespace Metallurgical.Crane.Video;
/// <summary>Обработчик кадров.</summary>
/// <seealso cref="Metallurgical.Crane.Video.IFrameProcessor" />
public class BucketFrameProcessor : IFrameProcessor
{
    public class BucketNnLogContext { }
    private readonly IBucketStateProcessor _bucketStateProcessor;
    private readonly IBucketRecognizerClient _bucketNnClient;
    private readonly IPipe<IBucketStateContext> _bucketStatePipe;
    private readonly long _appId;
    private readonly ILogger _rawBucketNnDataLogger;
    private readonly RecognitionRpmWatcher _fps;
    private readonly ILogger _log;
    public VideoChannelType ChannelType =>
VideoChannelType.Bucket;
    public FrameProcessorType ProcessorType =>
FrameProcessorType.BucketState;
    public int InitializationPriority =>
InitializationPriorities.AutoActivated;
    public bool IsInitialized { get; private set; }
    public BucketFrameProcessor(IBucketRecognizerClient
bucketNnClient,
    IBucketStateProcessor bucketStateProcessor,
    IPipe<IBucketStateContext> bucketStatePipe,
    IOptions<AppDescriptionConfiguration> appDescription)
    {
        _bucketStateProcessor = bucketStateProcessor;
        _bucketNnClient = bucketNnClient;
        _bucketStatePipe = bucketStatePipe;
        _appId = appDescription.Value.CraneId;
        _log = Log.ForContext<BucketFrameProcessor>();

        _rawBucketNnDataLogger =
Log.ForContext<BucketNnLogContext>();
        _fps = new RecognitionRpmWatcher(appDescription, "bucket");
    }
    public async Task ProcessAsync(ICutedVideoFrameContext
videoFrame)
    {
        BucketDetectionResult bucketState = default;
        try
        {
            bucketState = await
_bucketNnClient.ProcessImageAsync(videoFrame.FrameImageData
Reference)
                .ConfigureAwait(continueOnCapturedContext: false);
            _log.Debug("Received bucket state");
            if(!videoFrame.IsVirtual)
            {
                _fps.Update();
            }
            if(bucketState is null)
            {
                return;
            }
        }
        catch(Exception exc)
        {
            _log.Error(exc, "Unable to process bucket image.");
            return;
        }
        _log.Debug("Has detections: {HasDetections}.",
bucketState.HasDetections);
        AppMetrics
        .BucketNnHasDetectionsGauge
        .WithLabels(_appId.ToString())
        .Set(bucketState.HasDetections ? 1 : 0);
        await ProcessStateDataAsync(videoFrame.FrameId,
bucketState)
            .ConfigureAwait(false);
    }
    private async Task ProcessStateDataAsync(Guid frameId,
BucketDetectionResult bucketState)
    {
        var bucket = bucketState?.Detections?.FirstOrDefault();
        BucketStateContext context;
        if(bucketState.HasDetections)

```

Рис. П2.6. Модуль «BucketFrameProcessor.cs»

```

{
    AppMetrics
        .DetectedBucketClassGauge
        .WithLabels(_appId.ToString())
        .Set(bucket.ClassId);
    _rawBucketNnDataLogger
        .Debug("Bucket recognition details: " +
            $"frameId={frameId} " +
            "detectionsCount={DetectionsCount} " +
            "detectionResult={@DetectionResult}",
            bucketState.Detections.Length,
            bucketState);
}
else
{
    AppMetrics
        .DetectedBucketClassGauge
        .Unpublish();
    _rawBucketNnDataLogger
        .Debug("Bucket recognition details: " +
            $"frameId={frameId} " +
            "detectionsCount={DetectionsCount} " +
            "detectionResult={@DetectionResult}",
            0,
            bucketState);
}
try
{
    var state = _bucketStateProcessor.Process(bucket);
    context = new BucketStateContext(frameId,
        state.BucketAvailability,
        state.BucketState,
        state.BucketAction,
        bucket?.BucketBBox,
        bucketState.HasDetections,
        bucketState);
}
catch(Exception exc)
{
    _log.Error(exc, "Unable to process bucket state.");
    return;
}
_log.Debug("Have bucket: {HaveBucket}.",
context.HaveBucket);
_log.Debug("Bucket action: {BucketAction}.",
context.BucketAction);
_log.Debug("Bucket state: {BucketState}.",
context.BucketState);

_log.Verbose("Bucket position (BuckPosX): {BuckX}.",
bucket?.BucketBBox.X);
_log.Verbose("Bucket position (BuckPosY): {BuckY}.",
bucket?.BucketBBox.Y);
AppMetrics
    .HaveBucketGauge
    .WithLabels(_appId.ToString())
    .Set((int)context.HaveBucket);
AppMetrics
    .DetectedBucketActionGauge
    .WithLabels(_appId.ToString())
    .Set((int)context.BucketAction);
AppMetrics
    .DetectedBucketStateGauge
    .WithLabels(_appId.ToString())
    .Set((int)context.BucketState);
if(bucket != default)
{
    AppMetrics
        .DetectedBucketPosXGauge
        .WithLabels(_appId.ToString())
        .Set(bucket.BucketBBox.X);
    AppMetrics
        .DetectedBucketPosYGauge
        .WithLabels(_appId.ToString())
        .Set(bucket.BucketBBox.Y);
    _log.Verbose("Bucket confidence: {Conf}.", bucket.Conf);
    _log.Verbose("Bucket class confidence: {ClassConf}.",
bucket.ClassConfidence);
    AppMetrics
        .DetectedBucketConfGauge
        .WithLabels(_appId.ToString())
        .Set(bucket.Conf);
    AppMetrics
        .DetectedBucketClassConfGauge
        .WithLabels(_appId.ToString())
        .Set(bucket.ClassConfidence);
}
else
{
    AppMetrics
        .DetectedBucketPosXGauge
        .WithLabels(_appId.ToString())
        .Set(0);
    AppMetrics
        .DetectedBucketPosYGauge
        .WithLabels(_appId.ToString())
        .Set(0);
}

```

```
        _log.Debug("No buckets for frame.");
    }
    try
    {
        await _bucketStatePipe
            .Send(context)
            .ConfigureAwait(continueOnCapturedContext: false);
    }
    catch(Exception exc)
    {
        _log.Warning(exc, "Unable to export crane bucket state.");
    }
}

public async Task InitializeAsync(CancellationTokencancellationTokencancellationToken)
{
    await _fps
        .InitializeAsync(cancellationToken)
        .ConfigureAwait(false);
    IsInitialized = true;
}
}
```

### Спецификация

Настоящее приложение содержит спецификацию на разработанную программную документацию и программное обеспечение. Спецификация приведена в табл. ПЗ.1.

Таблица ПЗ.1

#### Спецификации на разработанную программную документацию

| Обозначение                     | Назначение                                                                                                           | Примечание |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------|------------|
| 1                               | 2                                                                                                                    | 3          |
| Документация                    |                                                                                                                      |            |
| Адамов_ДС_ВКР.docx              | Расчетно-пояснительная записка к магистерской диссертации                                                            |            |
| Компоненты                      |                                                                                                                      |            |
| IServiceCollectionExtensions.cs | Статический класс, расширяющий интерфейс IServiceCollection                                                          |            |
| StatesController.cs             | Обработчик запроса на получение текущего состояния мостовых кранов в цехе                                            |            |
| AggregationConfiguration.cs     | Класс описывающий файл конфигурации приложения                                                                       |            |
| AggregatedData.cs               | Класс, представляющий собой информацию о запросе состояния крана и состояние крана в формате JSON                    |            |
| AggregationWorker.cs            | Класс, реализующий фоновый процесс, который периодически опрашивает сервисы кранов цеха, зарегистрированные в Consul |            |



| 1                        | 2                                                                                                                  | 3 |
|--------------------------|--------------------------------------------------------------------------------------------------------------------|---|
| Program.cs               | Класс, представляющий собой точку входа в приложение SatesAggregator                                               |   |
| Startup.cs               | Класс, производящий настройку сервисов в контейнере внедрения зависимостей ASP.NET приложения SatesAggregator      |   |
| StatesStorage.cs         | Класс, реализующий сохранение и извлечение из памяти перечня состояний кранов цеха                                 |   |
| AppMetrics.cs            | Статический класс, содержащий набор статических полей, обозначающих ту или иную метрику приложения                 |   |
| ChannelKey.cs            | Структура, идентифицирующая видеоканал                                                                             |   |
| Program.cs               | Точка входа в программу Crane                                                                                      |   |
| Startup.cs               | Класс, производящий настройку сервисов в контейнере внедрения зависимостей ASP.NET приложения Crane                |   |
| WeightWorker.cs          | Класс, реализующий фоновый процесс, который периодически опрашивает АСУ ТП и извлекает вес объекта в захвате крана |   |
| IBucketStateProcessor.cs | Интерфейс, описывающий обработчик кадров камеры слежения за сталь-ковшом                                           |   |
| BucketStateProcessor.cs  | Класс, реализующий IBucketStateProcessor                                                                           |   |

| 1                                  | 2                                                                                                                                            | 3 |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|---|
| IFrameMarkersProcessor.cs          | Интерфейс, описывающий<br>обработчик кадров камеры<br>позиционирования                                                                       |   |
| FrameMarkersProcessor.cs           | Класс, реализующий<br>IFrameMarkersProcessor                                                                                                 |   |
| IScenePositionRecognitionClient.cs | Интерфейс, клиента<br>распознавателя позиции по<br>наблюдаемой сцене                                                                         |   |
| ScenePositionOnnxClient.cs         | Класс, реализующий<br>IScenePositionRecognitionClient                                                                                        |   |
| IBucketLocationMapper.cs           | Интерфейс, описывающий<br>обработчик наименования<br>текущей позиции ковша на<br>крановом пути                                               |   |
| BucketLocationMapper.cs            | Класс, реализующий<br>IBucketLocationMapper                                                                                                  |   |
| IBucketRecognizerClient.cs         | Интерфейс, клиента<br>распознавателя                                                                                                         |   |
| BucketOnnxClient.cs                | Класс, реализующий<br>IBucketRecognizerClient                                                                                                |   |
| CraneBucketState.cs                | Структура, описывающая<br>состояние сталь-ковша                                                                                              |   |
| RecognitionRequestType.cs          | Тип сообщения на<br>распознавание. Предоставляет<br>два возможных значения:<br>распознавание сталь-ковша и<br>распознавание позиции по сцене |   |
| ComplexAlgorithmDetectionResult.cs | Класс, реализующий ответ,<br>содержащий несколько<br>распознанных сталь-ковшей                                                               |   |

| 1                                      | 2                                                                                                                                                                                                           | 3 |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| PositionRecognitionRequestMessage.cs   | Класс, реализующий сообщение на распознавание позиции по наблюдаемой сцене                                                                                                                                  |   |
| CraneLadleRecognitionPipelineResult.cs | Класс, реализующий распознанный сталь-ковш                                                                                                                                                                  |   |
| RecognitionRequestBase.cs              | Базовый класс, реализующий сообщение на распознавание                                                                                                                                                       |   |
| IFrameProcessor.cs                     | Интерфейс, описывающий обработчик кадра                                                                                                                                                                     |   |
| BucketFrameProcessor.cs                | Класс, реализующий обработчик кадра, который распознаёт состояние захвата и сталь-ковша                                                                                                                     |   |
| PositionFrameProcessor.cs              | Класс, реализующий обработчик кадра, который распознаёт позицию по маркерам                                                                                                                                 |   |
| SceneFrameProcessor.cs                 | Класс, реализующий обработчик кадра, который распознаёт позицию по наблюдаемой сцене                                                                                                                        |   |
| FrameProcessorType.cs                  | Тип обработчика кадров                                                                                                                                                                                      |   |
| FFmpegFrameReciver.cs                  | Класс, реализующий получение кадров от камер. И скалывающий их в очередь, в которой слишком старые кадры удаляются                                                                                          |   |
| FrameQueueWorker.cs                    | Класс, реализующий фоновый процесс, который запускает несколько параллельных подпроцессов. Каждый подпроцесс периодически забирает кадр из очереди класса FFmpegFrameReciver и рассылает его через медиатор |   |

| 1                                | 2                                                                                                                                                                                  | 3 |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| CuttetFrameConsumer.cs           | Класс, реализующий получение обрезанного кадра. В зависимости от типа и идентификатора канала, кадр передаётся определённому обработчику кадров                                    |   |
| RoiTransformedFrameConsumer.cs   | Класс, реализующий получение преобразованного кадра и обрезание его по области ROI. Далее кадр рассылается через медиатор                                                          |   |
| StoreTransformedFrameConsumer.cs | Класс, реализующий получение преобразованного кадра. Полученный кадр сохраняется в память                                                                                          |   |
| TransformRawFrameConsumer.cs     | Класс, реализующий получение кадра от FrameQueueWorker. Над кадром выполняются преобразования. Например, убираются геометрические искажения. Далее кадр рассылается через медиатор |   |
| VirtualChannelConsumer.cs        | Класс, реализующий получение преобразованного кадра и формирующий виртуальный видеоканал. Далее кадр рассылается через медиатор                                                    |   |
| ICraneStateContext.cs            | Интерфейс, описывающий контекст состояния крана                                                                                                                                    |   |

| 1                             | 2                                                                                                                                                                       | 3 |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| IBucketStateContext.cs        | Интерфейс, описывающий контекст состояния сталь-ковша и захвата                                                                                                         |   |
| IScenePositionContext.cs      | Интерфейс, описывающий контекст позиции по наблюдаемой сцене                                                                                                            |   |
| IWeightContext.cs             | Интерфейс, описывающий контекст веса объекта в захвате крана                                                                                                            |   |
| MergePipesContextsFilter.cs   | Класс, реализующий фильтр, который объединяет такие контексты как:<br>IBucketStateContext,<br>IScenePositionContext,<br>IWeightContext в контекст<br>ICraneStateContext |   |
| OutputDataFilter.cs           | Класс, получающий сообщения с типом контекста<br>ICraneStateContext и дополняющий этот контекст новыми данными, на основании анализа полученных                         |   |
| MovingFilter.cs               | Класс, реализующий фильтр контекста ICraneStateContext. Этот фильтр определяет наличие движения мостового крана                                                         |   |
| InMemoryCraneStateExporter.cs | Класс, реализующий хранилище текущего состояния мостового крана                                                                                                         |   |
| BucketImageStorage.cs         | Класс, реализующий хранилище текущего кадра ковшевой камеры                                                                                                             |   |

| 1                               | 2                                                                                                                                                            | 3 |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| PosImageStorage.cs              | Класс, реализующий хранилище текущего кадра камеры позиционирования                                                                                          |   |
| CraneStateController.cs         | Класс, реализующий обработчик HTTP запросов на получение состояния крана                                                                                     |   |
| DetailedCraneStateController.cs | Класс, реализующий обработчик HTTP запросов на получение детального состояния крана                                                                          |   |
| LastImageController.cs          | Класс, реализующий обработчик HTTP запросов на получение текущего изображения видеоканала                                                                    |   |
| ApplicationBuilderExtensions.cs | Статический класс, расширяющий тип ApplicationBuilder                                                                                                        |   |
| ICraneStateContextExtensions.cs | Статический класс, расширяющий тип ICraneStateContext                                                                                                        |   |
| InfrastructureRegistrations.cs  | Статический класс, расширяющий тип IServiceCollection. Расширение специализировано на регистрации в контейнере внедрения зависимостей инфраструктуры сервиса |   |
| IServiceCollectionExtensions.cs | Статический класс, расширяющий тип IServiceCollection                                                                                                        |   |

| 1                                         | 2                                                                                                                        | 3 |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|---|
| AdditionalFrameProcessorsConfiguration.cs | Класс, реализующий конфигурацию дополнительного обработчика кадров                                                       |   |
| BucketToLocationConfiguration.cs          | Класс, реализующий конфигурацию привязки местоположения сталь-ковша на крановом пути к наименованию позиции              |   |
| MovingFilterConfiguration.cs              | Класс, реализующий конфигурацию фильтра, определяющего наличие движения крана                                            |   |
| OnnxClientConfiguration.cs                | Класс, реализующий конфигурацию клиента сервиса запуска нейронных сетей                                                  |   |
| PositionProcessingConfiguration.cs        | Класс, реализующий конфигурацию обработчика кадра камеры позиционирования, который получает позицию по маркерам          |   |
| ScenePositionProcessingConfiguration.cs   | Класс, реализующий конфигурацию обработчика кадра камеры позиционирования, которые получает позицию по наблюдаемой сцене |   |
| WeightWorkerConfiguration.cs              | Класс, реализующий конфигурацию фонового процесса, получающего с АСУ ТП значения крановых весов                          |   |
| BBox.cs                                   | Класс, реализующий рамку детекции                                                                                        |   |

Продолжение табл. ПЗ.1

| 1                                 | 2                                                              | 3 |
|-----------------------------------|----------------------------------------------------------------|---|
| BucketDetectionResult.cs          | Класс, реализующий результат детекции сталь-ковша              |   |
| DetailedSerializableCraneState.cs | Класс, реализующий детальное состояние крана                   |   |
| NotAggregatedData.cs              | Класс, реализующий состояние крана до фильтра OutputDataFilter |   |
| SerializableCraneState.cs         | Класс, реализующий состояние крана                             |   |
| BucketAction.cs                   | Перечисление действий, производимых над ковшом                 |   |
| BucketAvailability.cs             | Перечисление наличия ковша в захвате крана                     |   |
| BucketState.cs                    | Перечисление состояний сталь-ковша в захвате крана             |   |



## Руководство пользователя

## 1. Описание программы

Данное программное обеспечение предназначено для повышения оперативности локализации сталь-ковшей и идентификации их состояния на крановом пути.

Результатом работы программы является текущее состояние мостовых кранов цеха.

## 2. Минимальные системные требования

Перед запуском необходимо убедиться, что компьютер, на котором запускается программный продукт, соответствует следующим минимальным требованиям:

- Microsoft Windows Server 2019 Standard;
- NET Framework 4.8;
- NET 7 Runtime;
- установленный драйвер NVidia версии 451.48 или более поздняя версия, совместимая с CUDA 11.0;
- Nomad версии 1.3.1 и выше;
- Consul версии 1.12.2 и выше;
- Consul template версии 0.29.0 и выше;
- Nginx версии 1.23.0 и выше;
- Loki версии 2.7.1 и выше;
- Grafana версии 9.3.1 и выше;
- Prometheus версии 2.36.2 и выше.

Программа должна функционировать на серверах со следующими характеристиками:

- процессор Intel Xeon Gold с частотой 2,99 ГГц или большей;
- объем ОЗУ минимум 32 ГБ;
- видеокарта Nvidia GeForce 1070Ti или более мощная с поддержкой технологии CUDA 11.0;
- дополнительные 1 ТБ свободного места на жёстком диске.

### 3. Запуск программы

Для запуска программного продукта необходимо открыть браузер и ввести следующий адрес URL: `http://адрес_сервера_инфраструктуры:4302`

После запроса появится главное окно модуля визуализации данных системы контроля перемещения объектов мостовыми кранами.

Если вместо работы с графическим интерфейсом требуется работа с API системы, то по URL: `http://адрес_сервера:4300/swagger` доступен графический интерфейс набора инструментов для разработчиков API. Сервис, размещённый на порту 4300 работает с состояниями кранов. Для работ с API сервиса, агрегирующего детальные состояния, следует перейти по URL: `http://адрес_сервера:4301/swagger`

### 4. Работа с программой

Графический интерфейс набора инструментов для разработчиков API позволяет ознакомиться с предоставляемым сервисом REST API. Также есть возможность выполнить один из представленных запросов и посмотреть ответ сервиса. Графический интерфейс интегрированной в сервис утилиты swagger представлен на рис. П4.1.

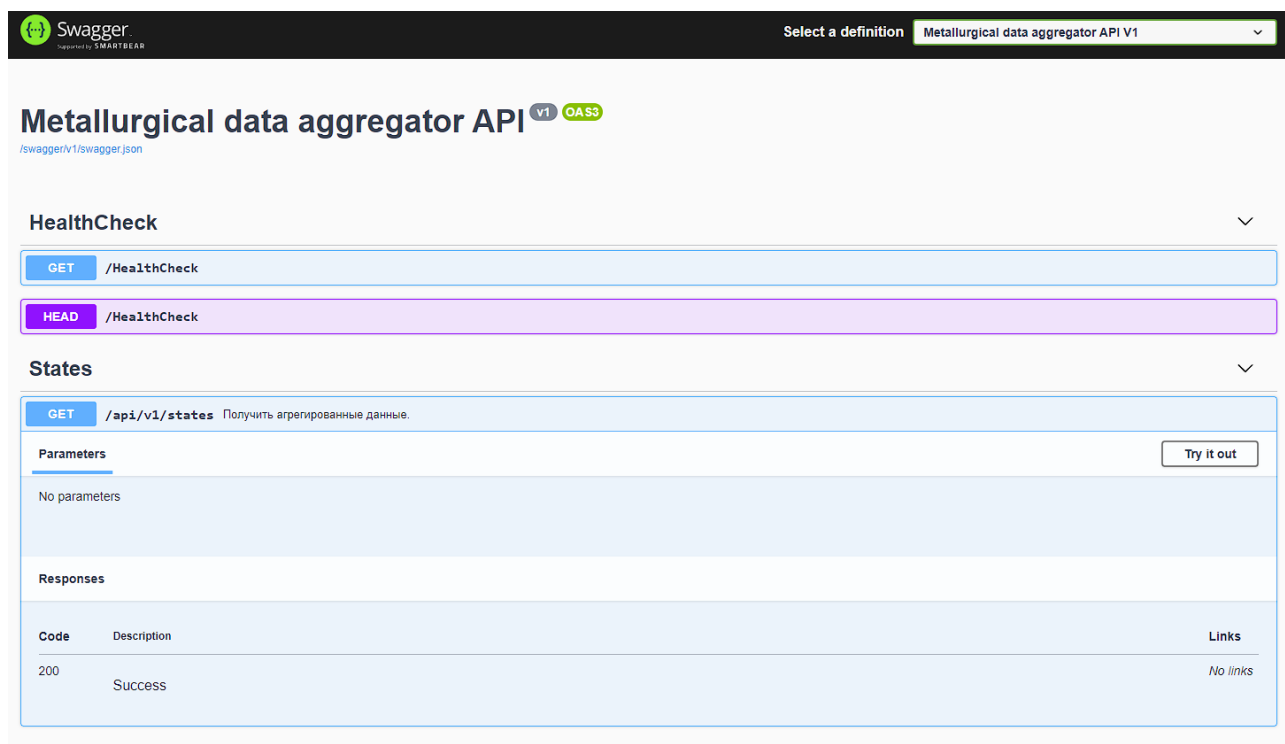


Рис. П4.1. Интерфейс интегрированной утилиты swagger

При обращении к модулю визуализации, пользователю будет представлена главная страница (рис. П4.2).

На ней отображается:

- наименование мостового крана и цветовой индикатор его сервиса;
- информация о сервере, сервисе, коде состояния запроса к сервису и агрегатору;
- информация о позиционировании, которая включает текущую позицию, информацию о наличии движения, информацию о позиции последней остановки мостового крана;
- информацию о состоянии сталь-ковша. К ней относится: наличие ковша в захвате, состояние ковша, текущее действие захвата, информация о весе ковша, наименование местоположения сталь-ковша;
- информацию о распознавании позиции и кадр камеры позиционирования;
- информацию о распознавании состояния сталь-ковша и кадр ковшевой камеры.

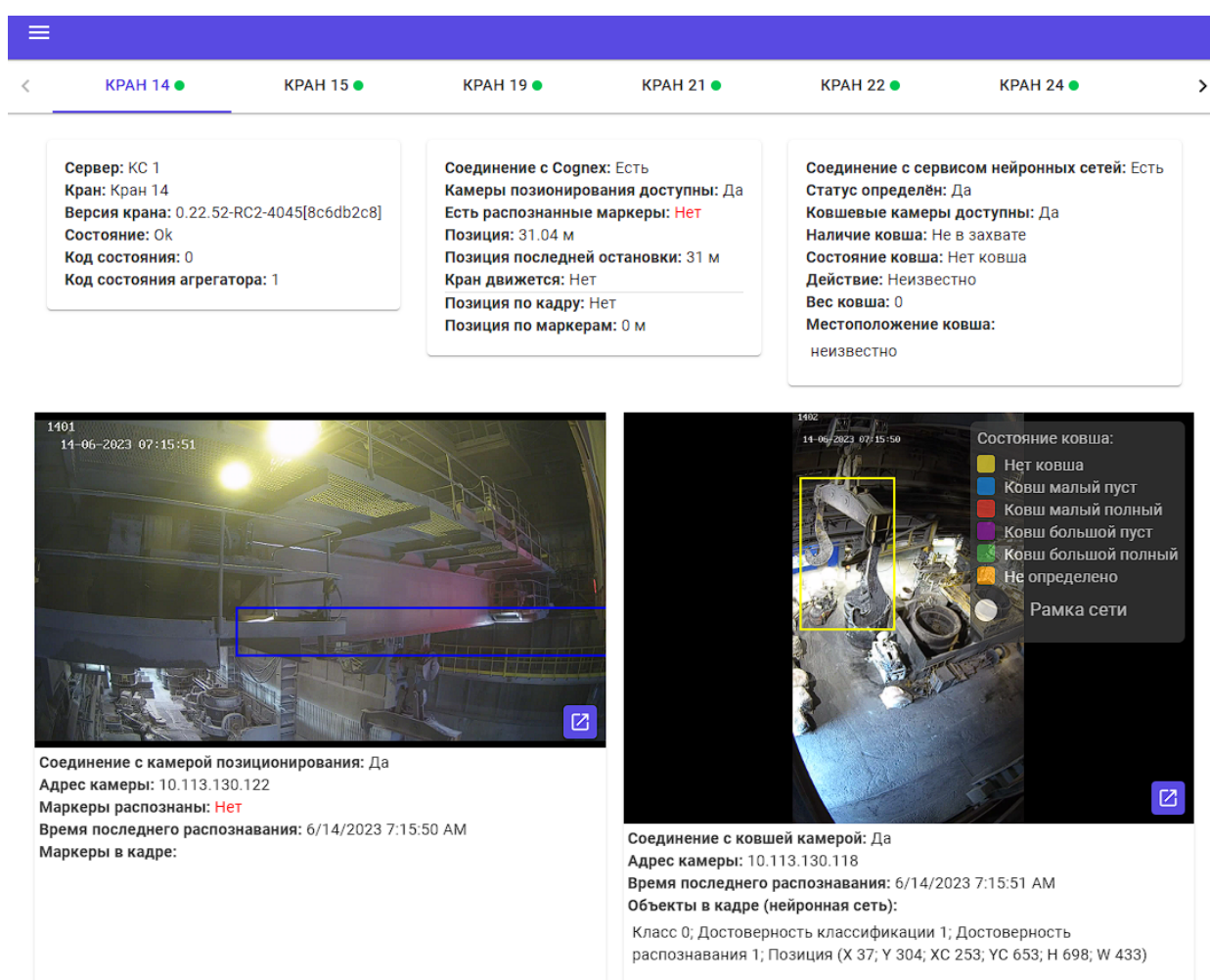


Рис. П4.2. Главная страница модуля визуализации данных системы

На странице также присутствует боковое меню (рис. П4.3), предоставляющее как ссылки на другие Web-интерфейсы системы, так и на другие части модуля визуализации.

Группа «Сервера» позволяет переключаться между цехами. Группа «Настройки» позволяет переключаться между настройкой направляющих линий (Рис. 4.4) сервиса сдвига камеры, который может быть запущен при необходимости отслеживать смещение камеры. И между конфигуратором отображаемой информации в модуле визуализации.

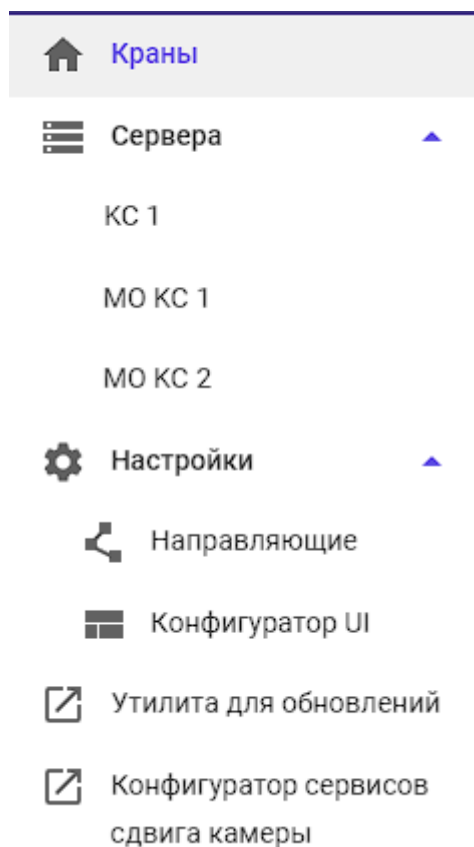


Рис. П4.3. Боковое меню модуля визуализации данных системы

Для настройки направляющих, пользователю необходимо выбрать сервер, кран, видеоканал крана. После этого, можно будет добавить по верх изображения линию. Линии, рисующиеся поверх изображения, могут иметь различную толщину и цвет. Они необходимы для того, чтобы можно было выделить некоторые конструктивные элементы на изображении, которые присутствуют на каждом кадре.

По линии можно будет понять, смещалась ли камера. Если линия не будет совпадать с конструктивным элементом, который ей выделяли, значит камера сместилась и требуется корректировка её положения.

В нижнем правом углу находятся две кнопки, которые позволяют загрузить перечень направляющих линий, либо выгрузить его.

Под изображением располагаются следующие элементы управления:

- обновить или задать периодичность обновления изображения;
- получить текущий кадр видеокамеры;

- переключиться на сохранённый опорный кадр;
- сохранить текущий кадр как опорный;
- загрузить изображение в качестве опорного кадра;
- скачать сохранённый опорный кадр.

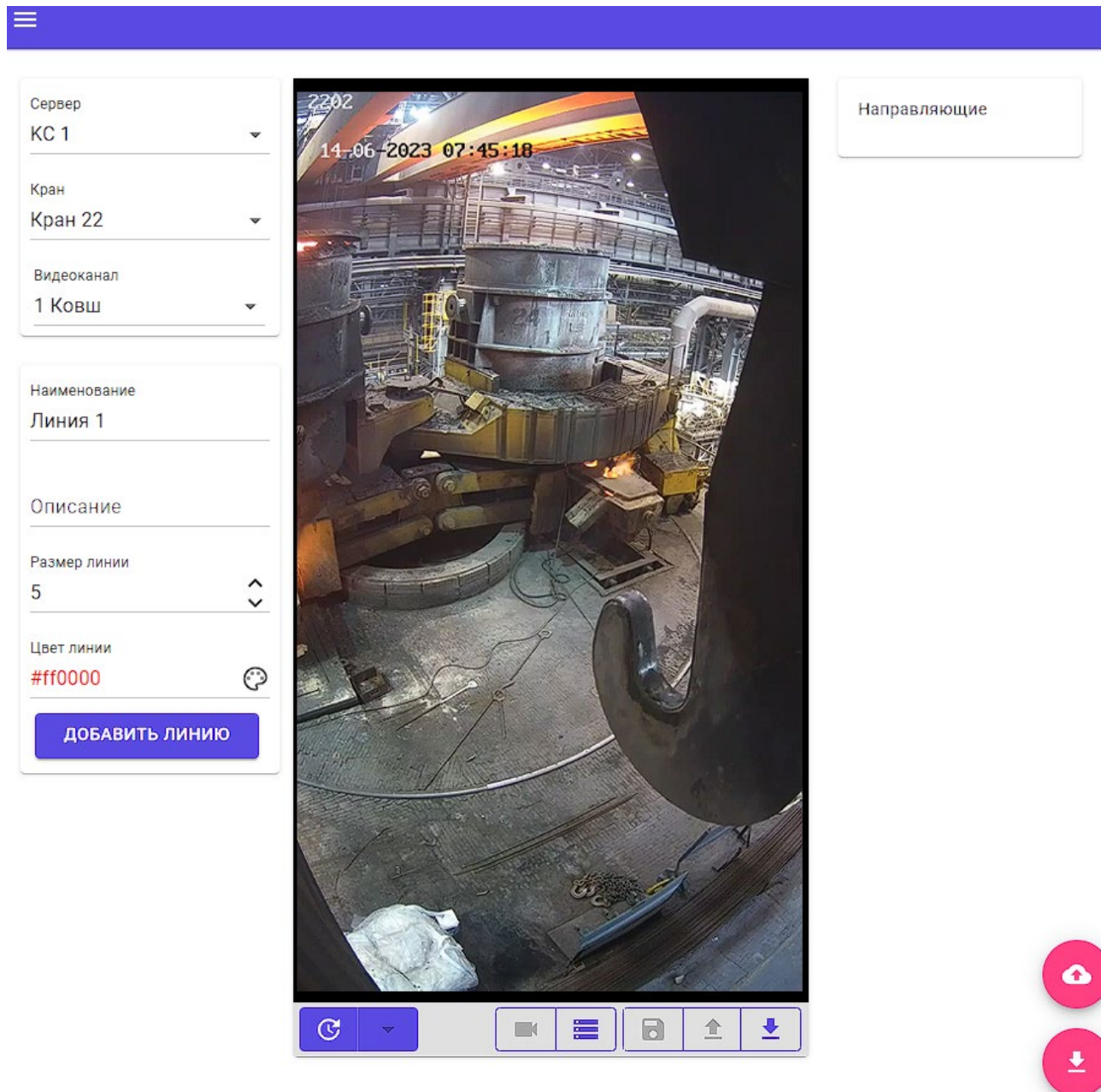


Рис. П4.4. Настройка направляющих линий

Конфигуратор отображаемой информации (рис. П4.5) позволяет настроить или загрузить список серверов, которые будут отображаться модулем визуализации. В дальнейшем настройки можно скачать.

У каждого цехового сервера можно настроить список отображаемых кранов.

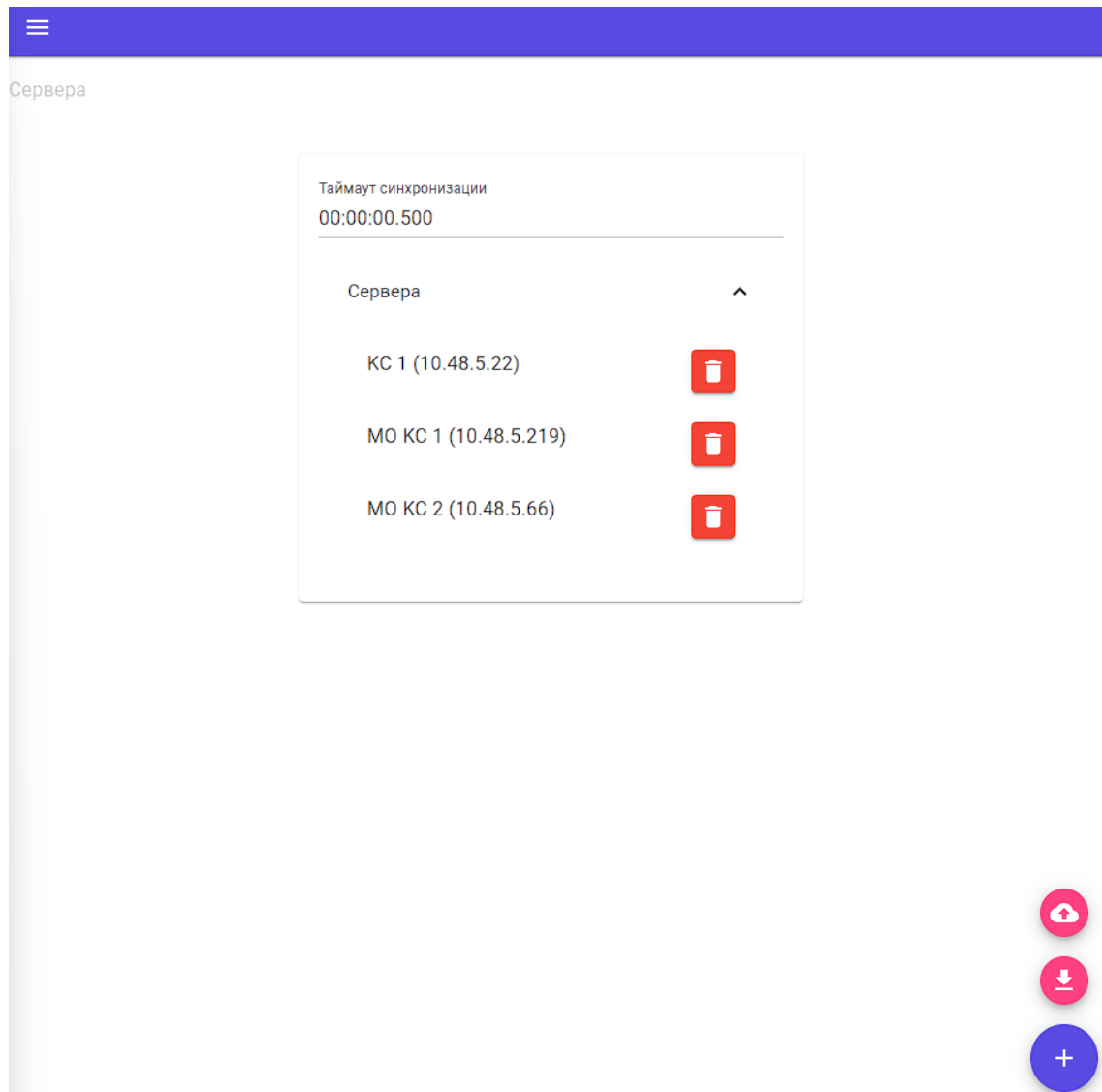


Рис. П4.5. Настройки отображаемой информации

## Пример конфигурации алгоритма позиционирования по маркерам

```

{
  "PositionProcessing": {
    "PositionOffset": 3,
    "Markers": [
      {
        "Code": "001",
        "Position": 104.3
      },
      {
        "Code": "002",
        "Position": 99.59
      },
      {
        "Code": "003",
        "Position": 86.9
      },
      {
        "Code": "004",
        "Position": 74.5
      },
      {
        "Code": "005",
        "Position": 63.7
      },
      {
        "Code": "006",
        "Position": 51.9
      },
      {
        "Code": "007",
        "Position": 34.2
      },
      {
        "Code": "008",
        "Position": 27.4
      },
      {
        "Code": "009",
        "Position": 14.47
      },
      {
        "Code": "010",
        "Position": 4.4
      }
    ]
  },
  "CraneDistanceMeterAlg": {
    "DistanceAlgConfigurations" : [
      {
        "RX": 0,
        "DX": -29
      },
      {
        "RX": 223,
        "DX": -22.8
      },
      {
        "RX": 283,
        "DX": -21
      },
      {
        "RX": 1008,
        "DX": -9
      },
      {
        "RX": 1807,
        "DX": 3
      },
      {
        "RX": 2430,
        "DX": 15
      },
      {
        "RX": 2507,
        "DX": 17.5
      },
      {
        "RX": 2560,
        "DX": 19
      },
      {
        "VirtualChannelId": 3,
        "RX": 0,
        "DX": -26.5
      },
      {
        "VirtualChannelId": 3,
        "RX": 392,
        "DX": -15
      },
      {
        "VirtualChannelId": 3,
        "RX": 645,
        "DX": -3.8
      },
      {
        "VirtualChannelId": 3,
        "RX": 890,
        "DX": 7.5
      },
      {
        "VirtualChannelId": 3,
        "RX": 1060,
        "DX": 15
      }
    ]
  }
}

```



## Пример конфигурации сервиса исполнения нейронных сетей

```

{
  "NetworkService": {
    "ZeroMQPort": 5556,
    "Networks": {
      "PosBySceneRecognizers": {
        "weights": {
          "OnnxPath": "D:\\Programs\\Bundles\\Crane-
v0.23.21RC1\\efficientnet-b1_crane-position.onnx",
          "IsBgrNetwork": false
        }
      },
      "CraneLadleDetectors": {
        "bucketDetector": {
          "OnnxPath": "D:\\Programs\\Bundles\\Crane-
v0.23.12RC2\\yolov4_crane.onnx",
          "IsBgrNetwork": false
        }
      },
      "CraneLadleClassifiers": {
        "efficientnet": {
          "OnnxPath": "D:\\Programs\\Bundles\\Crane-
v0.23.04RC2\\crane_class.onnx",
          "ClassesCount": 5
        }
      }
    },
    "Pipelines": {
      "CraneLadleRecognitionPipelines": {
        "pipeline1": {
          "FullZoneDetector": "bucketDetector",
          "CraneLadleClassifier": "efficientnet"
        }
      }
    }
  }
}

```

## Шаблон конфигурации Nginx

```

events {
    worker_connections 1024;
}

http {
    include mime.types;
    default_type application/octet-stream;
    access_log D:\Programs\Nginx\logs\access.log;
    {{range $tag, $services := services | byTag }}{{if eq $tag
"Crane"}}
        {{range $services}}
            {{ $service := service .Name -}}
            {{range $service -}}
                upstream {{ .Name }} {
                    keepalive 512;
                    {{range $service -}}
                        server {{ .Address }}:{{ .Port }} max_fails=5
fail_timeout=1m;
                    {{end -}}
                }
            {{end}}
        {{end}}
    {{end}}
    server {
        listen 80;
        proxy_next_upstream error timeout invalid_header http_500
http_503;
        {{range $tag, $services := services | byTag }}{{if eq $tag
"Crane"}}
            {{range $services}}
                {{ $service := service .Name -}}
                {{range service .Name -}}
                    {{ $iterInfo := .Name | regexReplaceAll ".*{1,3}-{1,3}-
.{1,10}-crane" "" }}
                    {{ $fromStr := $iterInfo | regexReplaceAll "to.*"
"" }}{{ $stepRawStr := $iterInfo | regexReplaceAll ".*step"
"" }}{{ $stepStr := $stepRawStr | regexReplaceAll "-.*"
"" }}{{ $indexStr := $iterInfo | regexReplaceAll ".*crane" "" }}
                    {{ $from := $fromStr | parseInt }}{{ $step := $stepStr |
parseInt }}{{ $index := $indexStr | parseInt }}{{ $id := add $from
(multiply $step $index) }}
                    #{{ $iterInfo }}
                    #crane{{ $id }}

                    location ~* ^/crane{{ $id }}/(.*) {
                        set $path /$1$is_args$args;
                        proxy_pass http://{{ .Address }}:{{ .Port }}$path;

```

## Пример рабочей нагрузки оркестратора Nomad

```

job "recognition-0-23-22RC1" {
  datacenters = ["dc1"]

  meta {
    logPass =
"aws_access_key_id=nomad&aws_access_key_secret=123@321"
    S3Addr = "10.11.11.12"
    bucket = "bucket-cranes"
    craneVersion = "0.23.21RC2"
    cfgVersion = "0.23.22RC1"
    eventsDetectorVersion = "0.23.21RC1"
    wrapperVersion = "0.23.20RC2"
    torchEnv = "TorchEnv"
    mixerLadlesApiPort = "5020"
    server = "MOKC1"
    lokiAddr = "http://10.11.11.12:3100"
  }

  update {
    max_parallel   = 3
    min_healthy_time = "15s"
    healthy_deadline = "9m"
    health_check    = "checks"
  }

  constraint {
    attribute = "${meta.location}"
    value = "MOKC1"
  }

  group "crane1to2step1" {
    count = 2
    task "prepareCfg" {
      lifecycle {
        hook = "prestart"
        sidecar = false
      }

      artifact {
        source =
"http://${NOMAD_META_S3Addr}/${NOMAD_META_bucket}/${NOMAD_META_cfgVersion}/cfg/Crane/${NOMAD_META_server}/configuration.json?${NOMAD_META_logPass}"
        destination = "${NOMAD_ALLOC_DIR}/configuration"
      }

      driver = "raw_exec"

      config {
        command = "powershell"
        args = ["echo", "done"]
      }
    }

    task "crane" {
      resources {
        memory = 2500
        cpu = 400
        network {
          port "servicePort" {}
        }
      }

      service {
        name = "${BASE}${NOMAD_ALLOC_INDEX}"
        tags = ["Monitoring", "Crane"]
        port = "servicePort"
        check {
          type = "http"
          port = "servicePort"
          interval = "15s"
          timeout = "2s"
          path = "/HealthCheck"
        }

        artifact {
          source =
"http://${NOMAD_META_S3Addr}/${NOMAD_META_bucket}/${NOMAD_META_craneVersion}/bin/metallurgical.crane.zip?${NOMAD_META_logPass}"
        }
      }
    }
  }
}

```

Рис. П8.1. Пример рабочей нагрузки Nomad

```
template {  
    source =  
"${NOMAD_ALLOC_DIR}/configuration/configuration.json"  
    destination =  
"${NOMAD_TASK_DIR}/configuration.json"  
}  
  
driver = "raw_exec"  
  
config {  
    command =  
"${NOMAD_TASK_DIR}/metallurgical.crane"  
    args = [  
        "configName=Crane_${NOMAD_ALLOC_INDEX}",  
        "craneNumberStart=1",  
        "craneNumberStep=1",  
        "craneIndex=${NOMAD_ALLOC_INDEX}",  
        "lokiAddress=${NOMAD_META_lokiAddr}"  
    ]  
}  
}  
}
```

## Акт внедрения



ОБЩЕСТВО С ОГРАНИЧЕННОЙ ОТВЕТСТВЕННОСТЬЮ  
«Малленом Системс»

ул. Metallургов, 216, г. Череповец, Вологодская обл., 162610  
тел.: (8202) 20-16-35, факс: (8202) 20-16-34, e-mail: info@mallenom.ru,  
http://www.mallemom.ru  
ОКПО 90443215, ОГРН 1113528001685, ИНН/КПП 3528176030/352801001

## АКТ ВНЕДРЕНИЯ

Данным актом подтверждается, что результаты выпускной квалификационной работы «Программное обеспечение системы контроля перемещения объектов мостовыми кранами» выполненной Адамовым Даниилом Сергеевичем, выпускником по направлению подготовки 09.04.04 «Программная инженерия (Разработка программно-информационных систем)» в федеральном государственном бюджетном образовательном учреждении высшего образования «Череповецкий государственный университет» под руководством Ершова Евгения Валентиновича внедрены в ООО «Малленом Системс».

Дата внедрения: 28 марта 2023 г.

Заявленные характеристики системы предполагали наличие следующих основных возможностей:

- определение местоположения сталь-ковша на крановом пути;
- определения состояния сталь-ковша в захвате мостового крана.

В ходе эксплуатации программы подтверждено, что она обладает всеми заявленными возможностями.

Технический директор компании  
ООО «Малленом Системс»

05.06.2023 г.

Е.Н. Веснин



Рис. П9.1. Акт внедрения